Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

Université Centrale

École centrale polytechnique privée de Tunis

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en système d'information

Spécialité : Système d'Information

Par

Noureddine RMILA

Conception et mise en place d'une solution de diffusion pour By-Systems

Encadrant professionnel :	**Mr Youssef Boujraf**	CEO By-SYstems
Encadrant académique :	**Mr Mohamed Naceur Boubakri**	Enseignant

Réalisé au sein de By-Systems

Année Universitaire 2018 - 2019

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Mr Youssef Boujraf**

Signature et cachet

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Mr Mohamed Naceur Boubakri**

Signature

Dedicaces

J'ai le grand plaisir de dédier ce travail en témoignage d'affection et de reconnaissance

À tous ceux qui me sont chers et particulièrement :

À mes Chers parents Houchine et Halima,

Pour leur tendresse, gentillesse, sacrifices et leur soutien moral durant ma vie,

À mes frères,

Avec tous mes respects et ma gratitude, pour leur sentiment de responsabilité et leurs sincères encouragements.

À ma belle-soeur Malika,

À mon neveu Bayrem,

Et finalement à toute ma famille et à tous mes amis spécialement Jamila Hassine,
Et à tous ceux qui m'aiment et veillent à mon bonheur

N.Rmila

Remerciements

Au terme de ce projet, je tiens à exprimer mes vifs remerciements et ma sincère gratitude à monsieur Mohamed Naceur BOUBAKRI, pour son encadrement, son assistance, son sou- tien et son encouragement continu, et pour les nombreuses discussions qu'on a eu ensemble. Ses compétences Scientifiques et ses qualités humaines m'ont permis de mener ce travail dans de bonnes conditions.

Ma profonde reconnaissance s'adresse également à monsieur Youssef BOUJRAF, mon encadrant au sein de Be-System Je le remercie de m'avoir bien accueilli au sein de sa société à Brussels et de m'avoir fait partagé ses expériences et ses réflexions.

Ce séjour était pour moi une expérience très agréable aussi bien sur le plan scientifique que culturel. Je remercie, également, mes amis ainsi que mes collègues dans la société pour leurs conseils et leurs bonnes humeurs.

Je remercie également les membres du jury pour avoir accepté de juger ce modeste travail.

Finalement, je ne manque pas d'exprimer ma profonde reconnaissance envers mes enseignants et tous ceux qui m'ont aidé, de près ou de loin, dans l'élaboration de mon travail.

Table des matières

Introduction générale **1**

1 Présentation générale **3**

1.1 Présentation de l'organisme d'accueil 4

1.2 Analyse de l'existant 4

1.3 Présentation du sujet 4

1.4 Méthodologie adoptée 5

1.4.1 Approches classiques Vs Approches agiles 5

1.4.2 Choix de la méthodologie de travail 7

1.4.3 Présentation de la méthode Scrum 8

1.4.4 Répartition des rôles 9

1.4.5 Les événements Scrum 9

1.5 Architecture logicielle 10

1.5.1 Architecture MVC 10

1.5.2 Architecture Microservices 11

1.6 Technologies 13

1.6.1 CEAN STACK 14

1.6.2 Logiciels et outils utilisés pour le développement 14

1.7 Chronogramme 16

2 Analyse et spécification des besoins **17**

2.1 Identification des besoins 18

2.1.1 Besoins fonctionnels 18

2.1.2 Besoins non fonctionnels 19

2.2 Identification des acteurs 19

2.3 Diagramme des cas d'utilisation global 20

2.4 Pilotage de projet avec scrum 21

2.4.1 Equipe et rôles 21

2.4.2 Le backlog produit 22

3 Sprint 1 : Mise en place des diverses fonctionnalités **24**

3.1 Backlog du sprint 25

3.2 Analyse . . . 25
3.3 Conception . . . 27
3.3.1 Raffinement des cas d'utilisation . . . 27
3.3.2 Diagrammes de séquence . . . 28
3.3.3 Diagramme de classe . . . 30
3.4 Réalisation . . . 31
3.5 Conclusion . . . 36

4 Sprint 2 : RSS FEED Consumer 37
4.1 Backlog du sprint . . . 38
4.2 Analyse . . . 38
4.3 Conception . . . 39
4.3.1 Raffinement des cas d'utilisation . . . 39
4.3.2 Diagrammes de séquence . . . 47
4.3.3 Diagramme de classe . . . 49
4.4 Réalisation . . . 50
4.5 Conclusion . . . 54

5 Sprint 3 : Facebook Live Streamer 55
5.1 Backlog du sprint . . . 56
5.2 Analyse . . . 56
5.3 Conception . . . 58
5.3.1 Raffinement des cas d'utilisation . . . 58
5.3.2 Diagrammes de séquence . . . 59
5.3.3 Diagramme de classe . . . 62
5.4 Réalisation . . . 63
5.5 Conclusion . . . 65

6 Sprint 4 : Contrôle de caméra et gestion de modèle des données 67
6.1 Backlog du sprint . . . 68
6.2 Analyse . . . 68
6.3 Conception . . . 69
6.3.1 Raffinement des cas d'utilisation . . . 69
6.3.2 Diagrammes de séquence . . . 71
6.3.3 Diagramme de classe . . . 73

6.4 Réalisation 74
6.5 Conclusion 79

7 Déploiement **80**

7.1 Virtualisation ou containérisation 81
7.1.1 Virtualisation 81
7.1.2 Containérisation 81
7.1.3 Etude comparative 81
7.2 L'outil Docker 83

Conclusion générale **85**

Table des figures

1.1 Gestion de projet SCRUM 8
1.2 Architecture MVC 11
1.3 Architecture microservices 12
1.4 Diagramme de Gantt réel 16

2.1 Diagramme de cas d'utilisation global 20

3.1 Maquette de l'interface d'authentification 25
3.2 Maquette de l'interface d'accueil de l'administrateur 26
3.3 Maquette gérer rôles 26
3.4 Maquette gérer utilisateurs 27
3.5 Diagramme de cas d'utilisation « S'authentifier » 27
3.6 Diagramme de séquence Ajouter Utilisateur 28
3.7 Diagramme de séquence Modifier Utilisateur 29
3.8 Diagramme de séquence Ajouter Rôle 29
3.9 Diagramme de séquence Supprimer Rôle 30
3.10 diagramme de classe de sprint 1 31
3.11 Interface de connexion 32
3.12 Interface d'accueil de l'administrateur 33
3.13 Interface Changement de Langue 33
3.14 Interface Changement de thème 34
3.15 Interface de la gestion des utilisateurs 35
3.16 Interface de l'ajout d'un utilisateur 35
3.17 Interface de la modification d'un utilisateur 36
3.18 Interface de la mise a jours de profile 36

4.1 Maquette de gestions des sources de médias 38
4.2 Maquette de flow de média global 39
4.3 Diagramme de cas Gérer flux 40
4.4 Diagramme de cas gestion des flux 42
4.5 Diagramme de cas d'utilisation Gérer Raw Data 43
4.6 Diagramme de cas Gérer la modération des feed-items 45

4.7 Diagramme de séquence global de Médias Feeds 47
4.8 Diagramme de séquence « Ajouter un nouveau feed » 47
4.9 Diagramme de séquence « Mise à jours de feed » 48
4.10 Diagramme de séquence « Supprimer feed » 48
4.11 Diagramme de séquence « Démarrer feed » 49
4.12 diagramme de classe de sprint 2 50
4.13 Interface Média 51
4.14 Interface détails de feed-item 52
4.15 Interface éditer feed-item 52
4.16 Interface Modérer feed-item 53
4.17 Interface gérer les catégories 54

5.1 Maquette de gestion des comptes Facebook. 56
5.2 Maquette de la diffusion en direct. 57
5.3 Maquette de la planification de diffusion en direct. 57
5.4 Diagramme de cas Gérer live stream 58
5.5 Diagramme de séquence gestion des comptes Facebook 60
5.6 Diagramme de séquence Apporter liste des pages Facebook 60
5.7 Diagramme de séquence Démarrer la diffusion en direct 61
5.8 Diagramme de séquence Arrêter la diffusion en direct 61
5.9 Diagramme de séquence Supprimer la diffusion en direct 62
5.10 diagramme de classe de sprint 3 62
5.11 Interface gérer les compte social média 63
5.12 Interface diffusion en direct 64
5.13 Interface planification de diffusion en direct 64
5.14 Interface d'ajout des commentaire sur la vidéo de diffusion en direct 65
5.15 Interface gérer les destination de la diffusion en direct 65

6.1 Maquette de contrôle de la caméra PTZ. 68
6.2 Maquette de l'intégration de contrôle dans la diffusions en direct 69
6.3 Diagramme de cas Gérer Caméra 70
6.4 Diagramme de séquence Déplacer la caméra 71
6.5 Diagramme de séquence Zoomer la caméra 72
6.6 Diagramme de séquence Alimenter la caméra 73
6.7 diagramme de classe de sprint 4 74

6.8 Interface la contrôle de la caméra 75
6.9 Interface le contrôle de la caméra intégré dans la diffusion en direct 75
6.10 Interface gérer les caméras 76
6.11 Interface gérer le modelé des données 76
6.12 Interface gérer les vues clients 77
6.13 Interface gérer les vues clients 77
6.14 Interface ajouter noeud 78
6.15 Interface gérer les flows 78

7.1 Docker VS Machine Virtuelle 82
7.2 Performances : Docker VS KVM 83
7.3 Architecture de déploiement Docker 84

Liste des tableaux

1.1 Tableau comparatif des deux approches classique et agile 5
1.2 Différences entre XP et Scrum 7

3.1 Tableau d'analyse du cas d'utilisation « s'authentifier » 28

4.1 Tableau d'analyse du cas d'utilisation « Ajouter flux » 40
4.2 Tableau d'analyse du cas d'utilisation « Supprimer flux » 41
4.3 Tableau d'analyse du cas d'utilisation « Modifier flux » 42
4.4 Tableau d'analyse du cas d'utilisation « Gérer flux » 43
4.5 Tableau d'analyse du cas d'utilisation « Consulter un feed-item » 44
4.6 Tableau d'analyse du cas d'utilisation « Supprimer feed-item » 44
4.7 Tableau d'analyse du cas d'utilisation « Déplacer feed-item » 44
4.8 Tableau d'analyse du cas d'utilisation « Déplacer feed-item vers on air» 45
4.9 Tableau d'analyse du cas d'utilisation « Modifier feed-item » 46
4.10 Tableau d'analyse du cas d'utilisation « Supprimer feed-item » 46

5.1 Tableau d'analyse du cas d'utilisation « Démarrer la diffusion en direct » 59

6.1 Tableau d'analyse du cas d'utilisation « Ajouter preset » 71

Liste des abréviations

- **BD** = **B**ase des **D**onnées
- **DHS** = **D**ata **H**ub **S**erver
- **FR24** = **FR**ance **24**
- **GLSI** = **G**énie **L**ogiciel et **S**ystèmes d'**I**nformation
- **HTML** = **H**yper**T**ext **M**arkup **L**anguage
- **HTTP** = **H**yper**T**ext **T**ransfer **P**rotocol
- **HTTPS** = **H**ypertext **T**ransfer **P**rotocol **S**ecure
- **ID** = **ID**entifiant
- **JS** = **J**ava**S**cript
- **JSON** = **J**ava**S**cript **O**bject **N**otation
- **MVC** = **M**odel **V**iew **C**ontroller
- **PTZ** = **P**an **T**ilt **Z**oom
- **REST** = **RE**presentational **S**tate **T**ransfer
- **RFI** = **R**adio **F**rance **I**nternationale
- **RTBF** = **R**adio **T**élévision **B**elge **F**rancophone
- **UML** = **U**nified **M**odeling **L**anguage

Introduction générale

L'un des défis auxquels se confronte une entreprise moderne est la nécessité de maintenir une vue d'ensemble sur les nombreux changements dans son système d'information, quelque soit leur nature. Ceci est l'un des piliers garantissant sa compétitivité et sa pérennité.

Le secteur de la diffusion fait face à un labyrinthe de défis alors que l'ensemble du secteur des médias devient de plus en plus concurrentiel à l'ère numérique.

Le paradigme existant de différents produits basés sur des «appareils» provenant de nombreux fabricants crée des couches de complexité qui sont coûteuses à l'achat, difficiles à entretenir, difficiles à intégrer et qui ont besoin de personnel spécialisé.

Dans cette perspective, et dans le cadre de notre projet de fin d'études, réalisé au sein de By-Systems, nous allons nous intéresser au développement d'une solution qui utilise les dernières technologies Web et les réutilise dans un environnement de diffusion.

Ce rapport est articulé autour de quatre chapitres :

Le premier chapitre comporte une brève présentation de l'organisme d'accueil By-Systems et du cadre général de ce projet. Il expose en effet, l'étude de l'existant et met l'accent sur la solution proposée. Et il aborde à la fin la méthodologie appliquée pour assurer le bon déroulement de notre travail.

Le deuxième chapitre présente en premier lieu une analyse détaillée des besoins fonctionnels et non fonctionnels de notre application et aborde l'analyse de ces besoins en se basant sur le langage de modélisation UML. En second lieu, il décrit les différents cas d'utilisation.

Le troisième chapitre présente tout d'abord le backlog du produit de notre système. Aussi il détaille l'approche conceptuelle adoptée pour mettre en place notre application. Il détaille aussi les besoins fonctionnels et non fonctionnels et aborde l'analyse de ces besoins en se basant sur le langage de modélisation UML.

Dans le quatrième chapitre, il y a une présentation de la phase de réalisation de notre application. Cette étape est basée sur la méthodologie AGILE SCRUM dans le but d'une bonne gestion de projet.

Nous clôturons par une conclusion générale qui présente une analyse du travail réalisé au sein de By-Systems. Cette partie met en évidence non seulement un résumé du travail effectué tout au long de la période du stage de projet de fin d'études mais aussi des propositions des diverses perspectives.

Chapitre 1

Présentation générale

Plan

1 Présentation de l'organisme d'accueil 4

2 Analyse de l'existant 4

3 Présentation du sujet 4

4 Méthodologie adoptée 5

5 Architecture logicielle 10

6 Technologies 13

7 Chronogramme 16

Introduction

Nous allons introduire dans ce premier chapitre le cadre de projet, à savoir l'entreprise d'accueil, par la suite nous passerons à l'étude de l'existant puis la présentation du sujet, et pour finir, nous parlerons de la méthodologie du développement à suivre dans notre projet.

1.1 Présentation de l'organisme d'accueil

BY-Systems est un intégrateur mondial de solutions multi-fournisseurs pour les productions TV, Radio et Press. Les services comprennent la gestion des projets, l'architecture du système, l'implémentation de flux de travail, le développement des logiciels et de matériel informatique et l'organisation de formations.

BY-Systems intègre des systèmes non linéaires d'édition, des graphiques de diffusion en temps réel, des solutions de serveur vidéo, des systèmes de gestion de réseau, des systèmes de contrôle de diffusion incluant les informations, le sport, la télé-réalité, les élections, les prévisions météorologiques et les solutions RadioTV.

Les solutions de BY-Systems rationalisent le flux de production, améliorent l'expérience du téléspectateur et augmentent la valeur de la production en étendant les investissements existants.

Spécialisations : Intégrateur de systèmes, Intégration avec des systèmes automatisés, gestion informatique de la diffusion, systèmes de salle de presse, systèmes de gestion de réseau, contrôles de routage, contrôles d'automatisation, gestion de projet, développement de logiciels, PMP, PRINCE2, centre d'exploitation de réseau et Proof of Concept.

Dans le monde entier, les grands diffuseurs adoptent les solutions de BY-Systems : (MOYEN-ORIENT), Al Jazeera, Télévision Dubaï, Radio Dubaï, Télévision Oman, Radio Oman, ... (EUROPE) RFI, FR24, ARTE, M6, RTL RADIO, FUN RADIO, FRANCE TV, BETV , RTBF, YLE, ... (USA-CANADA) RDS, CNN, ... (AFRIQUE) SABC.

1.2 Analyse de l'existant

1.3 Présentation du sujet

Le concept de notre application web cross plate-forme et compatible avec les écrans tactile. Elle consiste à développer une solution de diffusion multiplateforme qui réduit les flux de travail complexes, coûteux et inefficaces.

En effet notre solution permet la communication bidirectionnelle à partir de périphériques dotés de ressources matérielles variées vers des clients connectés en utilisant des protocoles et des supports

variés. Aussi la publication de contenu sur les médias sociaux de manière rapide et moins complexe. Pour suivre et s'adapter à l'évolution des modes de consommation des médias. Notre application permet d'échange les donner avec un minimum de consommation de bande passante. Aussi les mises ajours sont transparent et n'impacte l'expérience de 'utilisateur a l'aide de notre architecture microservice.

1.4 Méthodologie adoptée

La méthodologie est une démarche organisée rationnellement pour aboutir à un résultat déterminé. Ainsi, la méthodologie correspond à l'ensemble des processus qui permettent d'organiser la conception et le développement du projet dans le but de parvenir au résultat le plus performant et ce dans un climat d'efficacité et de convivialité.

On distingue deux familles de méthodes :

— s Les méthodes classiques : se caractérisent par recueillir les besoins, définir le produit, le développer et le tester avant de le livrer. Il s'agit ici de prévoir des phases séquentielles où il faut valider l'étape précédente pour passer à la suivante.

— Les méthodes agiles : utilisent un principe de développement itératif qui consiste à découper le projet en plusieurs étapes qu'on appelle "itérations".

1.4.1 Approches classiques Vs Approches agiles

Le tableau ci-dessous contient un comparatif entre les deux approches classique et agile :

Tableau 1.1: Tableau comparatif des deux approches classique et agile

Thème	**Approche Classique**	**Approche Agile**
Cycle de vie	En cascade ou en V, sans rétroaction possible, phases séquentielles.	Itératif et incrémental
Planification	Prédictive, caractérisée par des plans plus ou moins détaillés sur la base d'un périmètre et d'exigences définies et stables au début du projet.	Adaptative avec plusieurs niveaux de planification (macro- et microplanification) avec ajustements si nécessaires au fil de l'eau en fonction des changements survenus.

Documentation	Produite en quantité importante comme support de communication, de validation et de contractualisation.	Réduite au strict nécessaire au profit d'incréments fonctionnels opérationnels pour obtenir le feedback du client.
Equipe	Une équipe avec des ressources spécialisées, dirigées par un chef de projet.	Une équipe responsabilisée où l'initiative et la communication sont privilégiées, soutenue par le chef de projet.
Qualité	Contrôle qualité à la fin du cycle de développement. Le client découvre le produit fini.	Un contrôle qualité précoce et permanent, au niveau du produit et du processus. Le client visualise les résultats tôt et fréquemment.
Changement	Résistance voire opposition au changement. Processus lourds de gestion des changements acceptés.	Accueil favorable au changement inéluctable, intégré dans le processus.
Suivi de l'avancement	Mesure de la conformité aux plans initiaux. Analyse des écarts.	Un seul indicateur d'avancement : le nombre de fonctionnalités implémentées et le travail restant à faire.
Gestion des risques	Processus distinct, rigoureux, de gestion des risques.	Gestion des risques intégrée dans le processus global, avec responsabilisation de chacun dans l'identification et la résolution des risques. Pilotage par les risques.
Mesure de succès	Respect des engagements initiaux en termes de coûts, de budget et de niveau de qualité.	Satisfaction de client par la livraison de valeur ajoutée Pilotage par les risques.

L'étude comparative réalisée sur les deux types de méthodologies de conception résumée dans le tableau nous permet de tirer les conclusions suivantes :

— Les méthodes agiles seront plus utilisées pour les gros projets car elles offrent une meilleure adaptabilité, visibilité et gestion des risques. Elles pourraient tout aussi bien être utilisées pour les projets où il n'y pas de documentations détaillées, le client peut alors voir l'évolution du projet et l'adapter selon ses besoins.

— En revanche, les méthodes classiques seront plus utilisées si on possède une idée bien précise sur le projet avec un cahier des charges et planning très détaillé où on peut anticiper tous les risques possibles.

1.4.2 Choix de la méthodologie de travail

Suite à l'étude et à la comparaison des deux approches et à fin de contrôler les risques et de mener à bon terme notre projet vu sa complexité, nous optons pour l'utilisation d'une méthode agile. Cependant il existe plusieurs méthodes agiles qui se différencient, les unes des autres dont les plus utilisées sont Scrum et Extreme Programming (XP). Les différences entre ces deux dernières sont souvent très subtiles, le tableau ci-dessous souligne ces principales dissimilitudes.

Tableau 1.2: Différences entre XP et Scrum

Méthode XP	**Méthode Scrum**
— Durée de l'itération (1 à 2 semaines) — Possibilités de changements des scénarios ou des fonctionnalités à implémenter au cours de l'itération — Différents rôles sont attribués aux membres de l'équipe XP (programmeur, testeur, coach, manageur, etc.) — Pratiques managériales, pratiques d'ingénierie et outils de collaboration et de support au management	— Durée de l'itération (2 à 4 semaines) — Les changements des fonctionnalités durant l'itération ne sont pas acceptés — Les membres de l'équipe sont pluridisciplinaires et seuls trois rôles sont définis (scrum-master, product-owner et l'équipe) — Pratiques et outils de collaboration et de support au management

En revanche, ces deux méthodes valorisent la transparence, l'inspection et l'adaptabilité des projets informatiques. Elles privilégient le travail d'équipe, les feedback continus et les ajustements rapides aux changements. Elles concernent particulièrement les équipes de taille réduite (4 à 10 personnes) où la communication et la coordination, entre les membres d'un projet, sont faciles à réaliser. Le client se trouve au cœur de ces deux approches. Il participe à la définition et à la priorisation des fonctionnalités.

Pour bien conduire notre projet et nous assurer du bon déroulement des différentes phases, nous avons opté pour Scrum comme méthodologie de conception et de développement.

1.4.3 Présentation de la méthode Scrum

Scrum signifie mêlée au rugby. Scrum utilise les valeurs et l'esprit du rugby et les adaptent aux projets de développement. Scrum est issu des travaux de deux des signataires du Manifeste Agile, Ken Schwaber et Jeff Sutherland, au début des années 1990.Il appartient à la famille des méthodologies itératives et incrémentales et repose sur les principes et les valeurs agiles. Le plus souvent, les experts de Scrum, même ses fondateurs, le décrivent comme un cadre ou un patron de processus orienté gestion de projet et qui peut incorporer différentes méthodes ou pratiques d'ingénierie. S'il est difficile de définir la nature de Scrum, sa mise en place est beaucoup plus simple et peut être résumée par la Figure 1.1

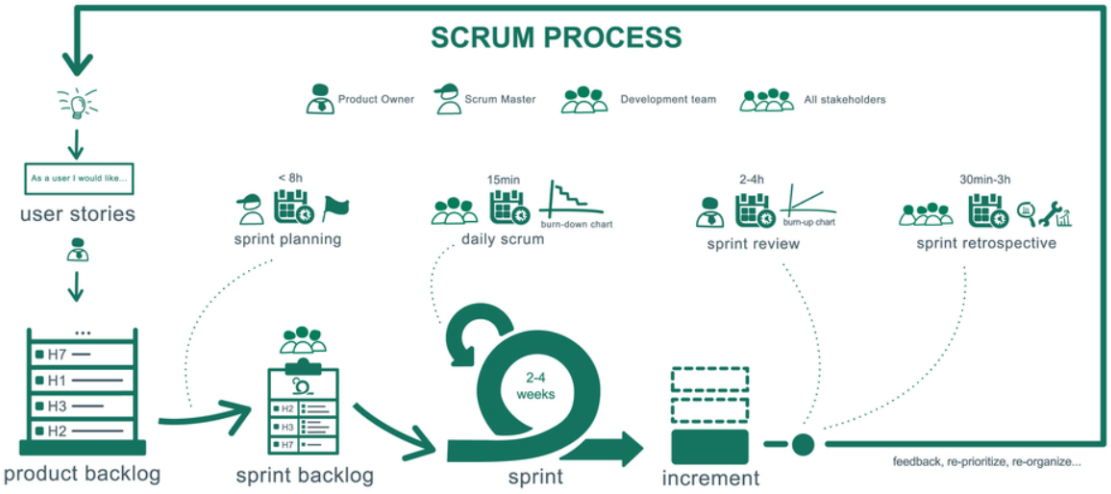


Figure 1.1: Gestion de projet SCRUM

En effet, comme le représente ce schéma, avec SCRUM le projet est décomposé en incréments qui traitent les fonctionnalités relevées dans le Backlog une à une d'une manière itérative jusqu'à l'aboutissement du produit final. Grâce à la mêlée quotidienne nous pouvons évaluer le progrès du travail accompli et nous foclisons sur les tâches qui restent à accomplir.

1.4.4 Répartition des rôles

L'Équipe Scrum comprend un propriétaire de produit (Product Owner), une Équipe de Développement (Development Team) et un Scrum Master. Les Équipes Scrum (Scrum Teams) sont auto-organisées et pluridisciplinaires.

1.4.4.1 Le Product Owner

Le Product Owner est la seule personne responsable de gérer le carnet de produit (Product Backlog). La gestion du Product Backlog comprend :

— Exprimer clairement les items du Product Backlog

— Ordonner les items du Product Backlog pour mieux réaliser les objectifs et missions Optimiser la valeur du travail eectué par l'Équipe de Développement

— S'assurer que le Product Backlog est visible, transparent, et clair pour tous, et qu'il montre ce sur quoi l'Équipe de Développement travaillera prochainement.

— S'assurer que l'Équipe de Développement comprend adéquatement les items du Product Backlog.

1.4.4.2 L'Équipe de Développement

L'Équipe de Développement est constituée de professionnels qui livrent à chaque Sprint un incrément "terminé" et potentiellement livrable du produit. Seuls les membres de l'Équipe de Développement créent l'incrément.

1.4.4.3 Le Scrum Master

Le Scrum Master est responsable de s'assurer que Scrum est compris et mis en ouvre. Les Scrum Masters remplissent leur rôle en s'assurant que l'Équipe Scrum adhère à la théorie, aux pratiques et aux règles de Scrum.

1.4.5 Les événements Scrum

1.4.5.1 Le Sprint

Au coeur de Scrum, le Sprint a une durée d'un mois ou moins au cours duquel une version "terminée ", utilisable et potentiellement livrable du logiciel est créée. Il est préférable que les Sprints gardent une durée constante tout au long de l'initiative de développement. Un nouveau Sprint débute immédiatement après la fin de la précédente.

1.4.5.2 Mêlée quotidienne

La mêlée quotidienne (Daily Scrum) est un événement limité à 15 minutes au cours duquel l'Équipe de Développement synchronise ses activités et crée un plan pour les prochaines 24 heures. Pour ce faire, l'équipe inspecte le travail effectué depuis la dernière mêlée quotidienne et envisage le travail qui peut être réalisé d'ici à la prochaine.

1.4.5.3 Revue du Sprint

Une revue de Sprint est tenue à la fin du Sprint pour inspecter l'incrément réalisé et adapter le Product Backlog si nécessaire. Pendant la réunion de revue de Sprint, l'Équipe Scrum et les parties prenantes échangent sur ce qui a été fait durant le Sprint. En se basant là-dessus, et en considérant les changements au Product Backlog effectués durant le Sprint, les participants collaborent pour déterminer les prochains items ayant le plus de valeur qui pourraient être faits. Cette réunion se veut informelle, pas une réunion de pilotage, et la présentation de l'incrément est destinée à susciter des réactions et à favoriser la collaboration.

1.5 Architecture logicielle

Les choix architecturaux d'une application sont décisifs dès lors qu'ils interviennent sur les performances, l'évolutivité, le temps de développement, et bien sûr le coût. Aujourd'hui nous parlons d'une séparation des applications en différentes couches, et nous parlons alors d'applications multi-niveaux. Nous avons choisi les architecture MVC et microservices pour notre projet.

1.5.1 Architecture MVC

Elle peut être décrite par la figure suivante. L'architecture MVC cherche à séparer nettement les couches de présentation, métier et d'accès aux données. Le but étant d'avoir une dépendance minimale entre les différentes couches de l'application, ainsi les modifications effectuées sur n'importe quelle couche de l'application n'affectent pas les autres couches.

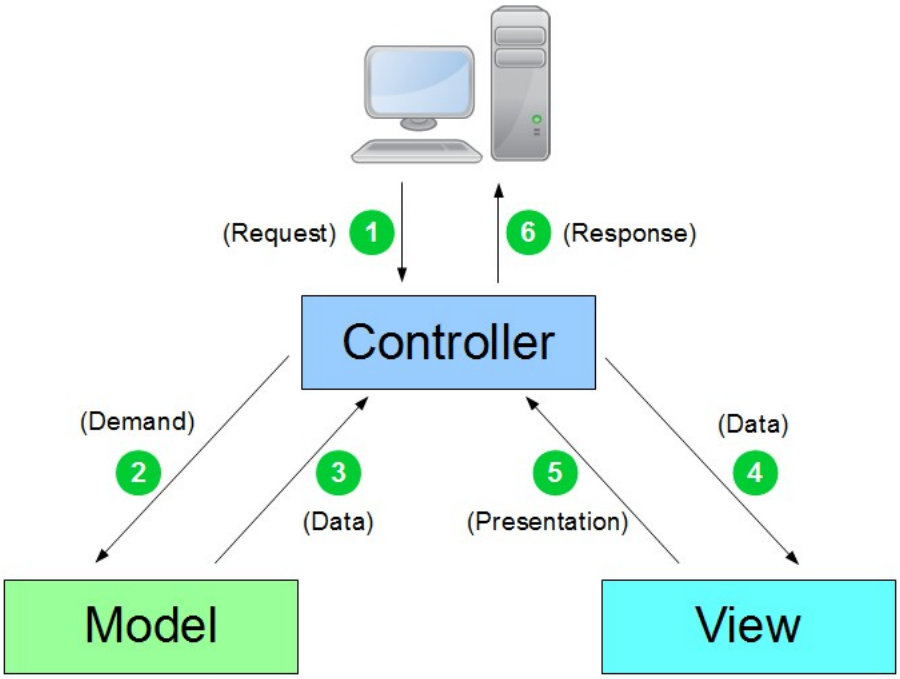


Figure 1.2: Architecture MVC

1. Modèle : cette partie gère les données de votre site. Son rôle est d'aller récupérer les informations " brutes " dans la base de données, de les organiser et de les assembler pour qu'elles puissent ensuite être traitées par le contrôleur.
2. View : cette partie se concentre sur l'affichage. Elle ne fait presque aucun calcul et se contente que de récupérer des variables pour savoir ce qu'elle doit afficher.
3. Contrôleur : cette partie gère la logique du code qui prend des décisions. C'est en quelque sorte l'intermédiaire entre le modèle et la vue : le contrôleur va demander au modèle les données, les analyser, prendre des décisions et renvoyer le texte à afficher à la vue.

1.5.2 Architecture Microservices

Le terme "Microservice" a connu une émergence au cours des dernières années pour décrire un style d'architecture bien particulier. Cette approche consiste à développer une seule application en un ensemble de petits services, isolés, autonomes et indépendamment déployés. Ces services peuvent communiquer ensemble afin de fournir les fonctionnalités nécessaires. Les microservices sont, dans la plus part du temps, implémentés et pilotés par des équipes de petite taille avec suffisamment d'autonomie. Chacune peut changer l'impléméntation de chaque microservice, ajouter ou supprimer des fonctionnalités de ce service avec un impact minimal sur les autres microservices. Ce style d'architecture présente plusieurs avantages comme l'hétérogénéité technologique, la résistance contre l'échec, la scalabilité sur mesure, la facilité de déploiement, l'alignement organisationnel, la réutilisabilité.

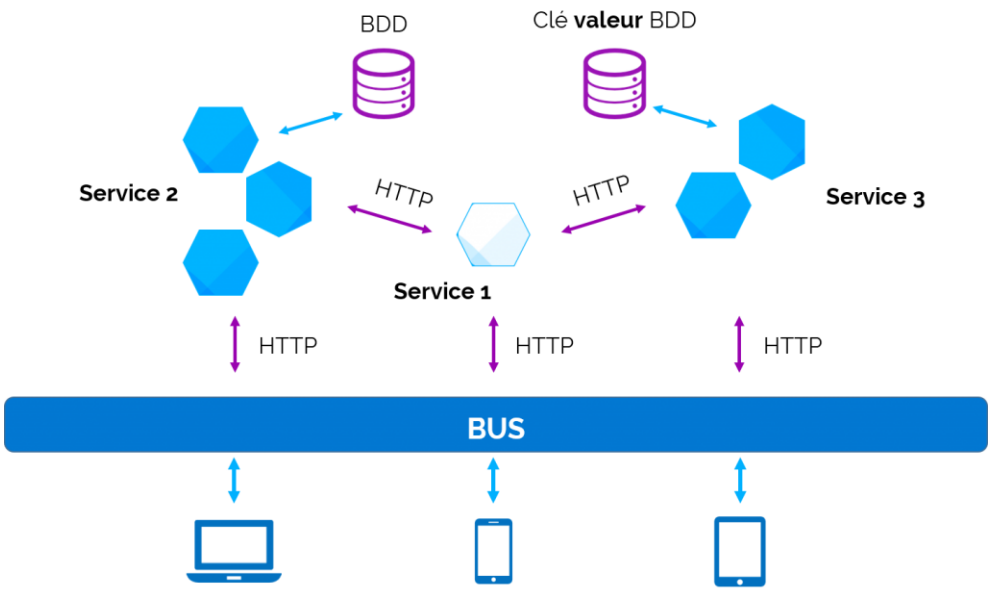


Figure 1.3: Architecture microservices

Caractéristiques de l'architecture microservices

D'après Martin FOWLER, l'architecture microservices possède neuf principales caractéristiques qu'il est essentiel d'appliquer durant la conception et le développement d'une application en microservices.

— **La division en composants via les services :** Cette caractéristique est héritée de l'architecture à base de composants. Les microservices sont indépendamment développés, testés et déployés. Un changement dans un service ne nécessite que son déploiement et n'affecte pas l'intégrité du système. Les services permettent d'éviter le couplage fort entre les différents composants en utilisant des mécanismes d'appel distants explicites.

— **L'organisation autour des capacités métiers :** La division d'une application en microservices est très différente de la décomposition classique qui est souvent basée sur les couches techniques. Chaque microservice est autonome vis à vis de la fonctionnalité qu'il réalise puisqu'il possède son propre code, sa propre interface et gère ses propres données.

— **Un produit, pas un projet :** Le but de l'utilisation des microservices est de livrer rapidement un morceau de logiciel qui est alors considéré comme terminé. Dans la vision microservices, une équipe est responsable d'un produit durant tout son cycle de vie. Elle est entièrement responsable du logiciel en production.

— **Une gouvernance décentralisée :** En effet, il est difficile de trouver une seule technologie permettant de résoudre tous les problèmes d'une façon efficace. D'où, il est préférable d'utiliser le bon outil au bon moment. Avec l'architectures microservices, nous pouvons utiliser pour

chaque service le langage d'implémentation et la plateforme technologique les plus adéquats pour accomplir le besoin.

— **Gestion de données décentralisée :** L'architecture en microservices admet l'utilisation de plusieurs bases de données. Dans le cadre d'une application monolithique, nous n'avons qu'une seule base données logique pour les entités persistantes alors que le cadre d'une application en microservices, chaque service a son propre modèle conceptuel et gère sa propre base de données.

— **Les extrémités intelligentes et les canaux stupides :** Plusieurs entreprises s'investissent dans les canaux de communication intelligents entre les services, alors qu'avec les microservices, l'utilisation de communications stupides est favorisée. Ces communication non intelligentes ne font que transmettre les messages, alors que le microservice s'en charge du reste. L'intercommunication entre les microservices via des protocoles ouverts est privilégiée et beaucoup d'autres interagissent les uns avec les autres via des appels REST ou à travers des systèmes de file d'attente.

— **Automatisation de l'infrastructure :** Les techniques d'automatisation de l'infrastructure ont connu une évolution considérable ces dernières années. L'évolution du cloud a réduit la complexité opérationnelle de la construction, du déploiement et de l'exploitation de microservices. Les entreprises, qui ont migré vers l'architecture microservices, ont gagné de l'expérience dans la livraison continue et l'intégration continue et elles utilisent maintenant des outils d'automatisation de l'infrastructure.

— **Conception pour l'échec :** L'un des atouts majeur des microservices est d'être tolérants aux pannes. Dans une application en microservices, si un service échoue, les autres services ne sont pas affectés et adaptent leurs fonctionnements selon l'état du système dans lequel ils évoluent.

— **Une conception évolutive :** L'un des éléments déterminants dans l'architecture en microservices, est la notion d'indépendance et d'évolutivité. En général, l'évolution d'une application consiste à l'ajout de nouvelles fonctionnalités qui se traduit par la création de nouveaux microservices et ou par la mise à jour des services existants qui implique seulement la mise à jour et le redéploiement du microservice concerné.

1.6 Technologies

Ce chapitre couvrira l'aspect technique de notre projet, en regardant en arrière sur la technologie qui a constitué la révision finale du projet. Nous expliquerons les différentes technologies que nous avons ajoutées et comment elles ont été mises en œuvre dans le cadre du projet. Nous examinerons la pile Web que nous avons utilisée et les technologies que nous avons ajoutées afin de créer un site de média social plus robuste et plus utile. Nous verrons pourquoi nous avons utilisé les technologies données et

les avantages que nous avons constatés par rapport à d'autres.

1.6.1 CEAN STACK

La CEAN stack est conçue pour fournir un point de départ simple et amusant pour les applications JavaScript en cloud. CEAN est un ensemble des framework Open Source qui, ensemble, constituent un cadre de bout en bout (end-to-end) pour la construction d'applications Web dynamiques, allant du haut (code exécuté dans le navigateur) au bas (base de données).
The stack est constitué de :

— **Couchbase** : C'est un projet open source, de base de données documentaire et distribuée. Il est spécialisé dans la gestion des données à faible temps de latence pour les applications Web, mobiles et IoT interactives à grande échelle.

— **Express** : C'est un micro framework pour construire des applications web en Node.js. Il fournit un ensemble robuste de fonctionnalités pour les applications Web et mobiles[?].

— **Angular 7** :C'est un framework Javascript très populaire actuellement, visant à structurer et simplifier le développement d'applications mono-pages (Single Page Application – SPA). Il est développé par Google sous licence open-source, il offre les outils nécessaires pour développer du code bien organisé, modulaire, basé sur l'injection de dépendances et prêt pour les tests unitaires.

— **Node.js** : Node.js est une technologie qui permet d'écrire du javascript coté serveur pour réaliser des applications webs.Node.js est particulièrement adapté aux projets ayant besoin de temps réel. Il supporte nativement des technologies comme les websockets, ce qui permet de créer facilement des applications réactives[?].

1.6.2 Logiciels et outils utilisés pour le développement

Au cours de cette partie, nous énumérons les différents outils utilisés tout au long de ce projet pour l'étude et la mise en place de notre application :

1.6.2.1 GraphQL

GraphQL est un langage de requêtes pour API ainsi qu'un environnement pour exécuter ces requêtes. Il est défini par une spécification indépendante des langages de programmation et des protocoles de transport, dans le but de s'inscrire comme un nouveau standard dans le développement d'API.

1.6.2.2 Confluence

Confluence est, un wiki d'entreprise. Cet outil de gestion de contenu permet de créer des espaces collaboratifs, d'alimenter une base de connaissances.

1.6.2.3 Jira

Jira Software est un outil de gestion de projet Agile qui prend en charge toute méthodologie Agile, qu'il s'agisse d'un scrum, d'un kanban ou de votre propre création. Grâce à un seul outil, nous pouvons planifier, suivre et gérer tous nos projets de développement logiciel Agile, des tableaux aux rapports Agile.

1.6.2.4 Balsamiq

Balsamiq est un outil permettant de créer facilement des prototypes d'IHM électronique. Avec Balsamiq Mockups il est ainsi possible de prototyper tout type d'applications (desktop, web, mobile).

1.6.2.5 Visual Studio Code

Visual Studio Code est un éditeur de code open-source développé par Microsoft supportant un très grand nombre de langages grâce à des extensions. Il supporte l'autocomplétion, la coloration syntaxique, le débogage, et les commandes git.

1.6.2.6 Postman

Postman est un outil de test d'API / services Web doté d'un puissant support client HTTP. Il possède un concepteur de requêtes convivial qui vous permet de rédiger des cas de test, de gérer des données et de réagir rapidement pour des tests et une gestion efficaces de l'API des cas de test, simplifie l'utilisation de l'API, prend en charge les développeurs à chaque étape de son flux de travail.

1.6.2.7 Bitbucket

Bitbucket est plus qu'un simple outil de gestion du code Git. Avec Bitbucket, les équipes disposent d'un espace unique où planifier des projets, collaborer autour du code, tester et déployer.

1.6.2.8 Slack

Slack est un outil de messagerie instantanée professionnel, l'outil est particulièrement pratique pour échanger des fichiers, slack fait directement le lien avec les autres outils comme Craft, github ou google docs.

1.6.2.9 Overleaf

Overleaf est une plateforme en ligne gratuite permettant d'éditer du texte en LATEX sans aucun téléchargement d'application. En outre, elle offre la possibilité de rédiger des documents de manière collaborative, de proposer ses documents directement à différents éditeurs ou plateformes d'archives ouvertes.

1.7 Chronogramme

Ce travail a été réalise durant une période de 4 mois. La répartition des tâches durant toute la durée de la réalisation de notre application est illustrée par la diagramme de Gantt réel de la figure suivante :

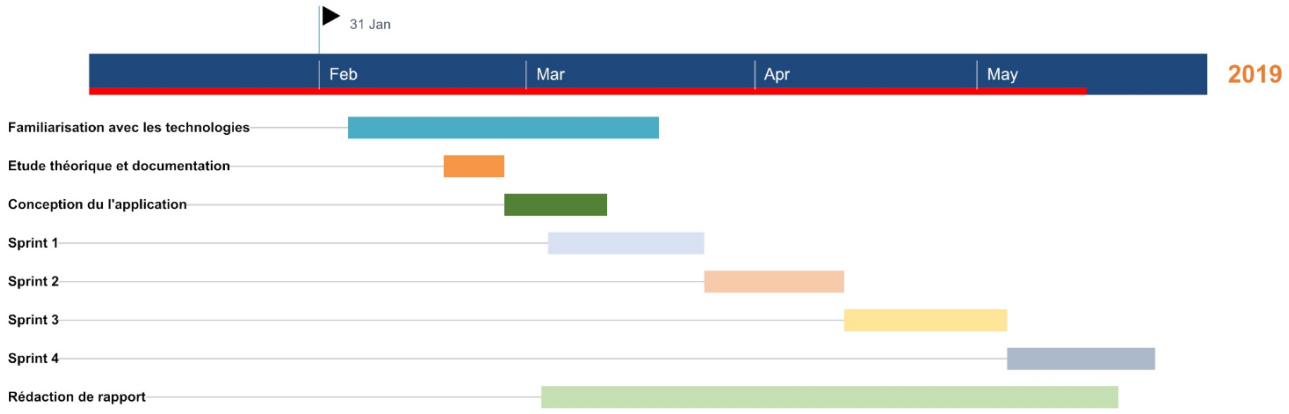


Figure 1.4: Diagramme de Gantt réel

Conclusion

Dans ce chapitres, nous avons présenté le cadre de notre projet, les différents processus de développement, le processus choisit qui nous aidera tout au long du projet. Nous définissons dans le deuxième chapitre, l'analyse et la spécification ses besoins.

Chapitre 2

Analyse et spécification des besoins

Plan

1 Identification des besoins 18

2 Identification des acteurs 19

3 Diagramme des cas d'utilisation global 20

4 Pilotage de projet avec scrum 21

Introduction

Ce chapitre vise à capter les besoins ainsi qu'identifier les rôles des utilisateurs qui utilisent le système et préparer le plan de réalisation. Dans un premier temps nous allons identifier les acteurs de notre projet, ceux qui toucheront de façon directe notre application, lister les exigences fonctionnelles et non fonctionnelles du produit puis nous allons présenter les besoins de notre système à travers le diagramme de cas d'utilisation global.
On finira par produire le backlog initial ainsi qu'une première planification des sprints.

2.1 Identification des besoins

Dans cette partie, nous allons passer à l'identification des differents besoins fonctionnels et non fonctionnels qui présente l'une des étapes majeurs sur les quelles se forge notre projet.

2.1.1 Besoins fonctionnels

Les besoins fonctionnels doivent répondre aux exigences du futur système en termes de fonctionnalités. Ils constituent une sorte de promesse ou de contrat au comportement de l'application à générer. De ce fait, notre application doit offrir un ensemble de services de base permettant de couvrir les besoins des différentes entités dans l'application.
Nous regroupons ces besoins dans les points suivants :

- Gestion des thèmes.
- Gestion des différents langues.
- Gestion des différents sources de média (RSS feed, Facebook).
- Gestion des feeds et collections des différents feeds dans RAW DATA.
- Modération des contenus.
- Gestion des flows de la modération.
- Gestion du contenu que va être diffusé en live (On Air).
- Gestion des flows de modération.
- Gestion des caméras.
- Contrôle des caméras.
- Gestion des comptes sociaux média.
- Création et planification des diffusions en direct
- Gestion des destinations de live diffusion.

- Apport le modèle de données de l'appareil.
- Création nouveau modèle de données personnalisé (Vue client).
- Gestion de différents vues client.
- Génération de contrôle à partir de la vue client.
- Gestion des feeds.
- Gestion des utilisateurs.
- Génération des rapports.

2.1.2 Besoins non fonctionnels

En plus des besoins fonctionnels, notre application doit répondre aux critères suivants qui ne concernent pas spécifiquement le comportement de l'application mais plutôt identifient des contraintes internes et externes du cette dernière :

- **La simplicité** : plus l'application est simple dans l'apprentissage et dans l'utilisation, plus ses fonctionnalités sont utilisées.
- **L'ergonomie** :L'application fournira une interface conviviale et simple à utiliser et qui ne requiert aucun pré requis, donc elle pourra être exploitable par tout type d'utilisateurs (même les non informaticiens).
- **L'extensibilité** : L'architecture de l'application permettra l'évolution et la maintenance (ajout ou suppression ou mise à jour) au niveau de ses différents modules d'une manière flexible.
- **L'évolutivité** : l'application peut avoir des extensions, en intégrant des nouveaux modules pour répondre aux nouveaux besoins fonctionnels et ceci sans modifier les modules déjà existants.
- **La confidentialité** : l'information échangée entre deux correspondants ne peut pas être consultée par un tiers.
- **Maintenabilité** : Les différents modules de l'application doivent être faciles à maintenir pour une détection rapide des éventuelles anomalies.
- **La sécurité** : L'accès aux informations n'est possible qu'après vérification des privilèges et des droits d'accès. Ainsi tout utilisateur passera par une phase d'authentification pour pouvoir consulter les services ouverts par l'application.

2.2 Identification des acteurs

L'analyse d'une application débute par la détermination de ses différents acteurs. Une étude de l'interaction du système avec son environnement extérieur a permis de dégager principalement :

1. **L'acteur Journaliste** : il est l'utilisateur principal de système. Il peut gérer les Média et les cameras (mouvement, zoom, vitesse de mouvement, enregistrer presets, ...).
2. **L'acteur Media Manager** : il est responsable de la modération de contenu.
3. **L'acteur Administrateur** : il est responsable des différents paramétrages (tel que les gestion des utilisateur et des rôles) de l'application. Son rôle est de gérer les environnements et les utilisateurs.

   Nous avons associé pour chaque utilisateur une interface qui lui offre l'ensemble des fonctionnalités et services qu'il peut effectuer á travers le système. Dans la suite, nous allons identifier les fonctionnalités requises pour chacun des acteurs cités ci-dessus.

2.3 Diagramme des cas d'utilisation global

Les cas d'utilisation représentent un moyen de modélisation essentiel pour exprimer les besoins de tout projet. Les cas d'utilisation interviennent très tôt dans la phase de spécification pour bien assurer le développement d'un outil adapté aux exigences de ses utilisateurs. A travers cette section nous présentons donc le diagramme des cas d'utilisation relatifs à notre travail.

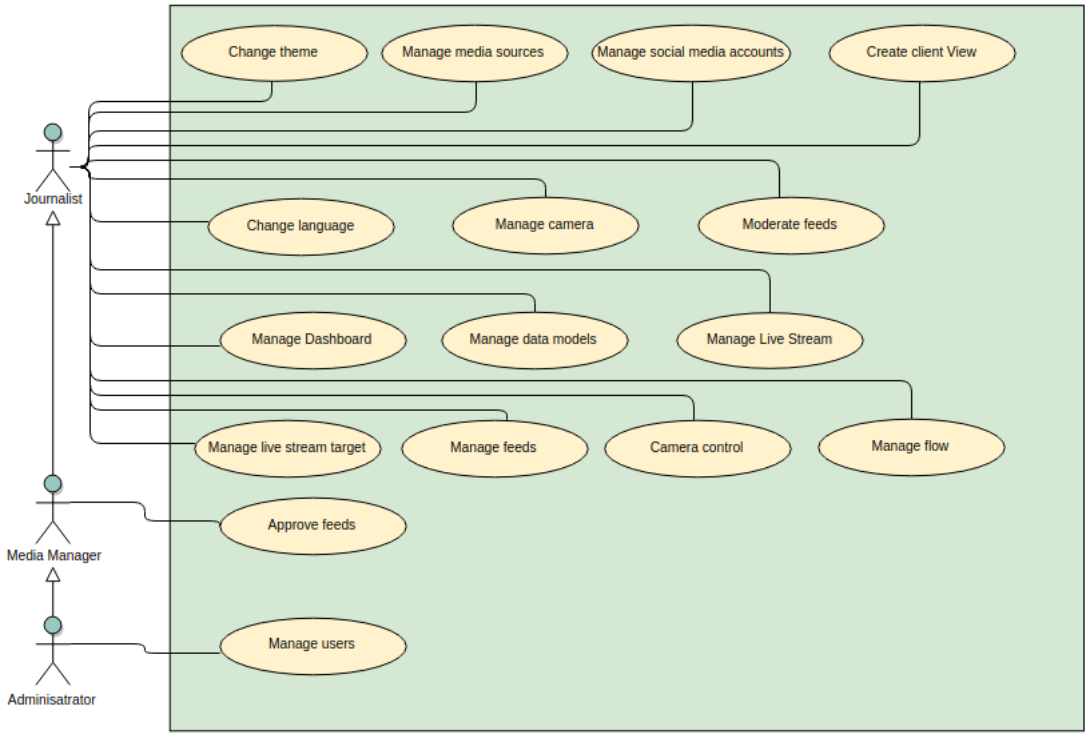


Figure 2.1: Diagramme de cas d'utilisation global

Pour effectuer les cas d'utilisation les utilisateurs doivent s'authentifier.

2.4 Pilotage de projet avec scrum

Dans cette partie nous mettons l'accent sur la méthodologie scurm. Nous déterminerons les différents rôles scrum dans notre projet, le backlog produit.

2.4.1 Equipe et rôles

Scrum définit trois rôles qui sont :

- Le Product Owner (le propriétaire du produit).

- Le Scrum Master (le directeur de produit).

- Le Scrum Team (l'équipe Scrum)

Dans le contexte de notre projet, M. Youssef BOUJRAF sera le directeur de produit (Scrum Master) et je forme moi, Eric ROGIEN, Mohamed Naceur Boubakri, les membres de l'équipe Scrum.

2.4.2 Le backlog produit

ID	Histoire utilisateur
1.1	Mise à place les composants d'en-tête, de barre latérale et de pied de page
1.2	Créer le composant de gestion des rôles
1.3	Créer le composant de gestion des utilisateur
1.4	Créer le service de gestion des rôles
1.5	Créer le composant de gestion des thèmes
1.6	Créer le service de gestion des thèmes
1.7	Créer le composant de gestion des langues
1.8	Créer le service de gestion des langues
2.1	Création du module de l'affichage
2.2	Création du module de l'ajout
2.3	Création du module de la modification
2.4	Création du module de démarrage le feed
2.5	Création du module d'arrêt le feed
2.6	Création du module de suspension le feed
3.1	Créer le composant de gestion des comptes
3.2	Créer le composant de diffusion en direct
3.3	Créer le service de gestion des comptes
3.4	Créer le service de diffusion en direct
3.5	Ajouter Le planification de diffusion en direct
3.6	Créer le service de planification de diffusion en direct
4.1	Mise à place les composants d'affichage le flux de caméra
4.2	Conversion le flux RTSP en flux HTTP
4.3	Créer les composants de contrôle
4.4	Créer le service de contrôle
4.5	Créer la partie d'enregistrement de presets
4.6	Intégration ce module composant dans la partie de diffusion

Conclusion

Dans ce chapitre, nous avons présenté la solution proposée tout en citant les fonctionnalités principales. Par la suite, nous avons procédé à une identification des acteurs et nous avons fait une analyse des besoins par le biais des diagrammes de cas d'utilisation pour mieux comprendre le

principe de fonctionnement de l'application. Cette étude a permis de résoudre certains problèmes lies au développement de l'application. Le prochain chapitre portera sur la phase de conception

Chapitre 3

Sprint 1 : Mise en place des diverses fonctionnalités

Plan

1 Backlog du sprint 25

2 Analyse 25

3 Conception 27

4 Réalisation 31

5 Conclusion 36

Introduction

Ce sprint a pour but de mise en place des diverses modules principaux tel que la gestion des utilisateur, les rôles, les thème et le support de plusieurs langues.

3.1 Backlog du sprint

ID	Histoire utilisateur
0.1	Mise à place les composants d'en-tête, de barre latérale et de pied de page
0.2	Créer le composant de gestion des rôles
0.3	Créer le composant de gestion des utilisateur
0.4	Créer le service de gestion des rôles
0.5	Créer le composant de gestion des thèmes
0.6	Créer le service de gestion des thèmes
0.7	Créer le composant de gestion des langues
0.8	Créer le service de gestion des langues

3.2 Analyse

Après avoir étudié les différentes tâches du sprint 1, nous allons établir quelques maquettes afin de les valider avec le Product Owner.

La figure 3.3 présente l'interface qui permet aux utilisateurs d'authentifier et d'accéder aux différentes fonctionnalités selon ses droits d'accès.

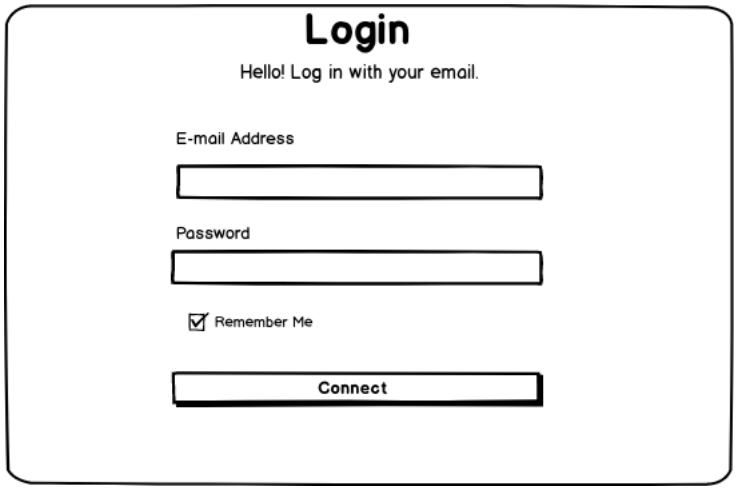


Figure 3.1: Maquette de l'interface d'authentification

La figure 3.2 présente l'interface d'accueil de l'administrateur. A partir de cet interface l'administrateur peut naviguer dans l'application et même en utilisant les écrans tactiles.

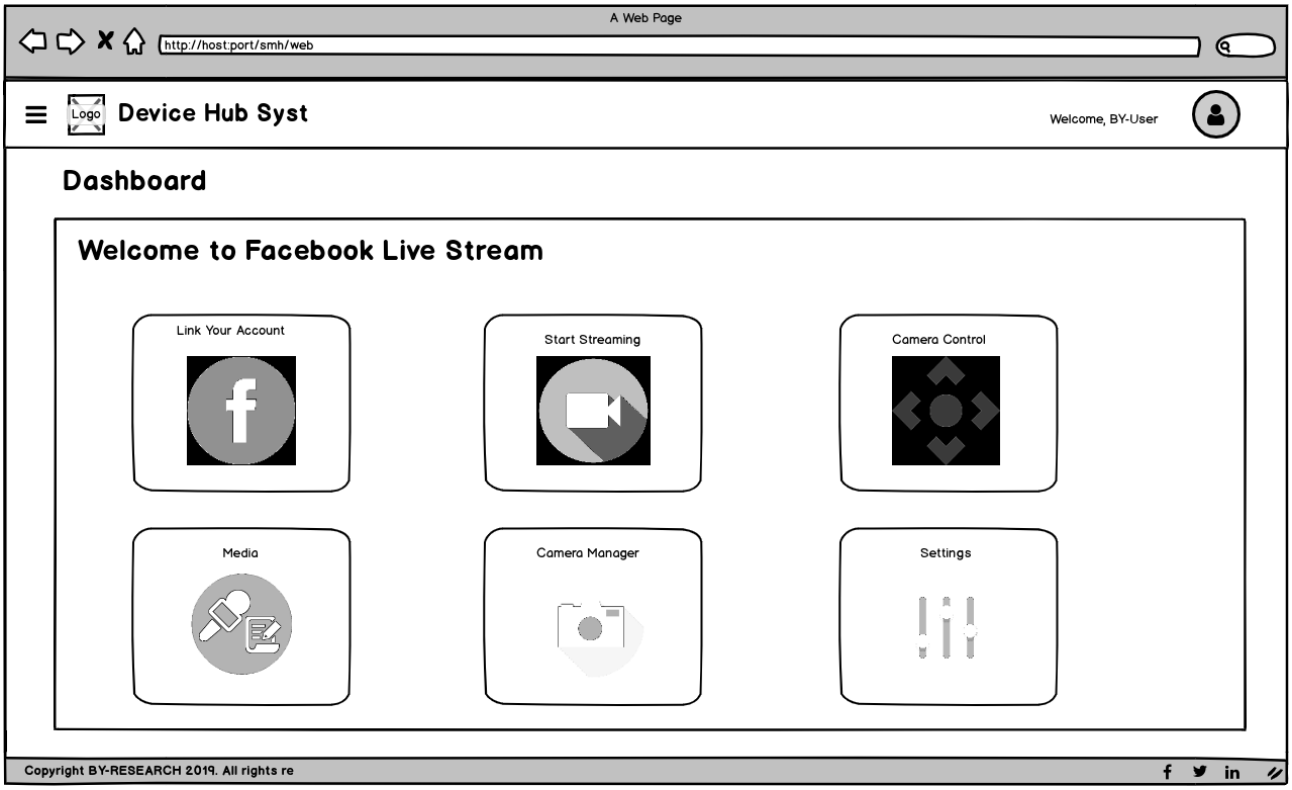


Figure 3.2: Maquette de l'interface d'accueil de l'administrateur

La figure 3.3 présente l'interface qui permet á l'administrateur de gérer les rôles a effectuer pour les utilisateurs.

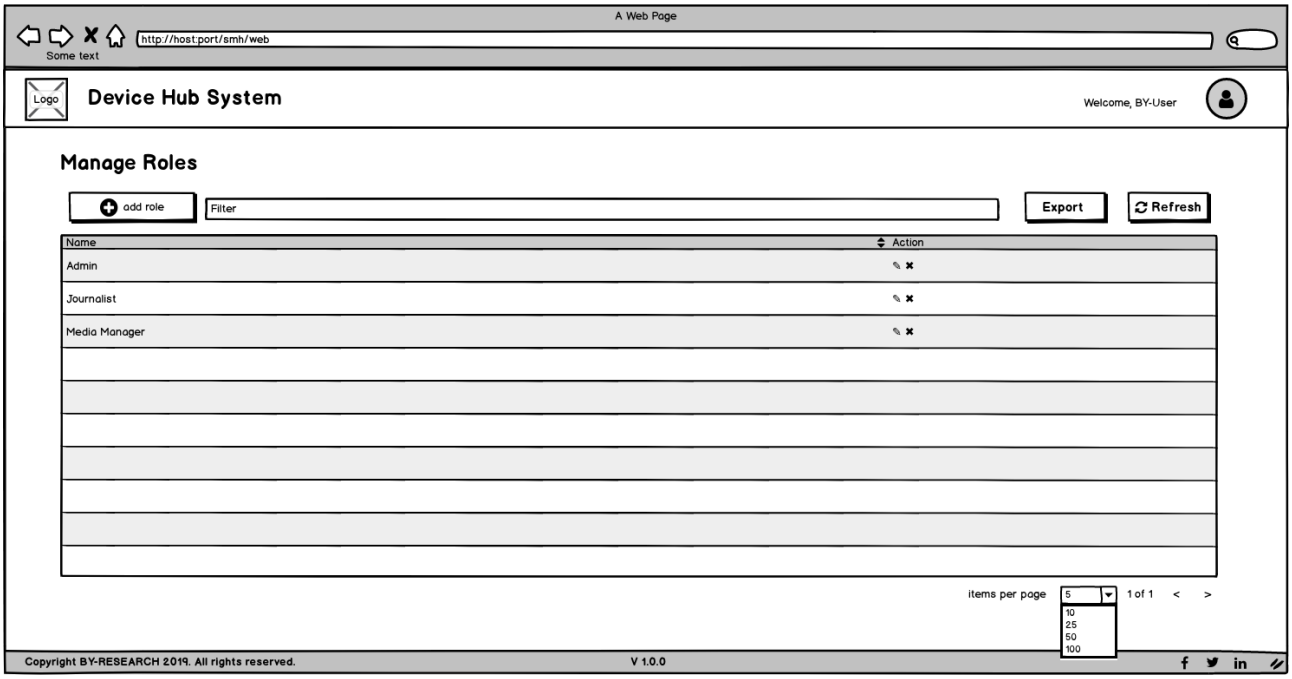


Figure 3.3: Maquette gérer rôles

La figure 3.3 présente l'interface qui permet á l'administrateur de gérer les utilisateurs et d'affecter pour chaque utilisateur un rôle et un login.

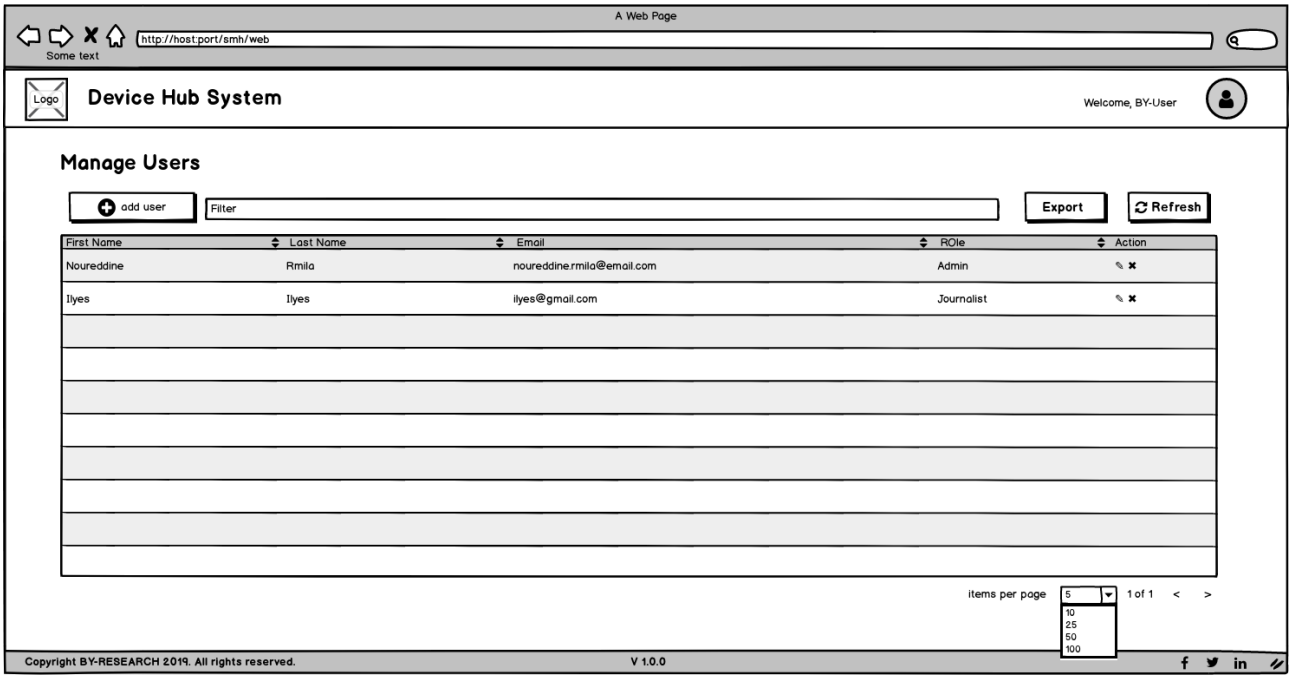


Figure 3.4: Maquette gérer utilisateurs

3.3 Conception

3.3.1 Raffinement des cas d'utilisation

Dans cette section, nous allons présenter un raffinement de quelque cas d'utilisation du diagramme global.

3.3.1.1 Diagramme de cas d utilisation « S'authentifier »

Toutes interactions avec le système ne peut avoir lieu qu'après l'authentification. Nous présentons dans ce paragraphe le diagramme de cas d'utilisation « S'authentifier » qui est destinée aux différents acteurs.

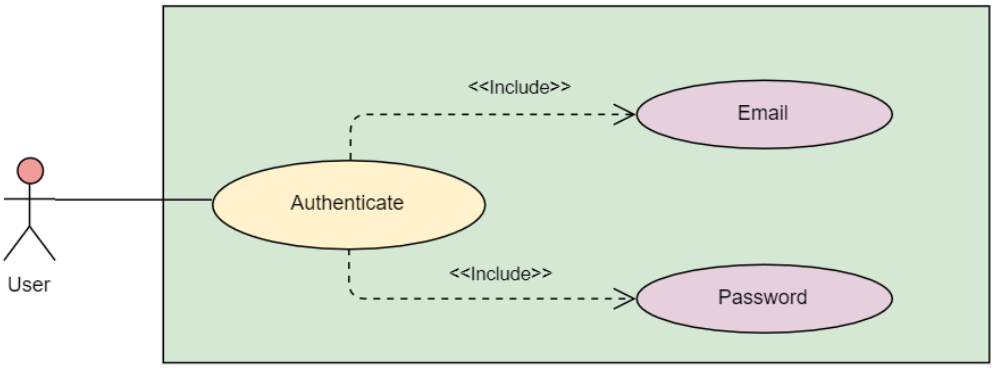


Figure 3.5: Diagramme de cas d'utilisation « S'authentifier »

Tableau 3.1: Tableau d'analyse du cas d'utilisation « s'authentifier »

Sommaire de Cas d'utilisation « S'authentifier »	
Titre	S'authentifier .
Acteur	Tous les acteurs
Description des enchainements	
Pré-condition	**Post-condition**
L'utilisateur possède un compte utilisateur valide.	L'utilisateur est authentifié sur l'application.
Scénario nominal : Ce cas d'utilisation commence lorsque l'utilisateur accède à l'interface d'authentification.	
1. L'utilisateur accède à la page d'authentification. 2.Le Système charge le formulaire d'authentification. 3. L'utilisateur tape son login et mot de passe. 4. L'utilisateur clique sur le bouton « Connexion ». 5. Le système redirige l'utilisateur vers la page d'accueil qui lui correspond.	
Enchaînements alternatifs :	
1. L'utilisateur saisit incorrectement son login et/ou son mot de passe. 2. Le Système affiche un message d'erreur. 3. L'utilisateur à oublié son mot de passe. 4. L'utilisateur il doit contacter l'administrateur. Ce cas d'utilisation se termine lorsque le processus d'authentification de l'utilisateur est terminé avec succès.	
Exceptions :	
1. L'utilisateur n'est pas en mesure de s'authentifier sur l'application.	

3.3.2 Diagrammes de séquence

À ce niveau, nous arrivons à présenter les diagrammes de séquence système des différents cas d'utilisation déjà expliqués :

3.3.2.1 Ajouter Utilisateur

La figure suivante présente le diagramme de séquence Ajouter Utilisateur

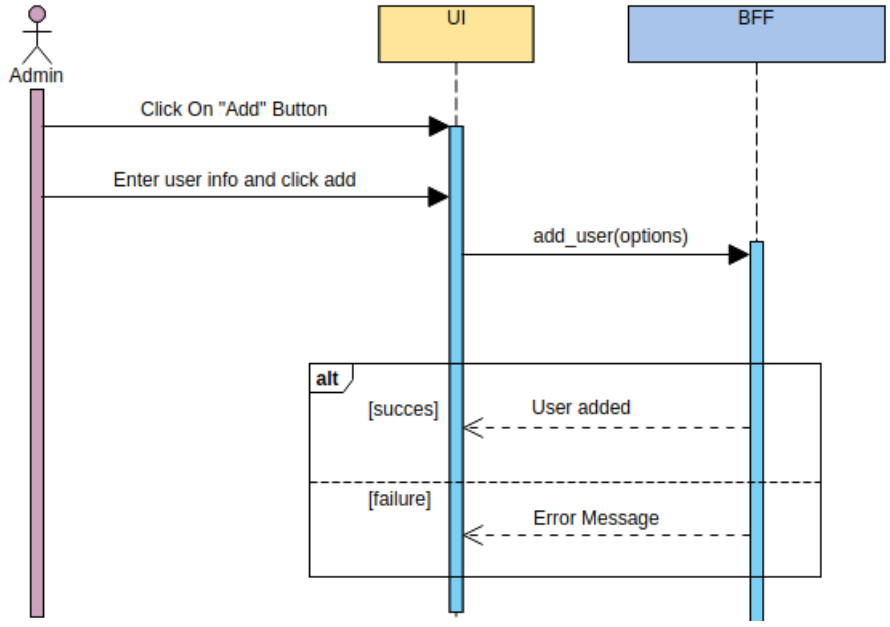


Figure 3.6: Diagramme de séquence Ajouter Utilisateur

3.3.2.2 Modifier Utilisateur

La figure suivante présente le diagramme de séquence Modifier Utilisateur

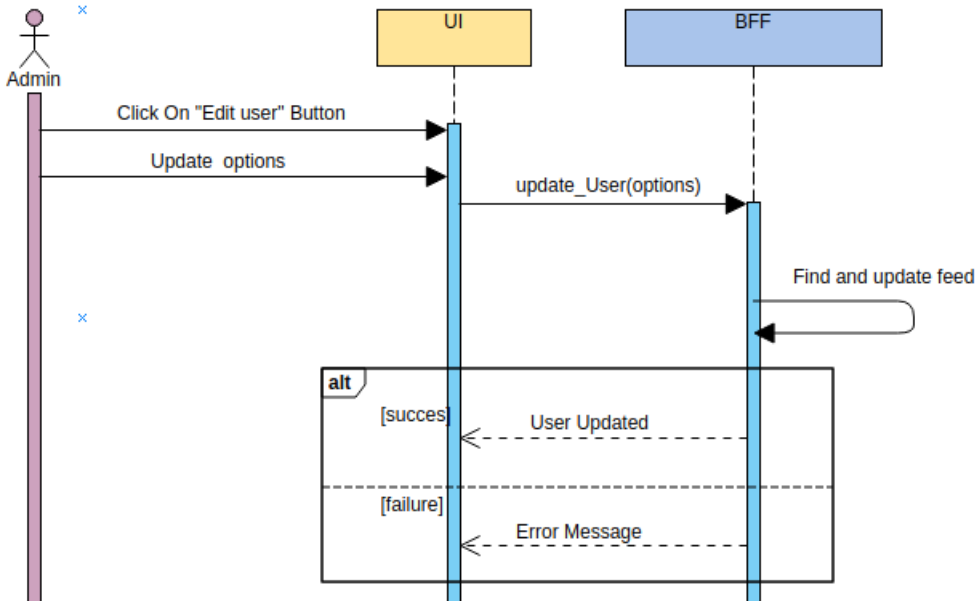


Figure 3.7: Diagramme de séquence Modifier Utilisateur

3.3.2.3 Ajouter Rôle

La figure suivante présente le diagramme de séquence Ajouter Rôle

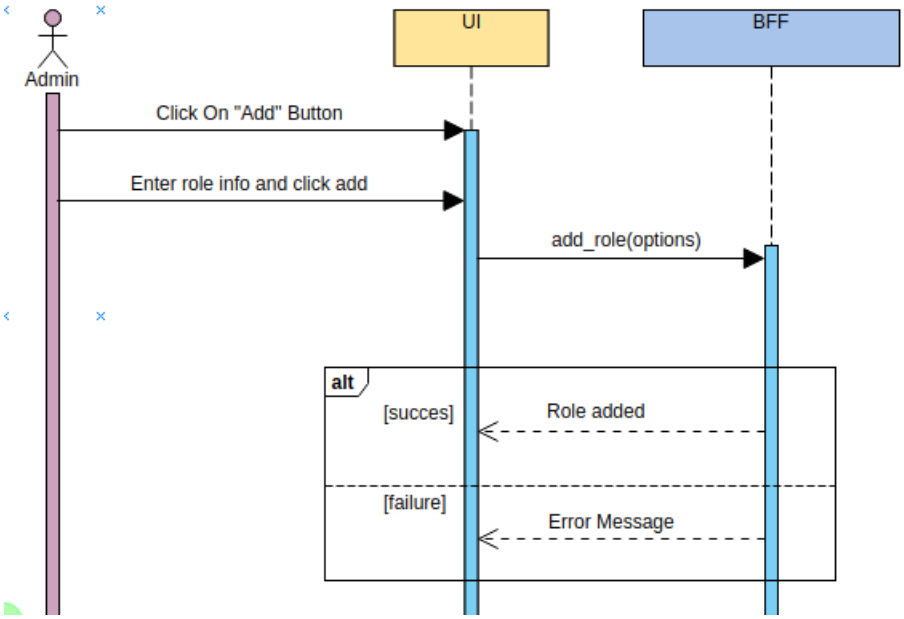


Figure 3.8: Diagramme de séquence Ajouter Rôle

3.3.2.4 Supprimer Rôle

La figure suivante présente le diagramme de séquence Supprimer Rôle

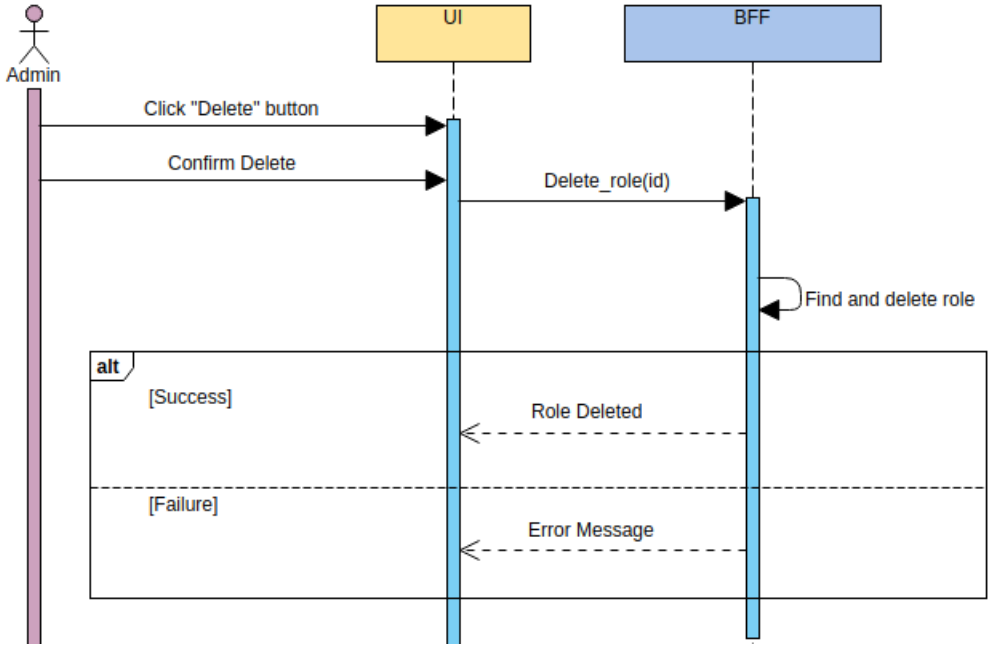


Figure 3.9: Diagramme de séquence Supprimer Rôle

3.3.3 Diagramme de classe

La figure suivante présente le diagramme de classe de sprint 1 :

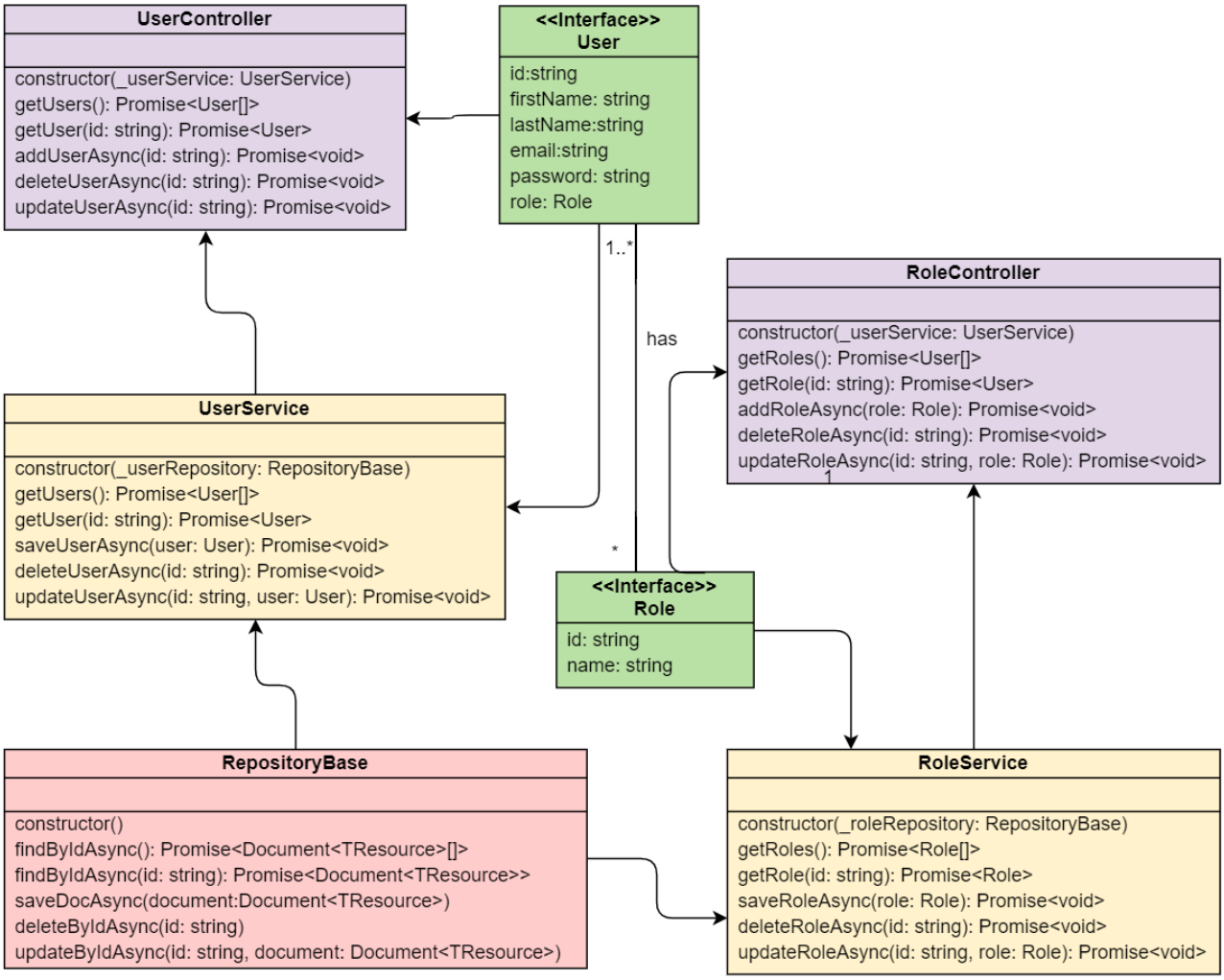


Figure 3.10: diagramme de classe de sprint 1

3.4 Réalisation

Dans cette section, nous allons présenter quelques interfaces qui concerne le Sprint 1.

Interface de connexion

Cette interface permet à l'utilisateur de s'authentifier sur l'application afin de l'identifier et le permettre d'accéder à son espace personnel selon les droits accordés.

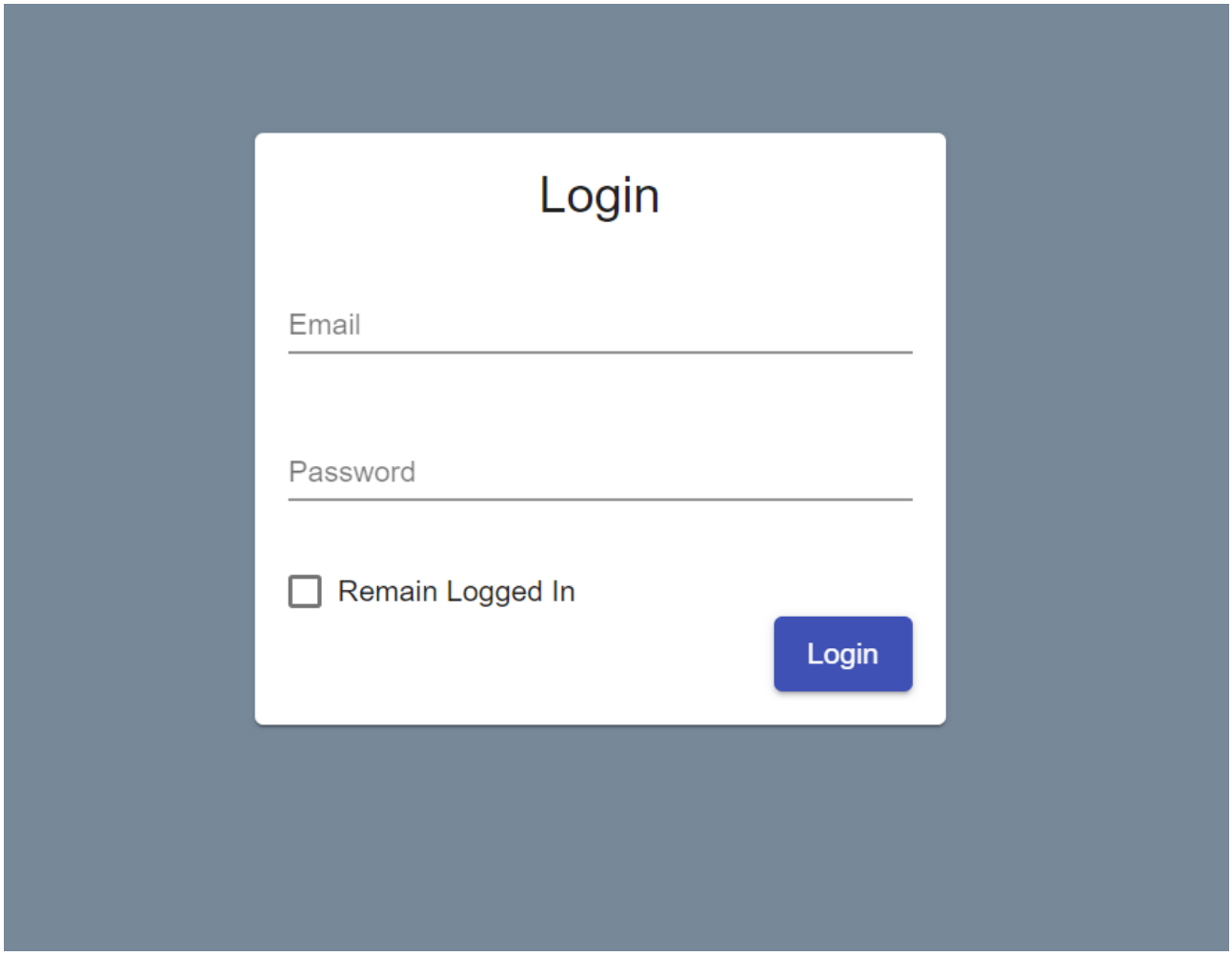


Figure 3.11: Interface de connexion

Interface d'accueil administrateur

Cette interface permet à l'administrateur du site d'accéder aux différentes fonctionnalités selon ses droits d'accès.

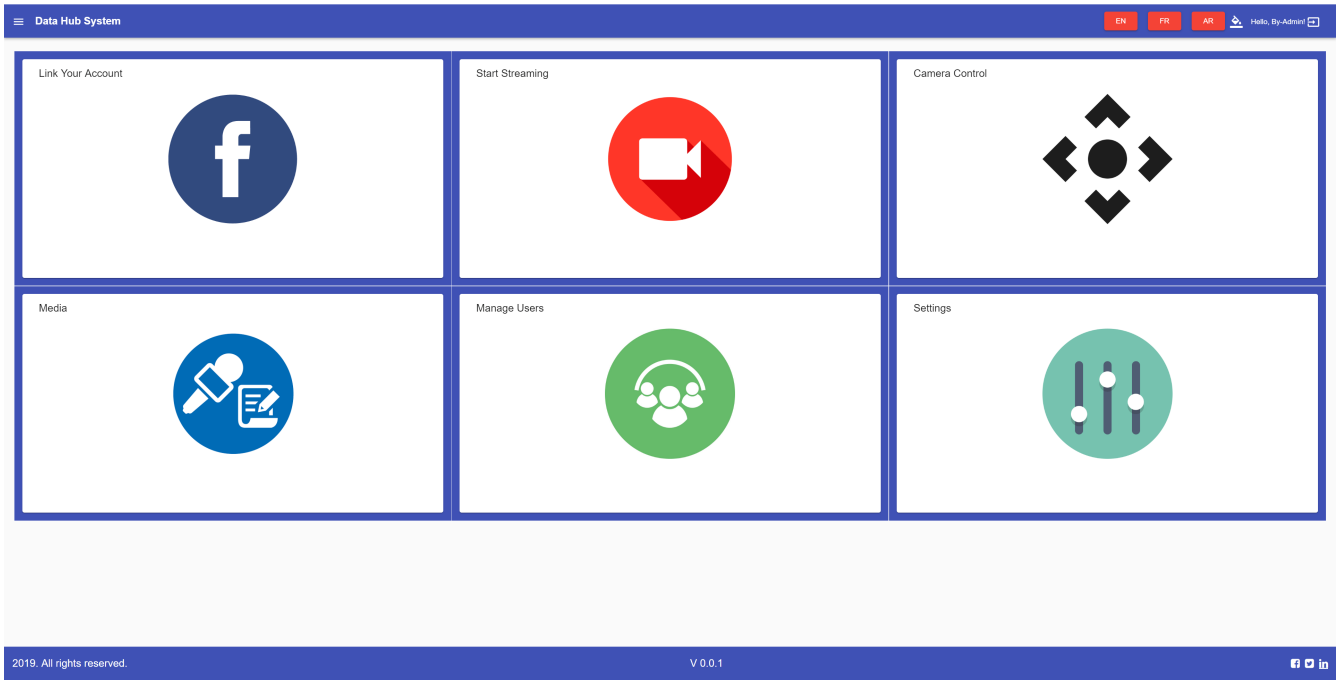


Figure 3.12: Interface d'accueil de l'administrateur

Changement de Langue

Cette interface permet à l'utilisateur du site de changer la langue de l'application.



Figure 3.13: Interface Changement de Langue

Changement de thème

Cette interface permet à l'utilisateur du site de changer la thème de l'application.

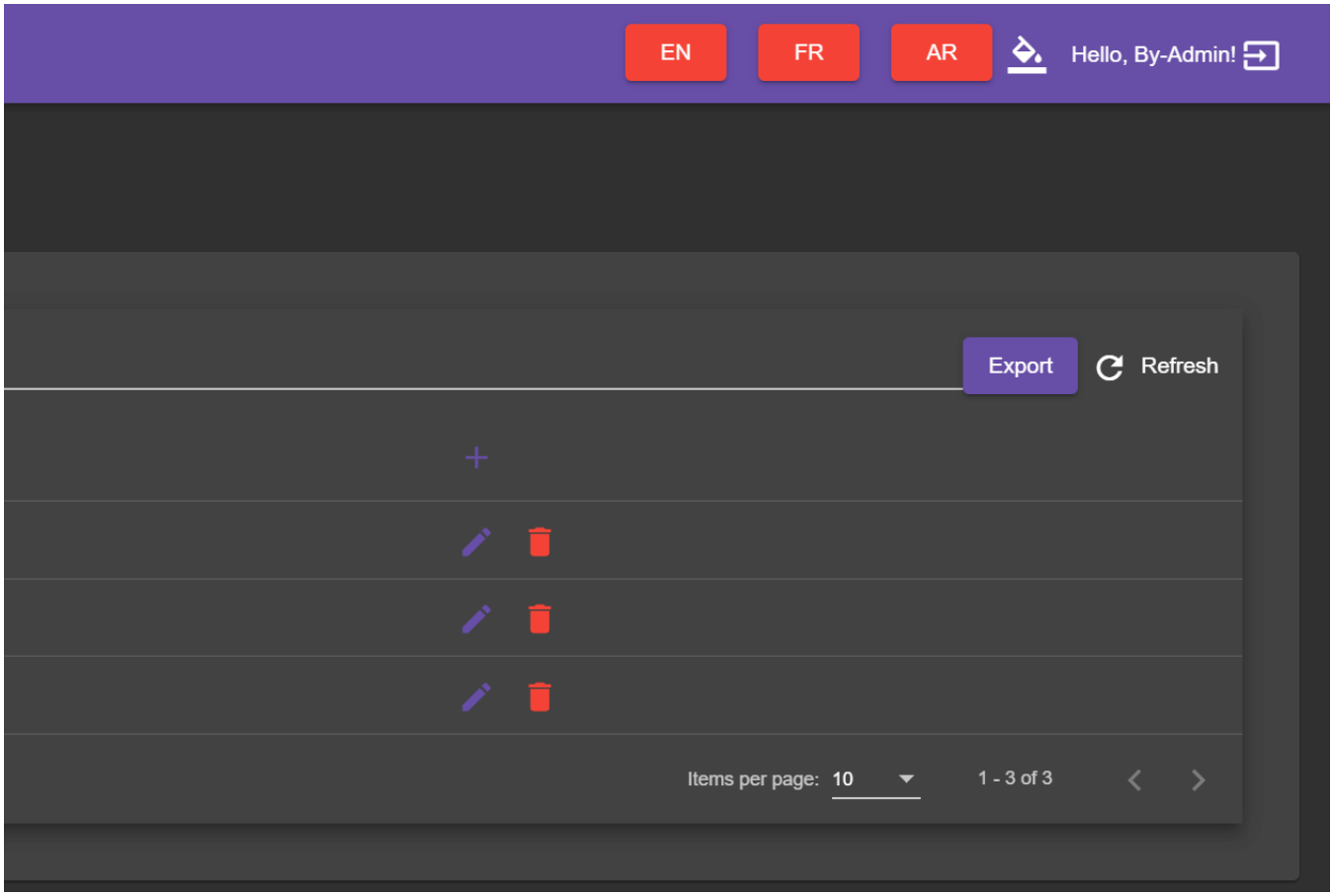


Figure 3.14: Interface Changement de thème

Interfaces de gestion des utilisateurs

Cette interface permet à l'administrateur du site de gérer les différents utilisateurs qui peuvent accéder 'a l'application et de leurs accorder les droits d'accès.

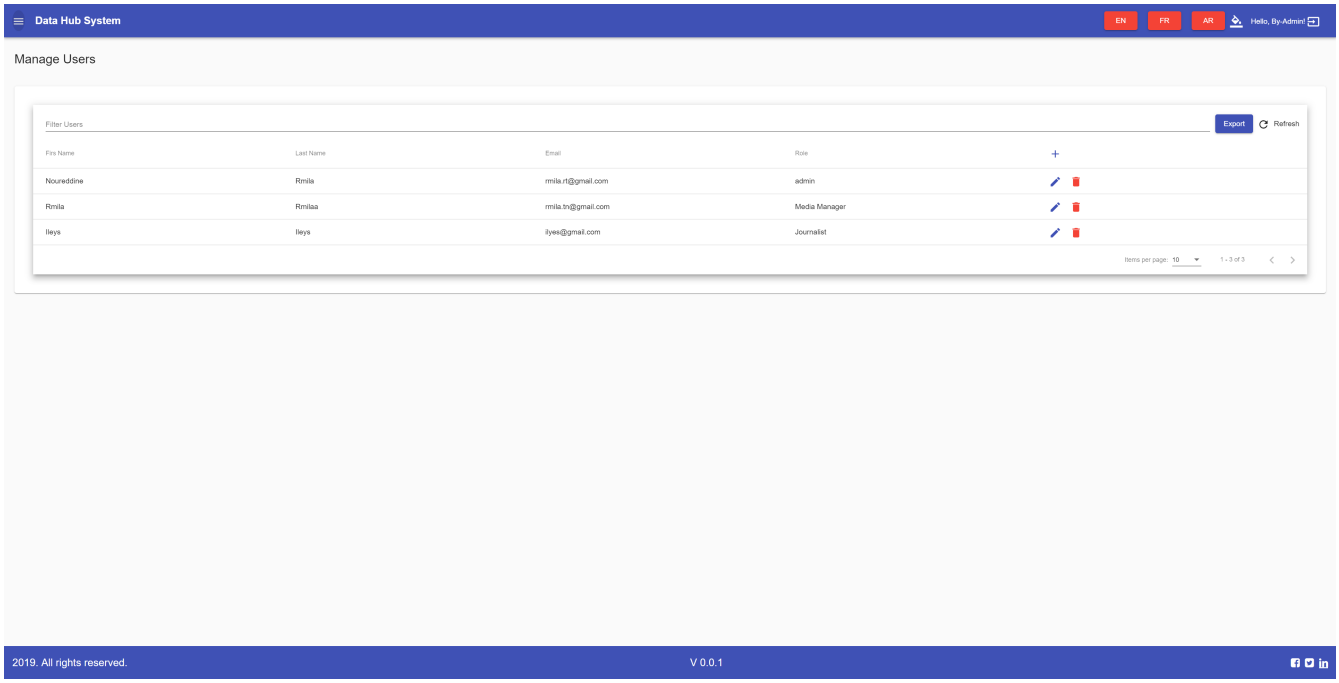


Figure 3.15: Interface de la gestion des utilisateurs

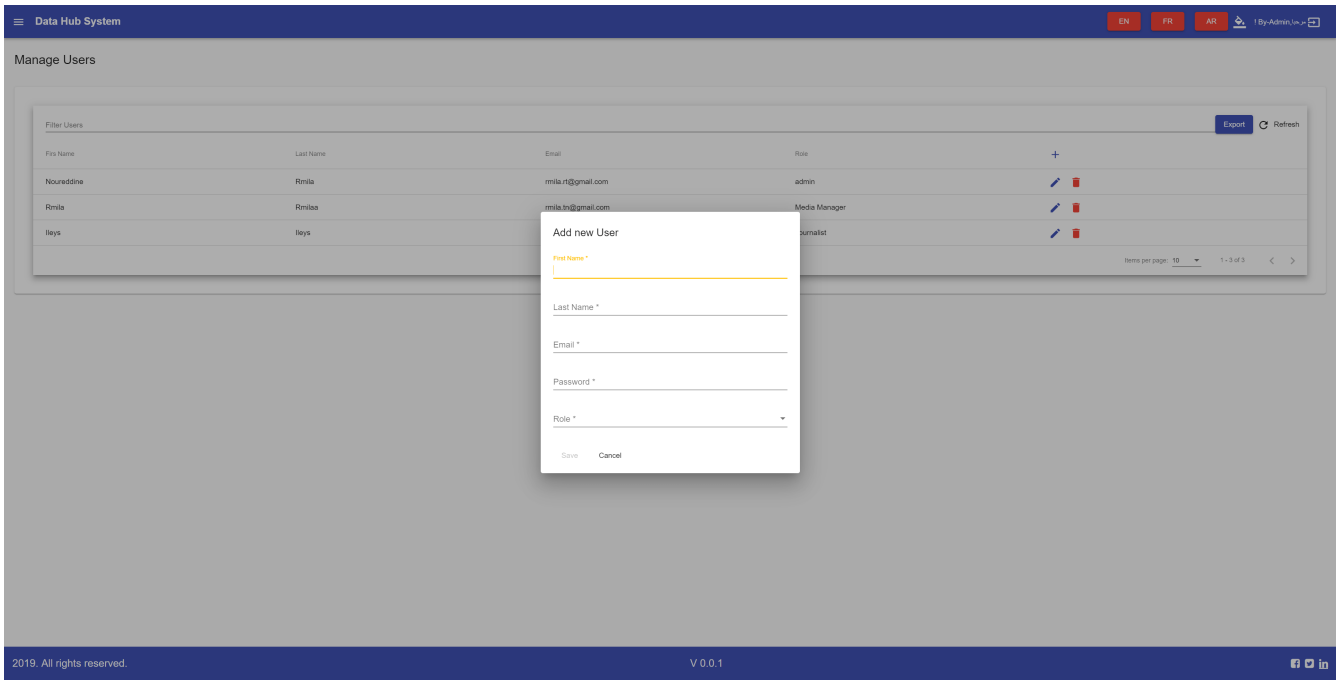


Figure 3.16: Interface de l'ajout d'un utilisateur

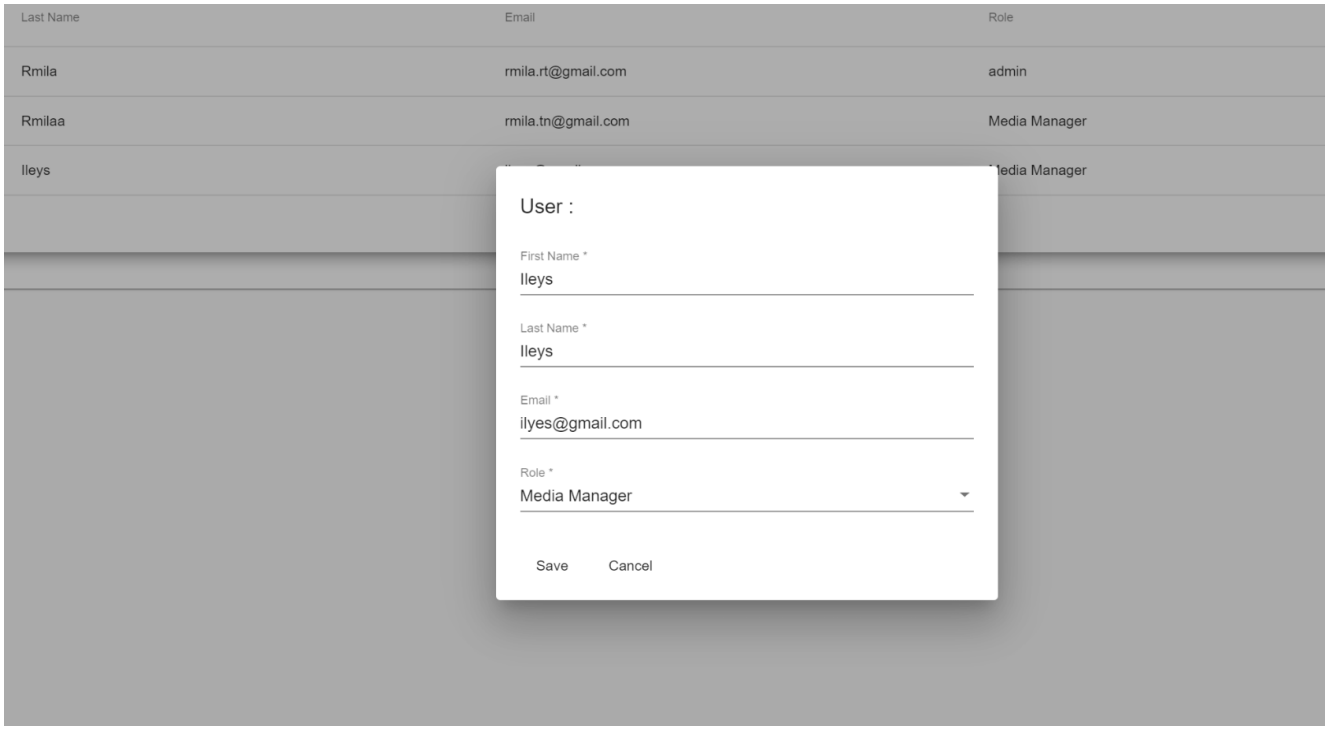


Figure 3.17: Interface de la modification d'un utilisateur

Interface de mise a jours de profile

Cette interface permet à l'utilisateur du site modifier les données de sont profile.

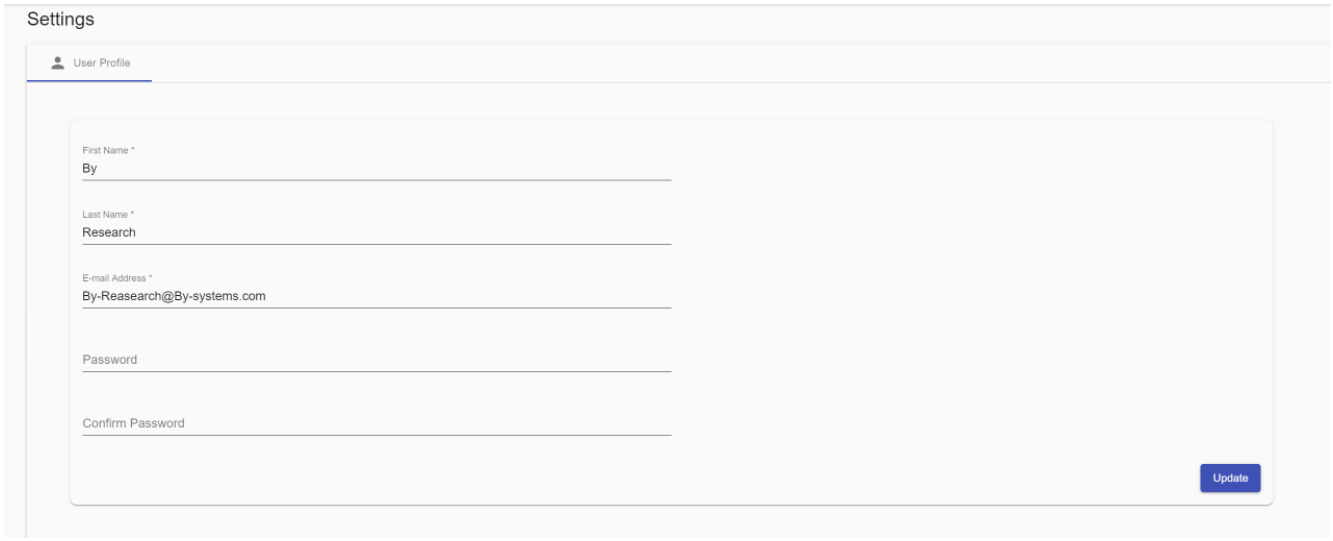


Figure 3.18: Interface de la mise a jours de profile

3.5 Conclusion

A la fin de ce chapitre nous avons réussir à produire un incrément ayant suffisamment de valeur pour le client et pourra être utilisé dans un environnement de production. Dans le chapitre suivant on va attaquer avec la même approche le deuxième sprint.

Chapitre 4

Sprint 2 : RSS FEED Consumer

Plan

1 Backlog du sprint 38

2 Analyse 38

3 Conception 39

4 Réalisation 50

5 Conclusion 54

Introduction

Nous commencerons, tout d'abord, par présenter le backlog du sprint suivi d'une analyse détaillée et la conception du RSS FEED Consumer, et nous présenterons par la suite, les interfaces homme-machine réalisées ainsi que la phase de test.

4.1 Backlog du sprint

ID	Histoire utilisateur
0.1	Création du module de l'affichage
0.2	Création du module de l'ajout
0.3	Création du module de la modification
0.4	Création du module de démarrage le feed
0.5	Création du module d'arrêt le feed
0.6	Création du module de suspension le feed

4.2 Analyse

Après avoir étudié les différentes tâches du sprint 2, nous allons établir quelques maquettes afin de les valider avec le Product Owner.

La figure 4.2 présente l'interface qui permet de gérer les sources de médias (RSS Feed).

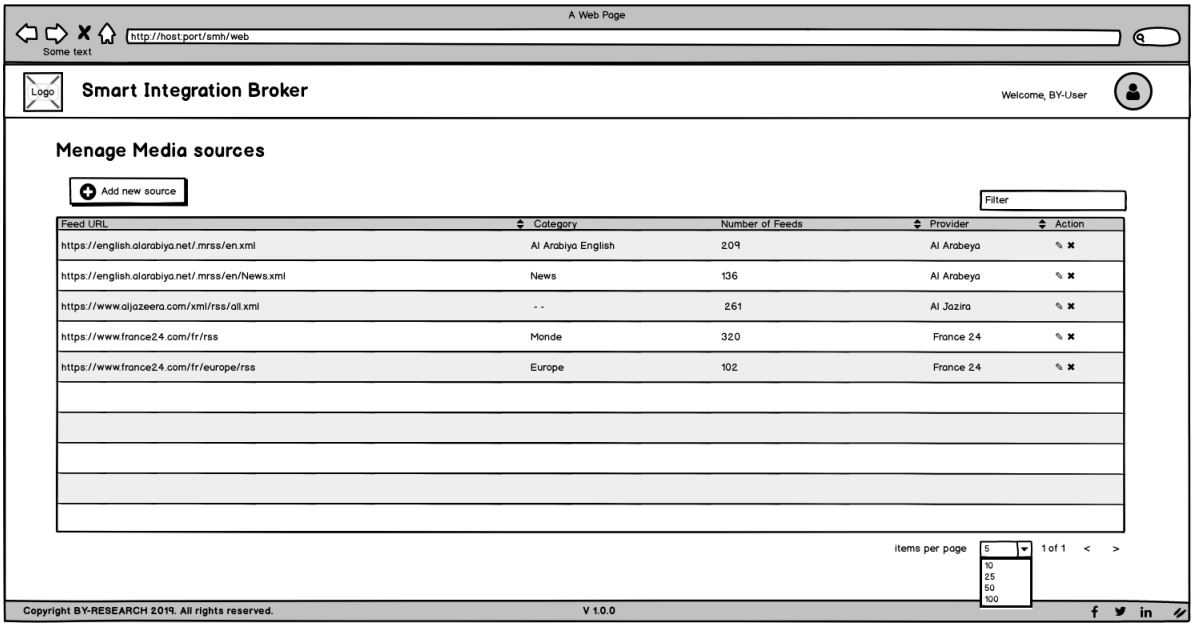


Figure 4.1: Maquette de gestions des sources de médias

La figure 4.2 présente l'interface qui permet de gérer les sources de médias (RSS Feed) démarrer,

arrêter ou suspendre les médias items dans le Raw Data.

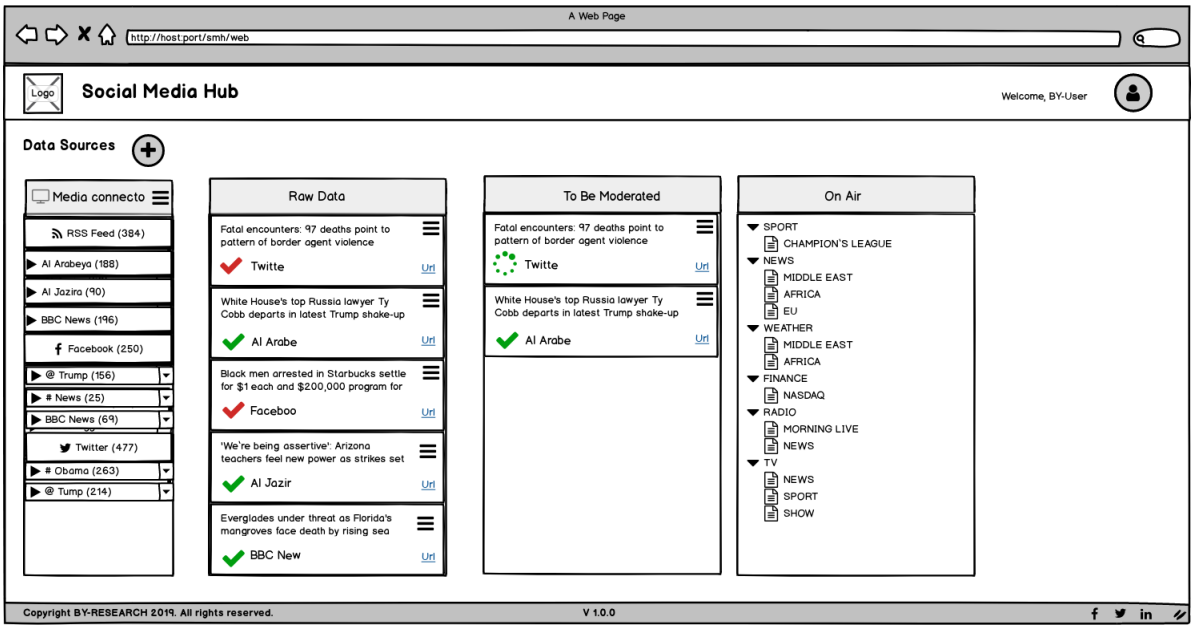


Figure 4.2: Maquette de flow de média global

4.3 Conception

4.3.1 Raffinement des cas d'utilisation

Dans cette section, nous allons présenter un raffinement de quelque cas d'utilisation du diagramme global.

4.3.1.1 Diagramme de cas d'utilisation «Gérer les sources flux»

La figure suivante présente le diagramme de cas d'utilisation Gérer flux.

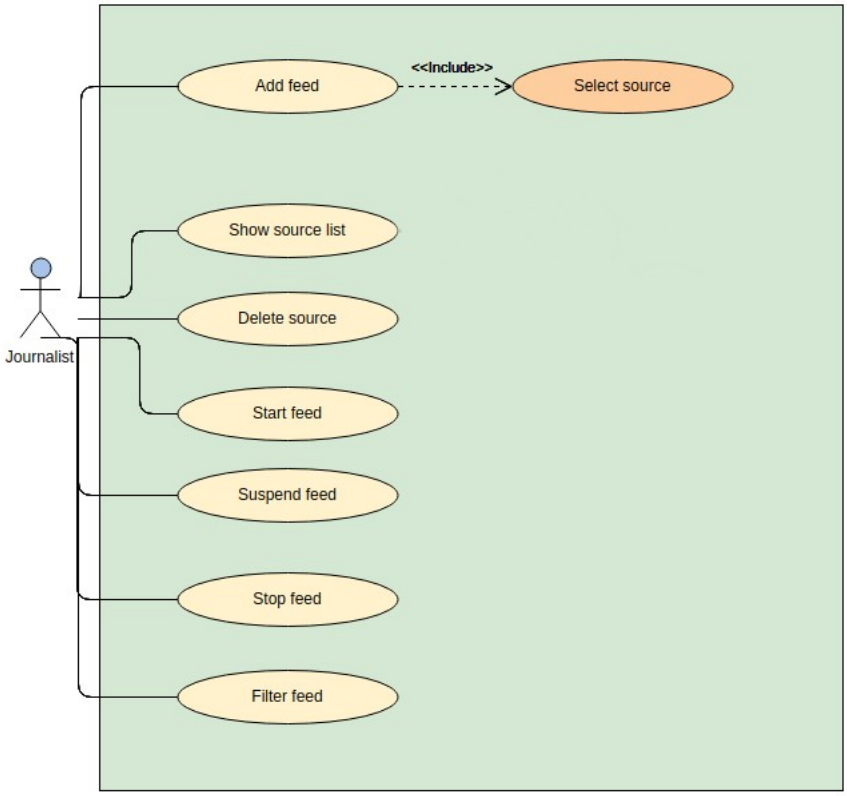


Figure 4.3: Diagramme de cas Gérer flux

Le tableau 4.1 présente la description textuelle du cas d'utilisation Ajouter flux.

Tableau 4.1: Tableau d'analyse du cas d'utilisation « Ajouter flux »

Sommaire de Cas d'utilisation « Ajouter flux »	
Titre	Ajouter une source de flux.
But	L'ajout d'une sources de flux par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Un nouveau flux est ajouté
Scénario nominal	
1. L'utilisateur demande le formulaire d'ajout d'un flux. 2. Le système affiche le formulaire. 3. L'utilisateur remplit les champs nécessaires. 4. Le système vérifie les données saisies 5. Le système enregistre un nouveau flux	
Scénario d'échec	
4. L'utilisateur saisie des données manquantes ou erronés. 4-a-1- Le système affiche une erreur. 4-a-2- Reprise à l'étape 3 de scénario nominal.	

Le tableau 4.2 présente la description textuelle du cas d'utilisation Supprimer flux.

Tableau 4.2: Tableau d'analyse du cas d'utilisation « Supprimer flux »

Sommaire de Cas d'utilisation « Supprimer flux »	
Titre	Supprimer une source de flux.
But	La suppression d'une sources de flux par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Le flux choisit est supprimé
Scénario nominal.	
1. L'utilisateur demande la liste des flux 2. Le système affiche la liste des flux. 3. L'utilisateur choisit le flux à supprimer. 4. Le système vérifie la possibilité de suppression de flux choisit. 5. Le flux est supprimé.	
Scénario d'échec	
4. L'utilisateur choisit un flux qui ne peut pas être supprimé. 4-a-1- Le système affiche un message d'erreur. 4-a-2- Reprise à l'étape 3 de scénario nominal.	

Le tableau 4.3 présente la description textuelle du cas d'utilisation Modifier flux.

Tableau 4.3: Tableau d'analyse du cas d'utilisation « Modifier flux »

Sommaire de Cas d'utilisation « Modifier flux »	
Titre	Modifier une source de flux.
But	La modification d'une sources de flux par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Les données du flux choisit sont bien mises à jour.
Scénario nominal	
1. L'utilisateur demande la liste des flux 2. Le système affiche la liste des flux. 3. L'utilisateur choisit le flux à modifier. 4. Le système vérifie les données qui peuvent être modifié pour le flux choisit et affiche le formulaire de modification 5. Le flux est est modifié.	
Scénario d'échec	
4. L'utilisateur saisie données manquantes ou erronés 4-a-1- Le système affiche une erreur. 4-a-2- Reprise à l'étape 3 de scénario nominal.	

4.3.1.2 Diagramme de cas d'utilisation « Gérer flux »

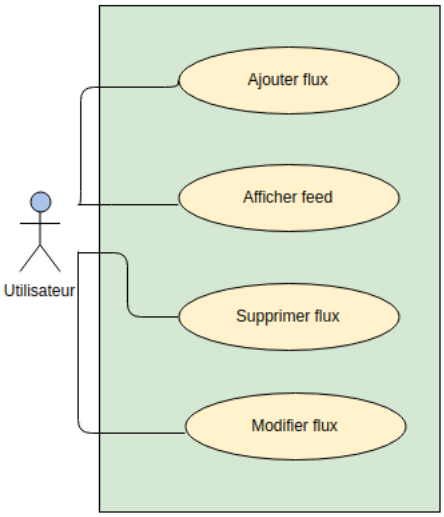


Figure 4.4: Diagramme de cas gestion des flux

Tableau 4.4: Tableau d'analyse du cas d'utilisation « Gérer flux »

Sommaire de Cas d'utilisation « Gérer flux »	
Titre	Gérer flux.
But	L'ajout de flux par le journaliste.
Acteur	Le journaliste
Description des enchaînements	
Pré-condition	**Post-condition**
L'authentification de La réceptionniste.	Le flux est ajouté, modifier ou supprimer.
Scénario nominal	
1. Le journaliste accède à son interface principale. 2. Le journaliste choisit la menue de Gérer flux. 3. Le journaliste clique sue le bouton « ajouter ». 4. Le journaliste remplit le formulaire par les informations nécessaires. 5. Le système enregistre les données dans la base des données. 6. Si l'ajout est effectué avec succès, la est affiché dans la liste des flux , si non un message d'erreur qui s'affiche. 7. Le journaliste peut de même modifier un flux. 8. Le journaliste peut de même supprimer un rendez-vous.	
Scénario d'échec	
1. Le journaliste entré une URL non valide. 2. Des champs obligatoires non remplis. 3. La réceptionniste n'est pas authentifiée	

4.3.1.3 Diagramme de cas d'utilisation « Gérer Raw Data »

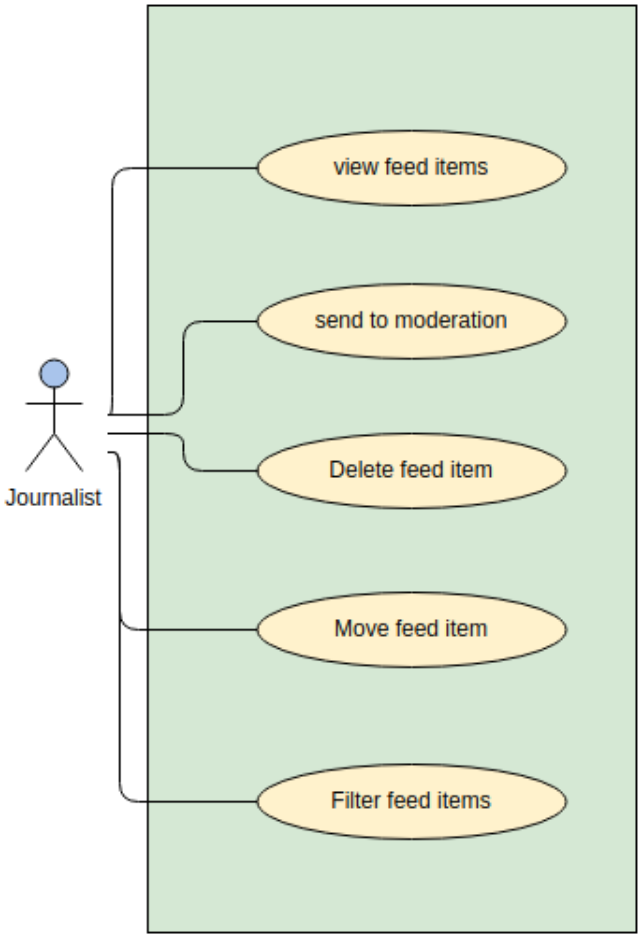


Figure 4.5: Diagramme de cas d'utilisation Gérer Raw Data

Le tableau 4.5 présente la description textuelle du cas d'utilisation Consulter un feed-item.

Tableau 4.5: Tableau d'analyse du cas d'utilisation « Consulter un feed-item »

Sommaire de Cas d'utilisation « Consulter un feed-item »	
Titre	Consulter un feed-item.
But	L'ajout d'une sources de flux par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Les détails de feed-item choisi sont affichés sur l'écran.
Scénario nominal	
1. L'utilisateur demande la consultation de la liste des feed-items. 2. Le système affiche la liste des feed-items. 3. L'utilisateur choisit un feed-ietm parmi la liste pour consulter ses détails. 4. Le système affiche les détails du feed-items.	

Le tableau 4.6 présente la description textuelle du cas d'utilisation Supprimer feed-item.

Tableau 4.6: Tableau d'analyse du cas d'utilisation « Supprimer feed-item »

Sommaire de Cas d'utilisation « Supprimer feed-item »	
Titre	Supprimer un feed-item.
But	La suppression d'un feed-item par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Le feed-item choisit est supprimé
Scénario nominal	
1. L'utilisateur demande la liste des feed-item 2. Le système affiche la liste des feed-item. 3. L'utilisateur choisit le feed-item à supprimer. 4. Le système vérifie la possibilité de suppression de feed-item choisit. 5. Le feed-item est supprimé.	
Scénario d'échec	
4. L'utilisateur choisit un feed-item qui ne peut pas être supprimé. 4-a-1- Le système affiche un message d'erreur. 4-a-2- Reprise à l'étape 3 de scénario nominal.	

Le tableau 4.8 présente la description textuelle du cas d'utilisation Déplacer feed-item.

Tableau 4.7: Tableau d'analyse du cas d'utilisation « Déplacer feed-item »

Sommaire de Cas d'utilisation « Déplacer feed-item »	
Titre	Déplacer un feed-item.
But	Déplacer un feed-item par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Le feed-item choisit est Déplacé
Scénario nominal	
1. L'utilisateur demande la liste des feed-item 2. Le système affiche la liste des feed-item. 3. L'utilisateur choisit le feed-item à Déplacer. 4. Le système vérifie la possibilité de glisser le feed-item choisit. 5. L'utilisateur glisse le feed-item à Déplacer. 6. L'utilisateur dépose le feed-item à l'endroit choisit dans la liste. 7. Le feed-item est Déplacé.	
Scénario d'échec	
4. L'utilisateur choisit un feed-item qui ne peut pas être Déplacé.	

4.3.1.4 Diagramme de cas d'utilisation «Gérer la modération des feed-items»

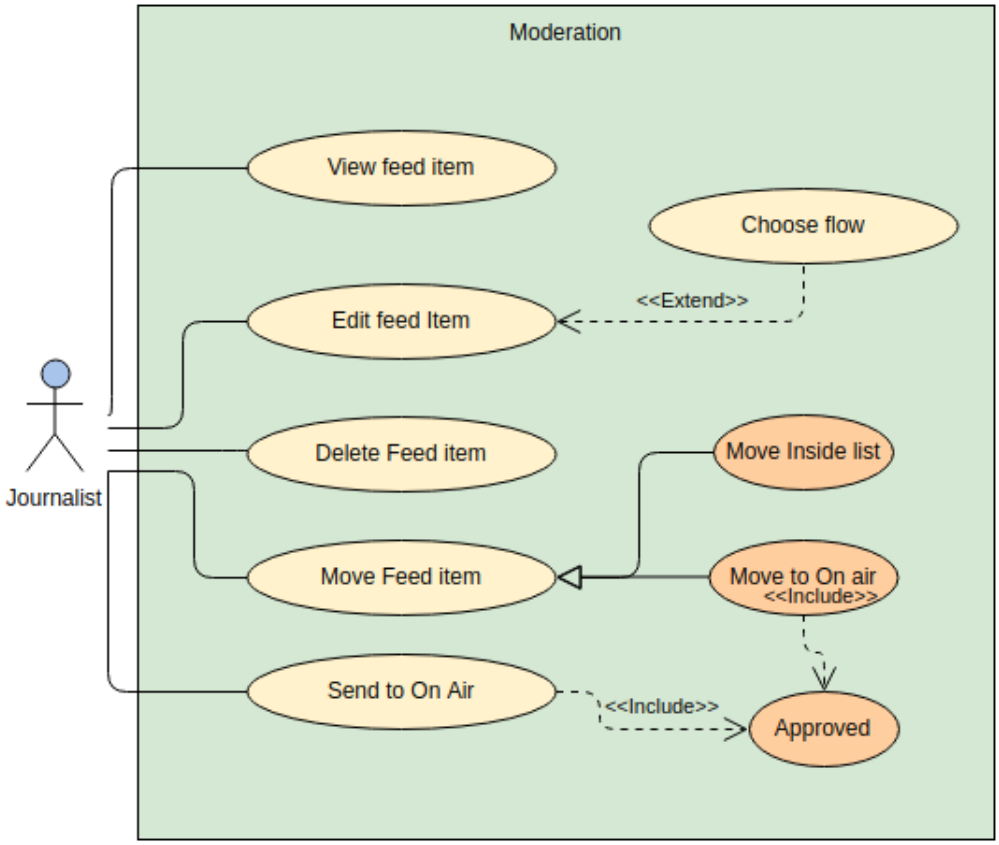


Figure 4.6: Diagramme de cas Gérer la modération des feed-items

Le tableau 4.8 présente la description textuelle du cas d'utilisation Déplacer feed-item vers on air.

Tableau 4.8: Tableau d'analyse du cas d'utilisation « Déplacer feed-item vers on air»

Sommaire de Cas d'utilisation « Déplacer feed-item vers on air»	
Titre	Déplacer un feed-item vers on air.
But	Déplacer un feed-item vers on air par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Le feed-item choisit est Déplacé vers on air
Scénario nominal	
1. L'utilisateur demande la liste des feed-item 2. Le système affiche la liste des feed-item. 3. L'utilisateur choisit le feed-item à Déplacer. 4. Le système vérifie la possibilité de glisser le feed-item choisit. 5. L'utilisateur glisse le feed-item à Déplacer. 6. L'utilisateur dépose le feed-item à la catégorie choisit dans on air. 7. Le feed-item est Déplacé.	
Scénario d'échec	
4. L'utilisateur choisit un feed-item qui non approuvé.	

Le tableau 4.9 présente la description textuelle du cas d'utilisation Modifier feed-item.

Tableau 4.9: Tableau d'analyse du cas d'utilisation « Modifier feed-item »

Sommaire de Cas d'utilisation « Modifier feed-item »	
Titre	Modifier un feed-item.
But	La modification d'un feed-item par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Les données du feed-item choisit sont bien mises à jour.
Scénario nominal :.	
1. L'utilisateur demande la liste des feed-item	
2. Le système affiche la liste des feed-item .	
3. L'utilisateur choisit le feed-item à modifier.	
4. Le système vérifie les données qui peuvent être modifié pour le feed-item choisit et affiche le formulaire de modification	
5. Le feed-item est est modifié.	
Scénario d'échec	
4. L'utilisateur saisie données manquantes ou erronés	
4-a-1- Le système affiche une erreur.	
4-a-2- Reprise à l'étape 3 de scénario nominal.	

Le tableau 4.10 présente la description textuelle du cas d'utilisation Supprimer feed-item.

Tableau 4.10: Tableau d'analyse du cas d'utilisation « Supprimer feed-item »

Sommaire de Cas d'utilisation « Supprimer feed-item »	
Titre	Supprimer un feed-item.
But	La suppression d'un feed-item par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Le feed-item choisit est supprimé
Scénario nominal	
1. L'utilisateur demande la liste des feed-item	
2. Le système affiche la liste des feed-item.	
3. L'utilisateur choisit le feed-item à supprimer.	
4. Le système vérifie la possibilité de suppression de feed-item choisit.	
5. Le feed-item est supprimé.	
Scénario d'échec	
4. L'utilisateur choisit un feed-item qui ne peut pas être supprimé.	
4-a-1- Le système affiche un message d'erreur.	
4-a-2- Reprise à l'étape 3 de scénario nominal.	

4.3.2 Diagrammes de séquence

À ce niveau, nous arrivons à présenter les diagrammes de séquence système des différents cas d'utilisation déjà expliqués :

4.3.2.1 Afficher la liste des feeds

La figure suivante présente le diagramme de séquence global de Médias Feeds.

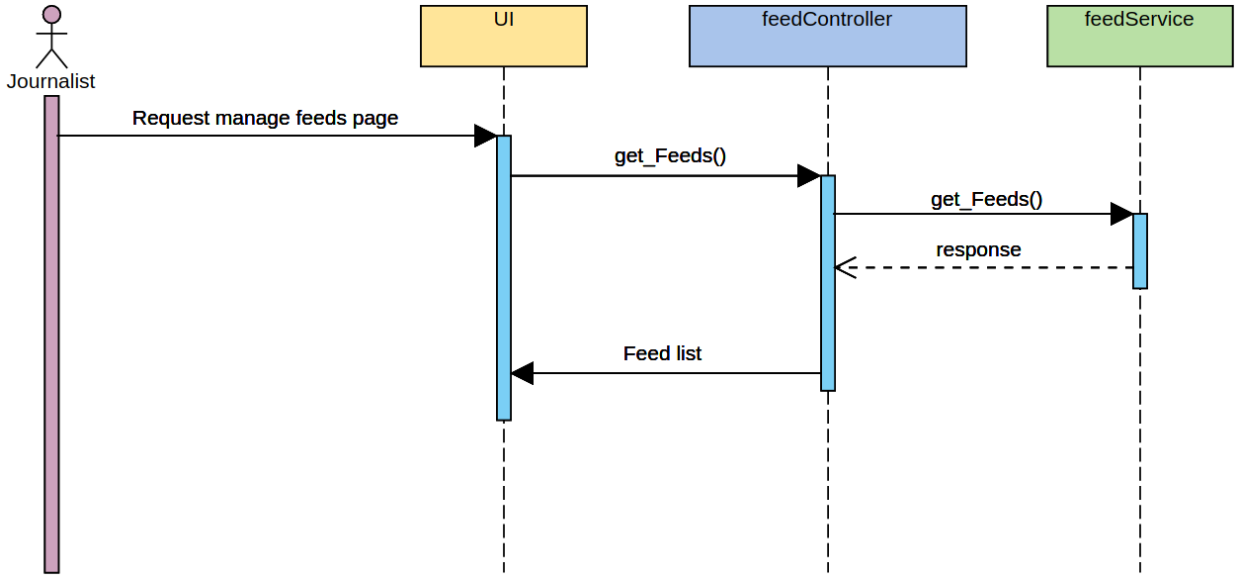


Figure 4.7: Diagramme de séquence global de Médias Feeds

4.3.2.2 Ajouter un nouveau feed

La figure suivante présente le diagramme de séquence ajouter un nouveau feed :

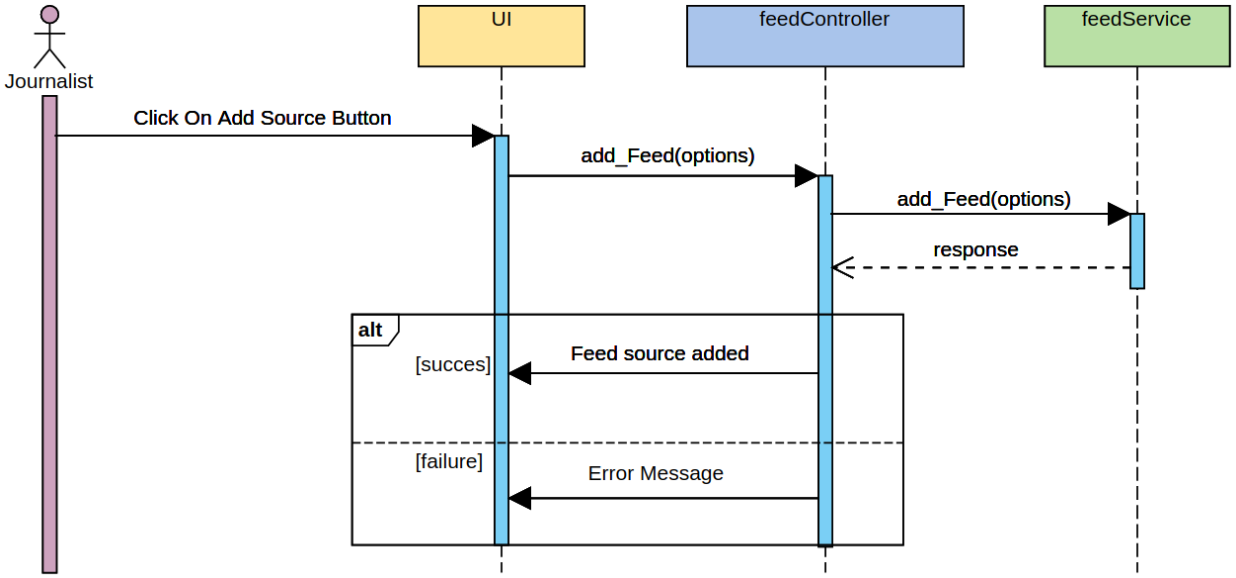


Figure 4.8: Diagramme de séquence « Ajouter un nouveau feed »

4.3.2.3 Mise a jours de feed

La figure suivante présente le diagramme de séquence mise à jours de feed :

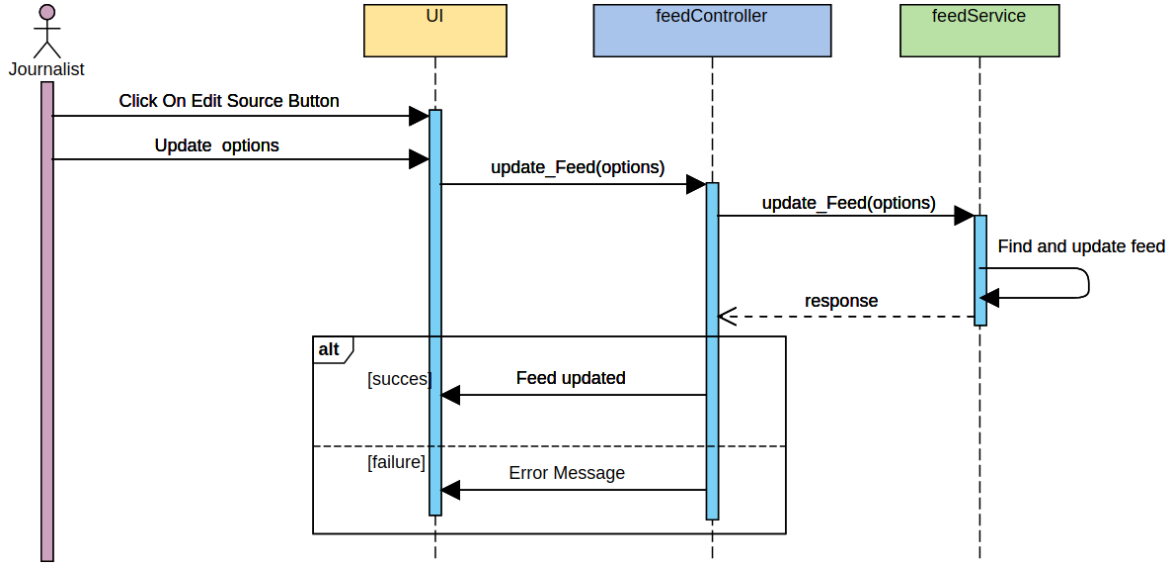


Figure 4.9: Diagramme de séquence « Mise à jours de feed »

4.3.2.4 Supprimer feed

La figure suivante présente le diagramme de séquence supprimer feed :

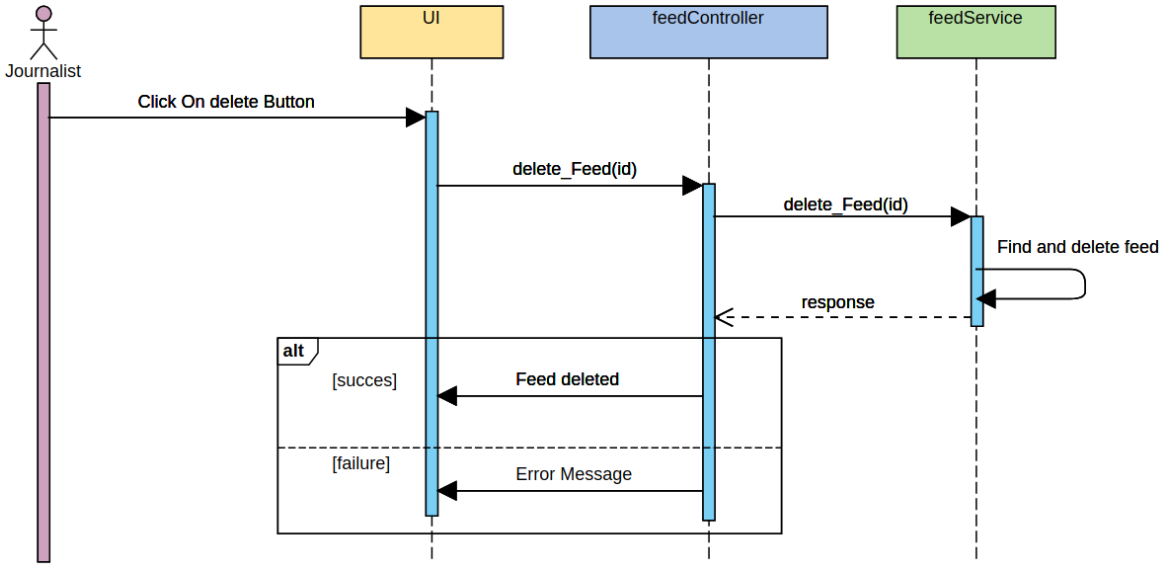


Figure 4.10: Diagramme de séquence « Supprimer feed »

4.3.2.5 Démarrer feed

La figure suivante présente le diagramme de séquence démarrer feed feed :

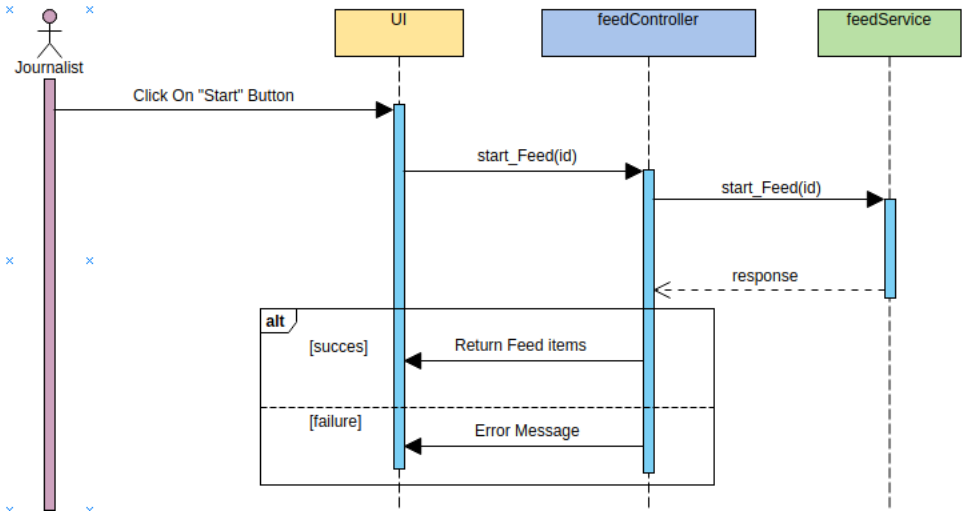


Figure 4.11: Diagramme de séquence « Démarrer feed »

4.3.3 Diagramme de classe

La figure suivante présente le diagramme de classe de sprint 2 :

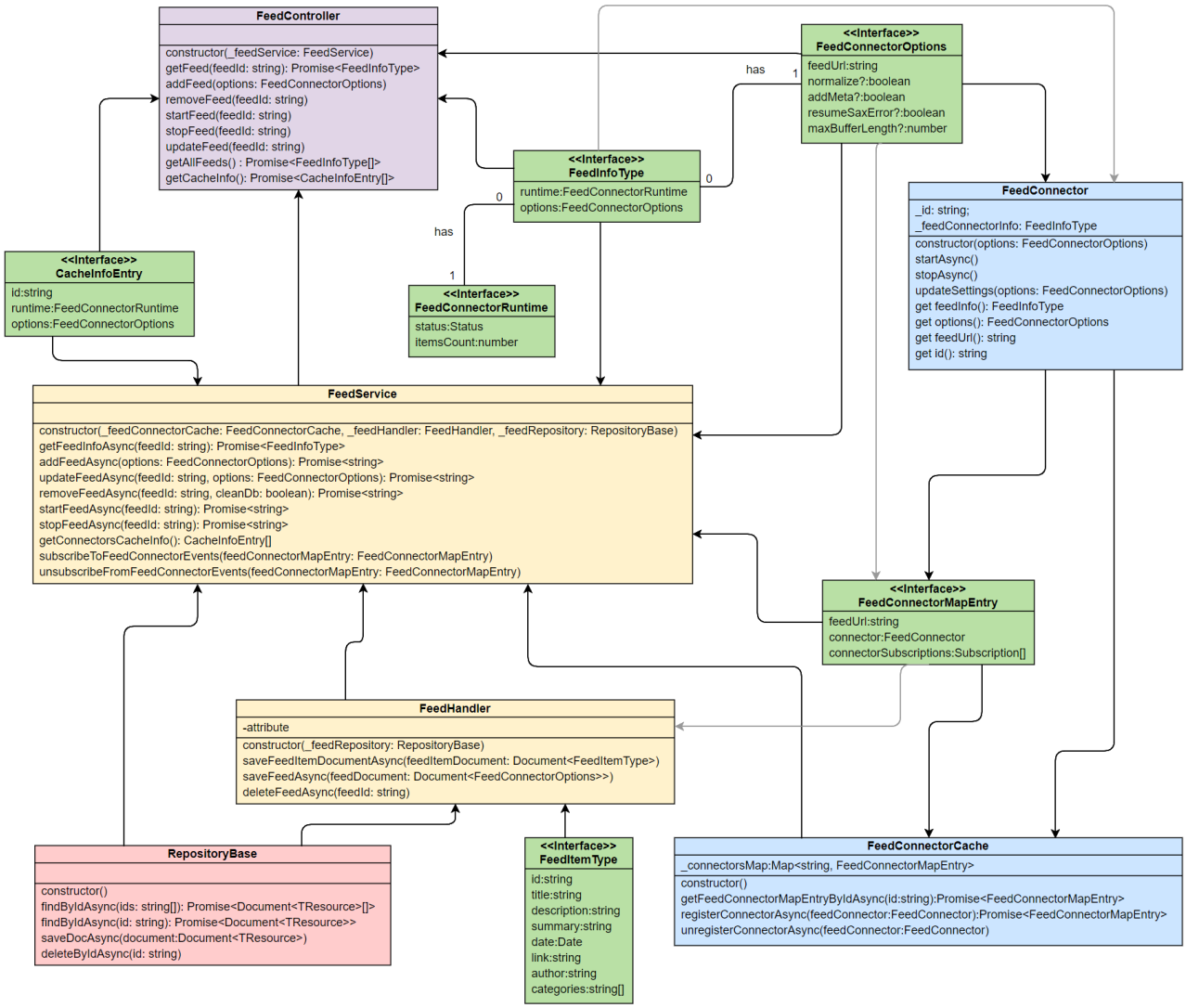


Figure 4.12: diagramme de classe de sprint 2

4.4 Réalisation

Dans cette section, nous allons présenter quelques interfaces qui concerne le Sprint 2.

Interface de Média

Cette interface permet à l'utilisateur du site de charger, gérer, modérer,Glisser et déposer les feed-items entre les colonnes afin d'envoyer finalement en Air.

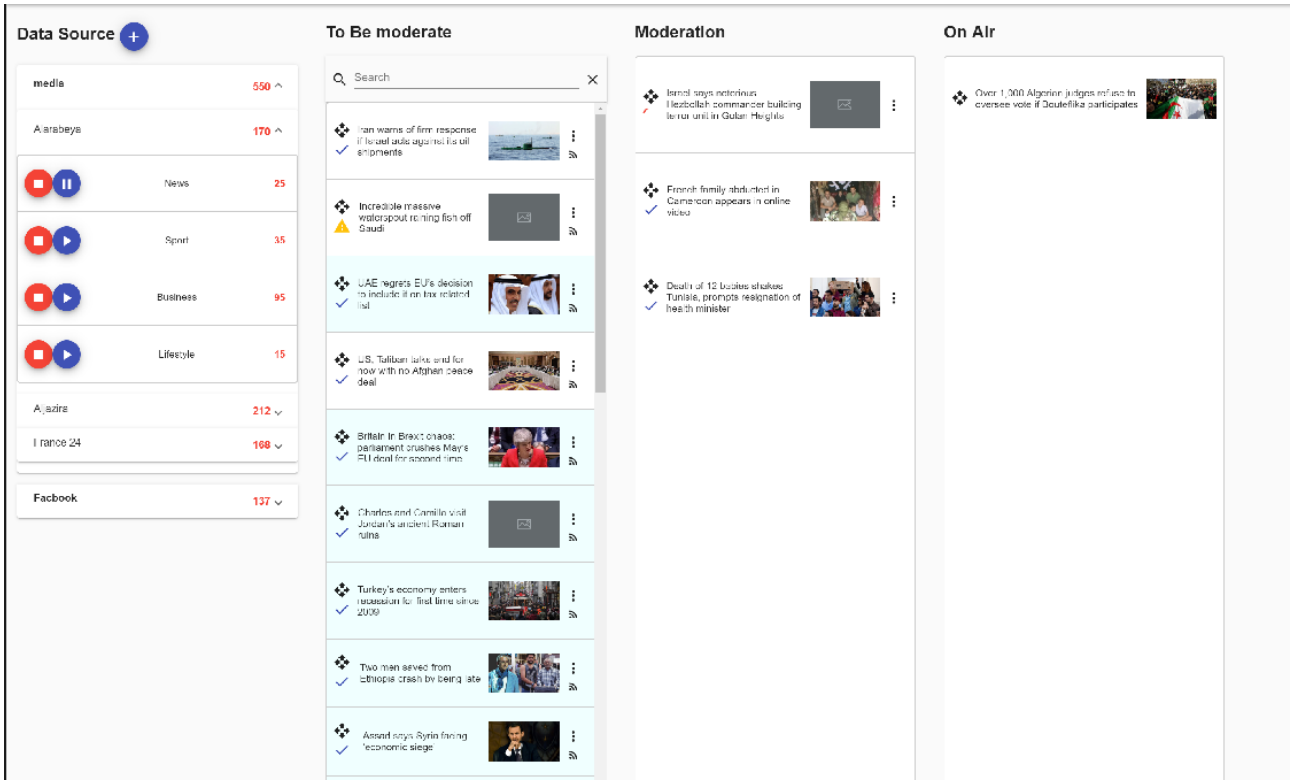


Figure 4.13: Interface Média

Cette interface permet à l'utilisateur du site de voir les détails d'un feed-items.

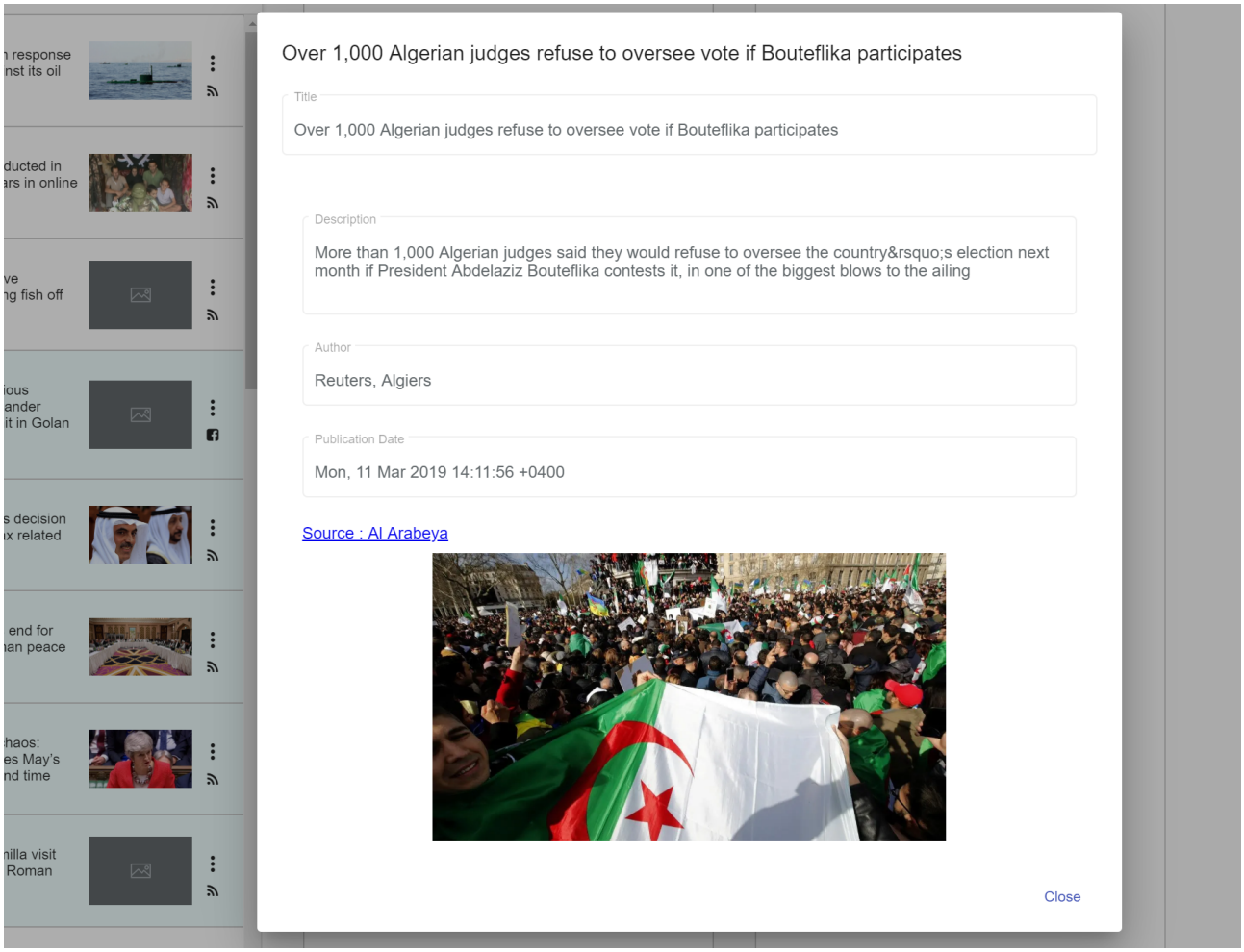


Figure 4.14: Interface détails de feed-item

Cette interface permet à l'utilisateur du site de éditer les détails d'un feed-items.

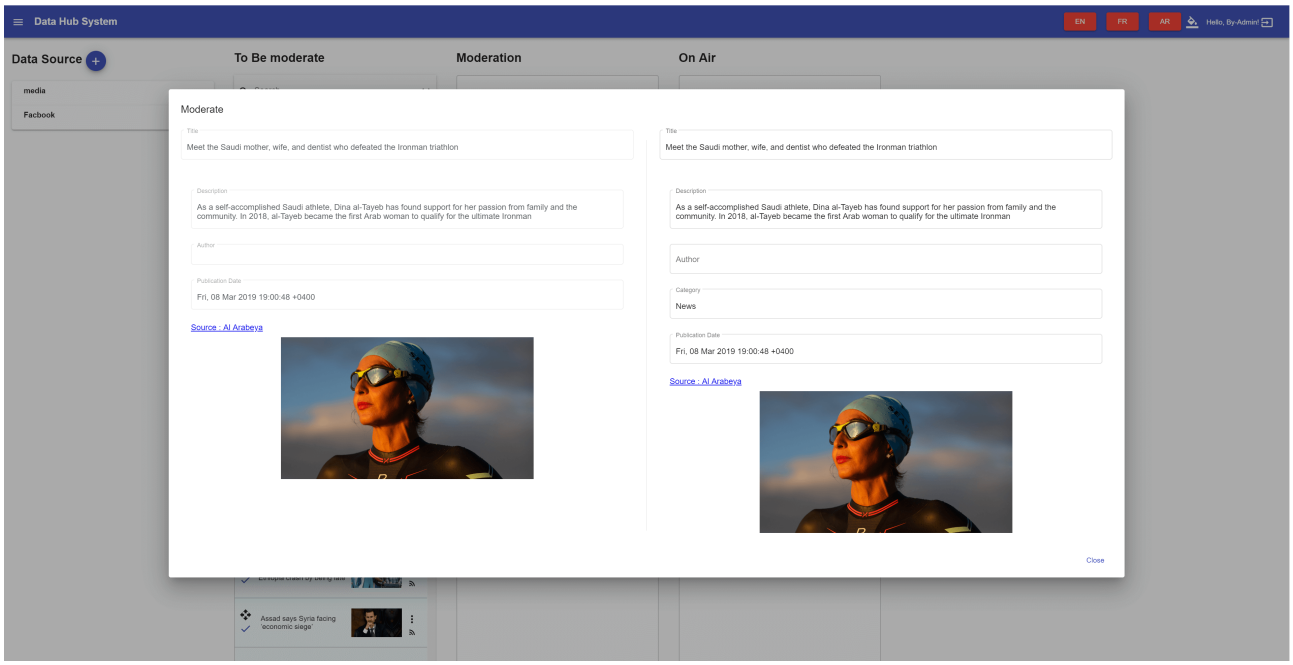


Figure 4.15: Interface éditer feed-item

Cette interface permet à la gestionnaire de médias du site de de modérer les feed-items passer par le journaliste dans la colonne "Modération". Il peut effectuer plusieurs opérations sur les feed-items soit de voir les détails puis le valide en par glisser et déposer le feed-item dans la colonne de Approuvée ou bien d'ajouter quelque modification ou bien de supprimer le feed items dans le menu de contexte de chaque feed-item.

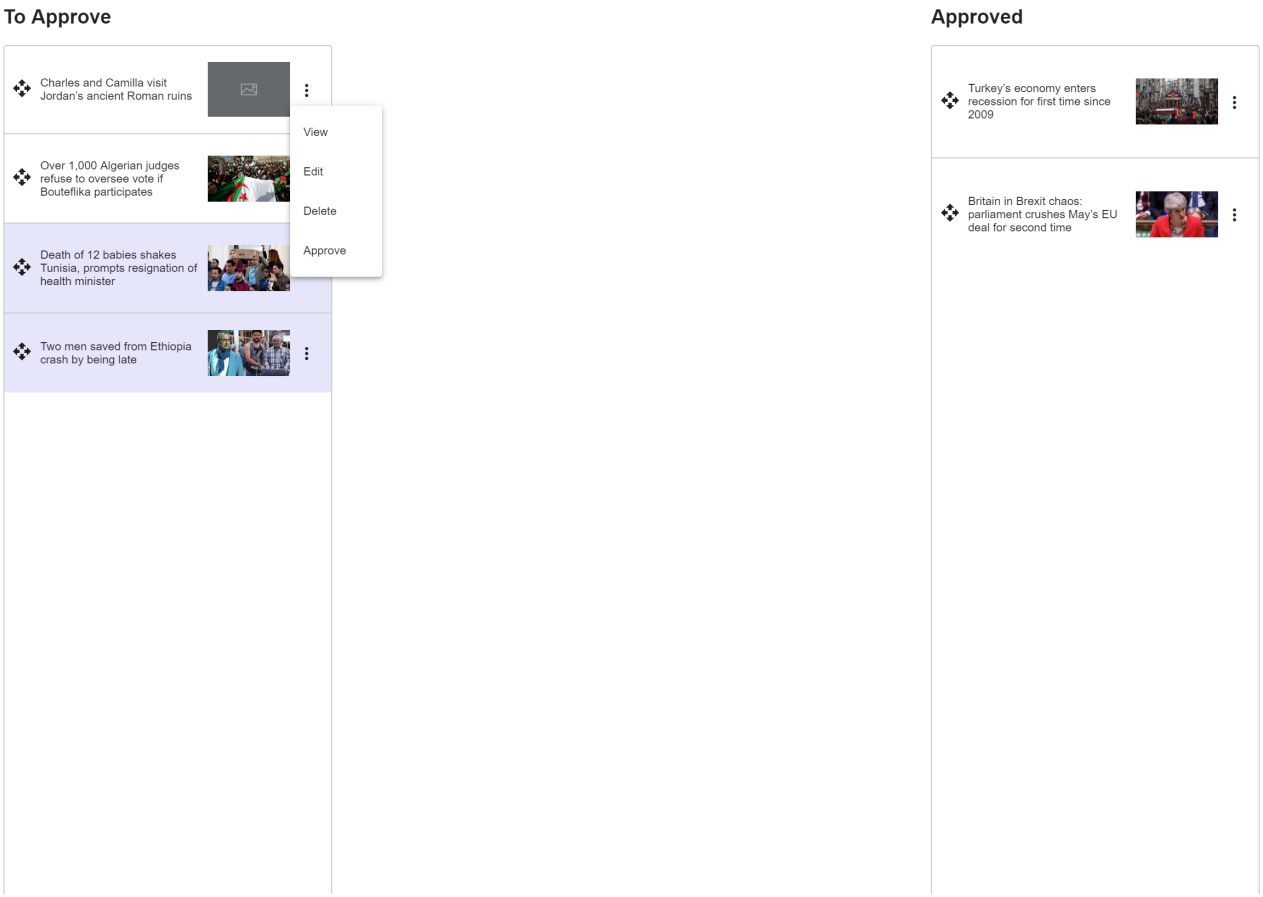


Figure 4.16: Interface Modérer feed-item

Interface de gestion des catégories

Cette interface permet à l'utilisateur du site de gérer les catégories.



Figure 4.17: Interface gérer les catégories

4.5 Conclusion

Dans cette partie nous avons réalisé et testé le deuxième sprint du projet. Dans la partie suivante on va attaquer, avec la même approche le troisième sprint de ce projet.

Chapitre 5

Sprint 3 : Facebook Live Streamer

Plan

1 Backlog du sprint 56

2 Analyse 56

3 Conception 58

4 Réalisation 63

5 Conclusion 65

Introduction

Ce sprint a pour but de développer le module de notre application qui est le Facebook Live Streamer.Notre application donne la possibilité de lancer la diffusion en direct de deux manières différentes :

— Le lancement la diffusion en direct.

— La Planification de la diffusion en direct.

5.1 Backlog du sprint

ID	Histoire utilisateur
0.1	Créer le composant de gestion des comptes
0.2	Créer le composant de diffusion en direct
0.3	Créer le service de gestion des comptes
0.4	Créer le service de diffusion en direct
0.5	Ajouter Le planification de diffusion en direct
0.6	Créer le service de planification de diffusion en direct

5.2 Analyse

Après avoir étudié les différentes tâches du sprint 3, nous avons établi quelques maquettes afin de les valider avec le Product Owner.

La figure 5.1 présente l'interface qui permet de gérer les compte Facebook et envoyer le jeton d'accès vers le backend.

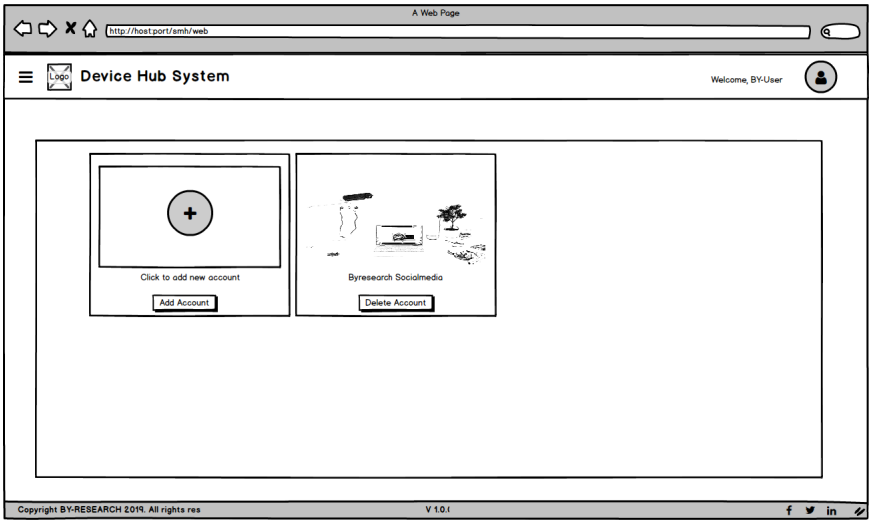


Figure 5.1: Maquette de gestion des comptes Facebook.

La figure 5.2 présente l'interface qui permet de lancer la diffusion en direct en spécifiant le titre et la description de la diffusion en direct puis le affiche le Permalien de vidéo en direct sur Facebook.

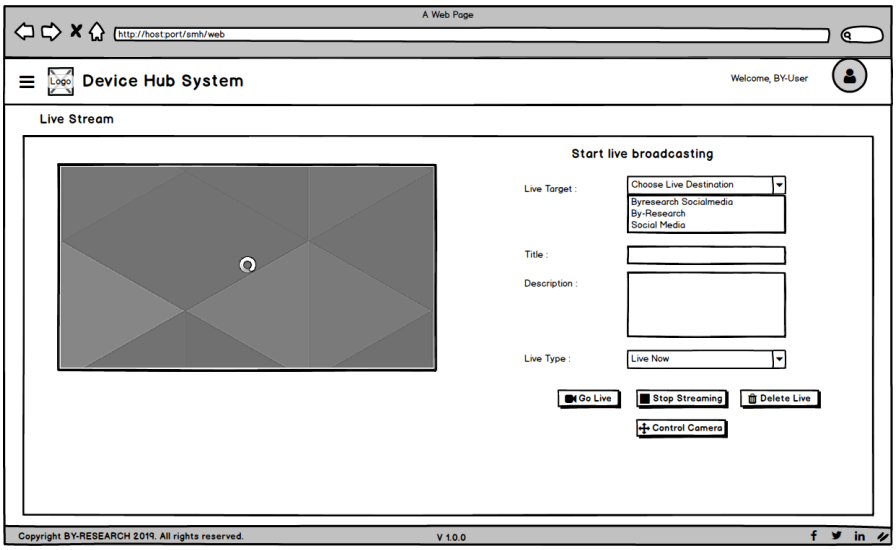


Figure 5.2: Maquette de la diffusion en direct.

La figure 5.3 présente l'interface qui permet de planifier la diffusion en direct en spécifiant le titre, la description, la date et le temps de la diffusion en direct puis il envois une notifications sur Facebook pour tous les abonnées concernons la planification de la diffusion .

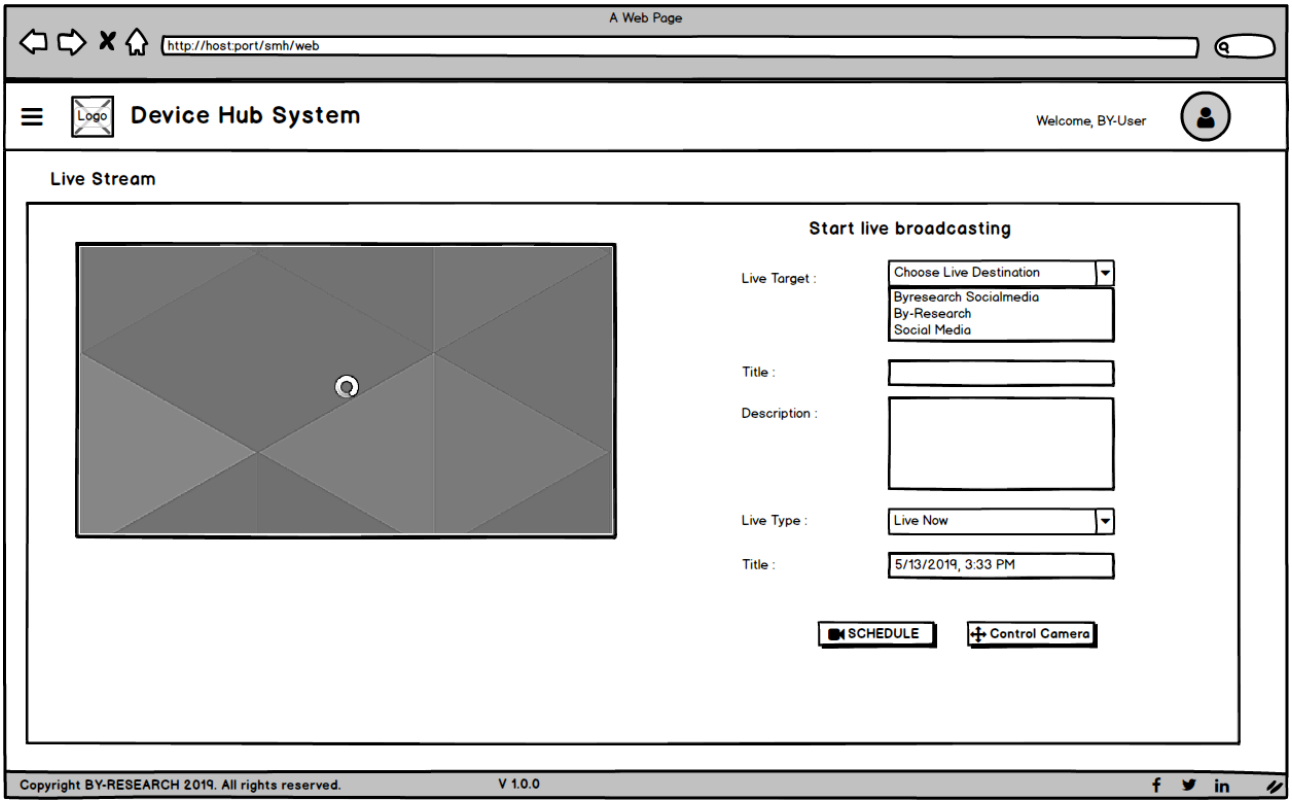


Figure 5.3: Maquette de la planification de diffusion en direct.

5.3 Conception

5.3.1 Raffinement des cas d'utilisation

Dans cette section, nous allons présenter un raffinement de quelque cas d'utilisation du diagramme global.

5.3.1.1 Diagramme de cas d'utilisation « Gérer live stream »

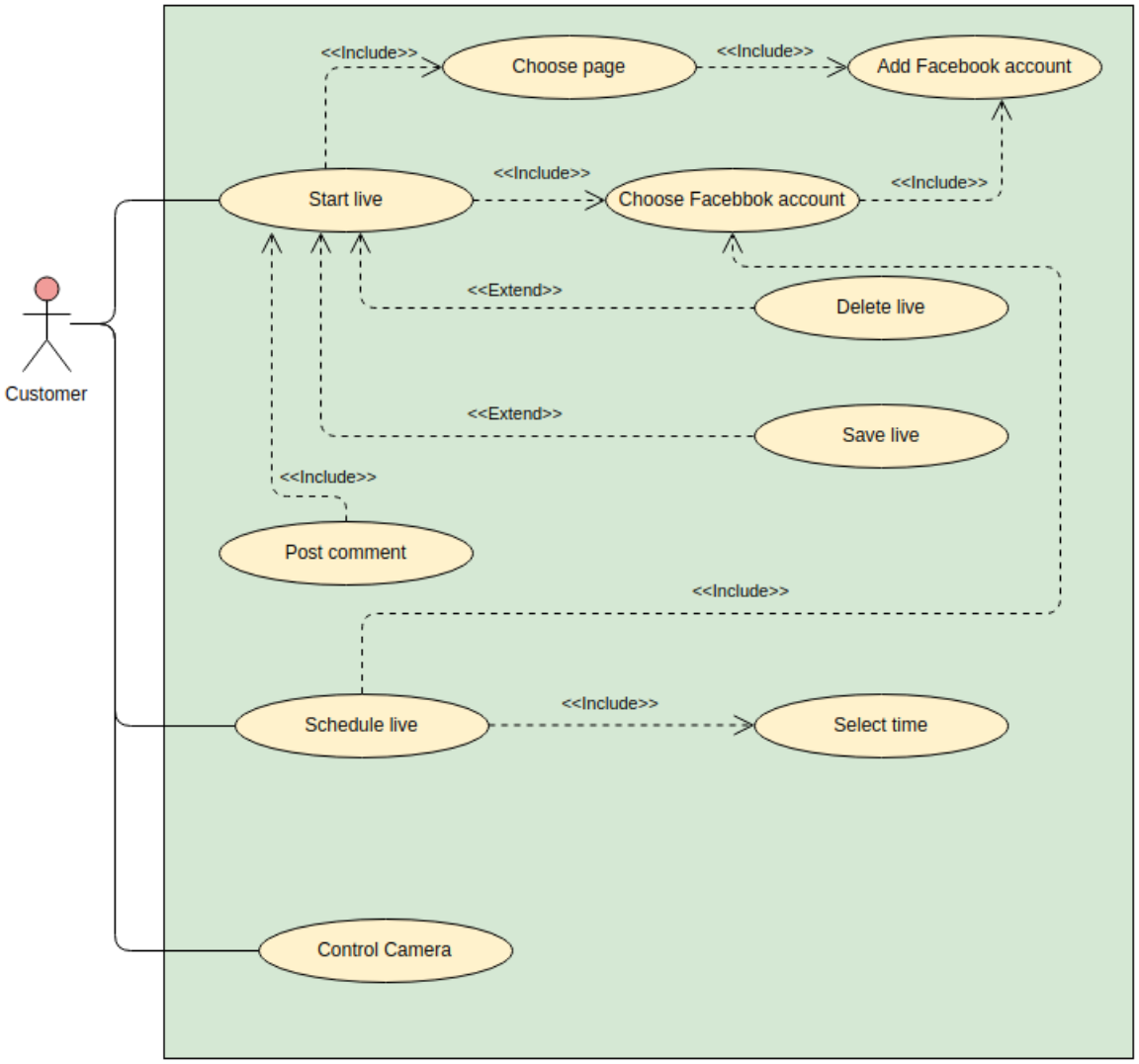


Figure 5.4: Diagramme de cas Gérer live stream

Le tableau 5.1 présente la description textuelle du cas d'utilisation Démarrer la diffusion en direct.

Tableau 5.1: Tableau d'analyse du cas d'utilisation « Démarrer la diffusion en direct »

Sommaire de Cas d'utilisation « Démarrer la diffusion en direct »	
Titre	Démarrer la diffusion en direct.
But	Démarrer la diffusion en direct par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	le diffusion en direct est démarre
Scénario nominal	
1. L'utilisateur demande le page de diffusion en direct. 2. Le système affiche la liste des pages Facebook. 3. L'utilisateur choisit la page cible pour la diffusion en direct. 3. L'utilisateur saisie le titre, la description et le type de diffusion en direct. 4. Le système vérifie la possibilité de démarré le diffusion en direct. 5. Le diffusion en direct est commencé.	
Scénario d'échec	
1. Le jeton d'accès d'utilisateur Facebook est expiré . 4. L'utilisateur choisit une page qu'ont peut pas démarré la diffusion sur elle.	

5.3.2 Diagrammes de séquence

5.3.2.1 Ajouter compte Facebook

La figure suivante présente le diagramme de séquence Ajouter compte Facebook :

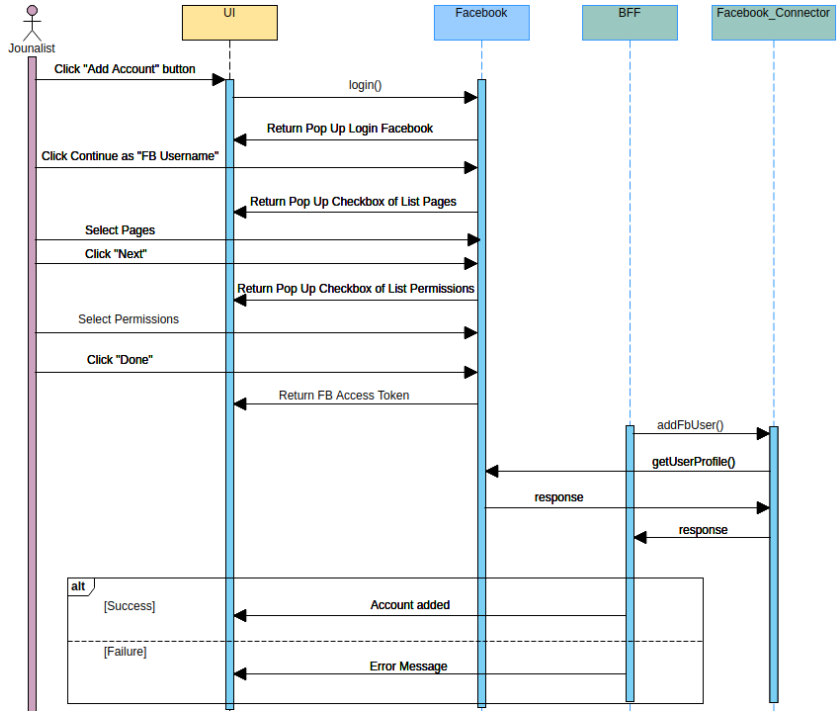


Figure 5.5: Diagramme de séquence gestion des comptes Facebook

5.3.2.2 Apporter liste des pages Facebook

La figure suivante présente le diagramme de séquence apporter liste des pages Facebook :

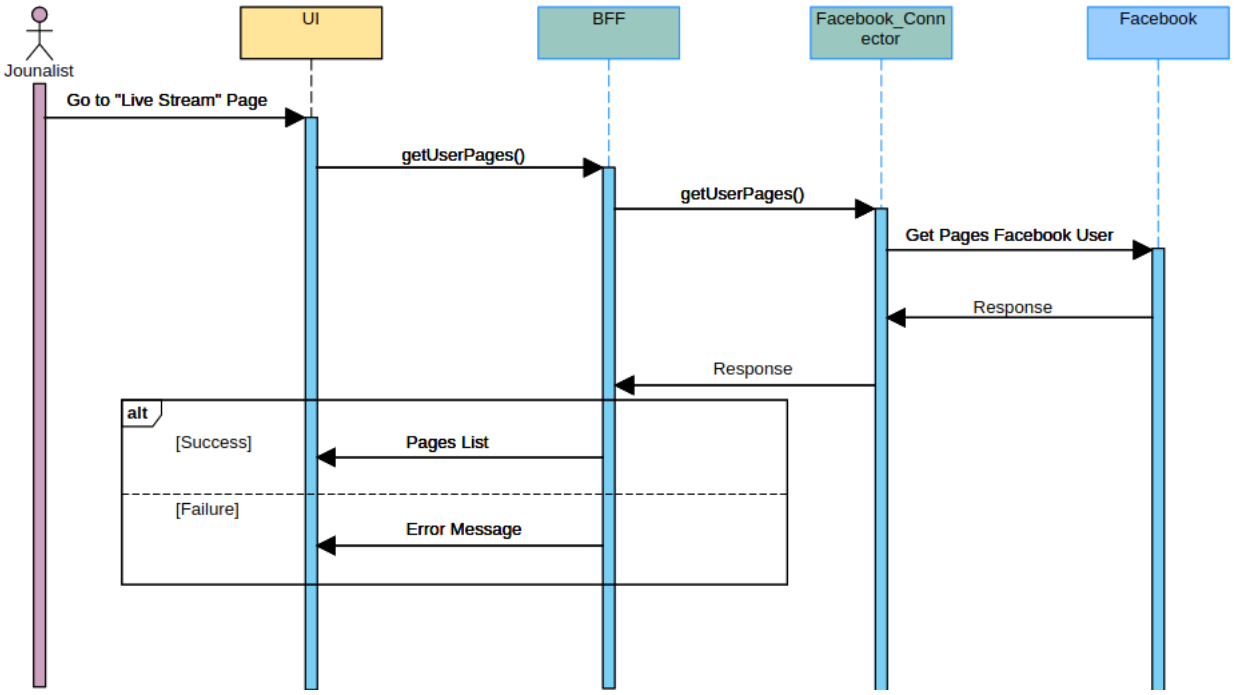


Figure 5.6: Diagramme de séquence Apporter liste des pages Facebook

5.3.2.3 Démarrer la diffusion en direct

La figure suivante présente le diagramme de démarrer la diffusion en direct :

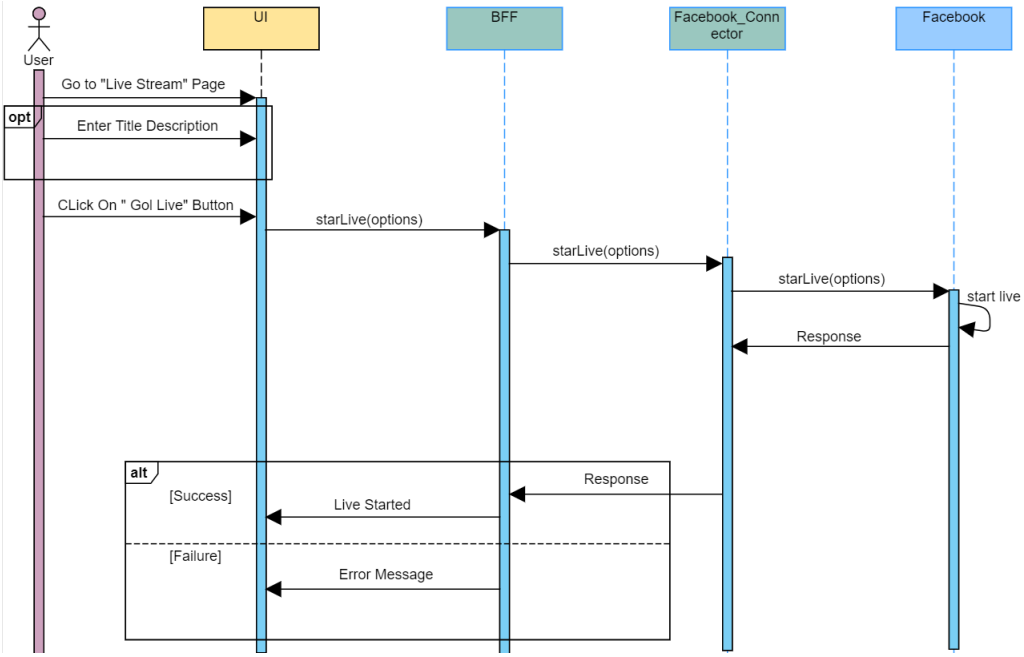


Figure 5.7: Diagramme de séquence Démarrer la diffusion en direct

5.3.2.4 Arrêter la diffusion en direct

La figure suivante présente le diagramme de Arrêter la diffusion en direct :

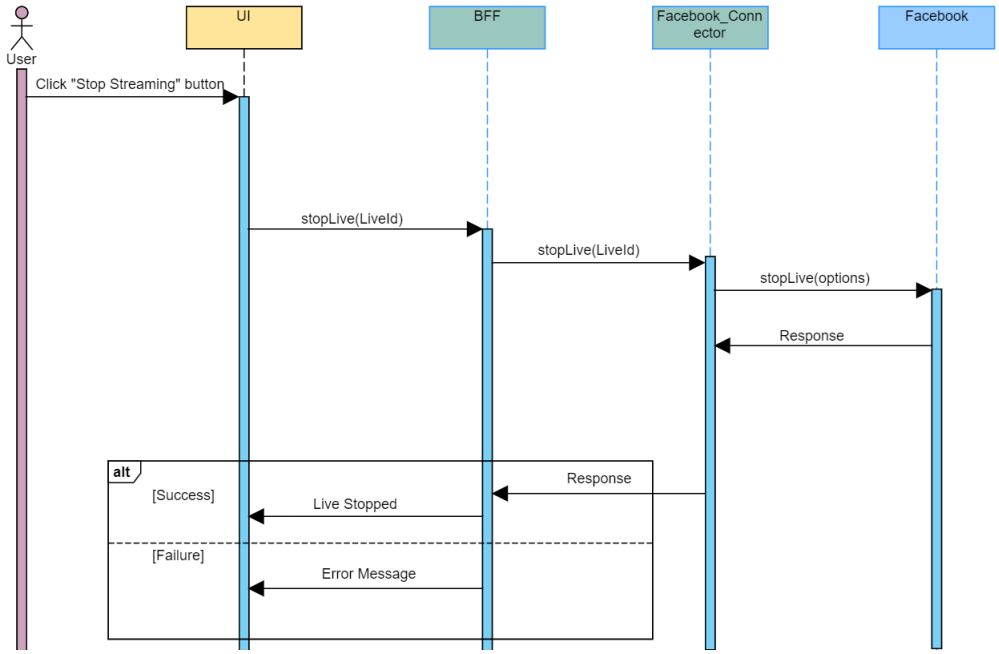


Figure 5.8: Diagramme de séquence Arrêter la diffusion en direct

5.3.2.5 Supprimer la diffusion en direct

La figure suivante présente le diagramme de Supprimer la diffusion en direct :

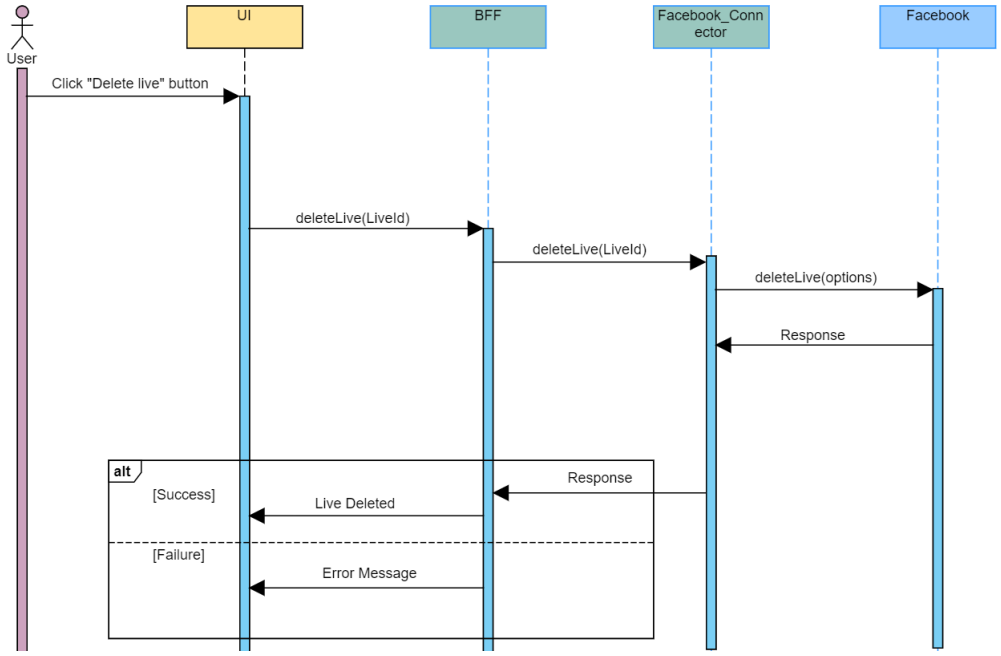


Figure 5.9: Diagramme de séquence Supprimer la diffusion en direct

5.3.3 Diagramme de classe

La figure suivante présente le diagramme de classe de sprint 3 :

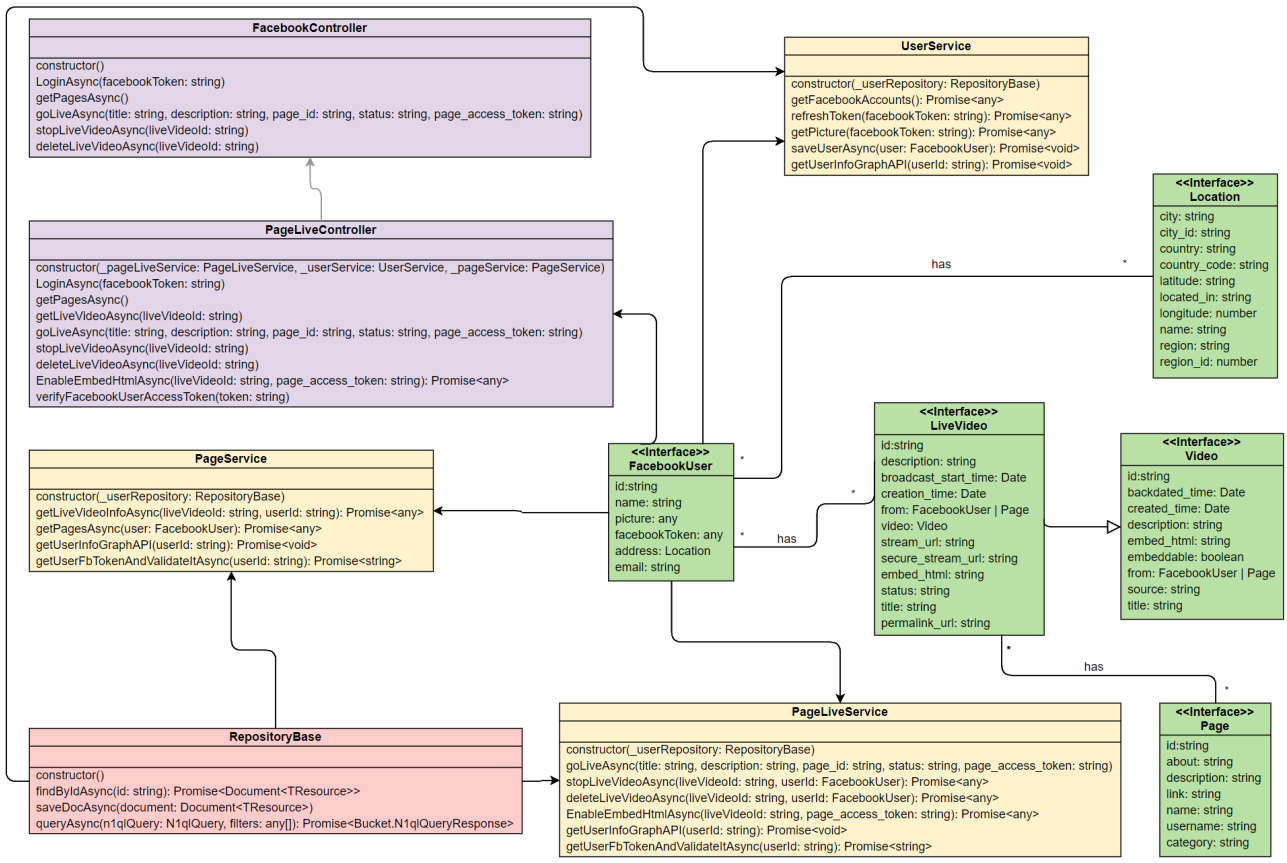


Figure 5.10: diagramme de classe de sprint 3

5.4 Réalisation

Dans cette section, nous allons présenter quelques interfaces qui concerne le Sprint 3.

Interface de gestion des comptes sociaux

Cette interface permet à l'utilisateur du site gérer les comptes sociaux Facebook.

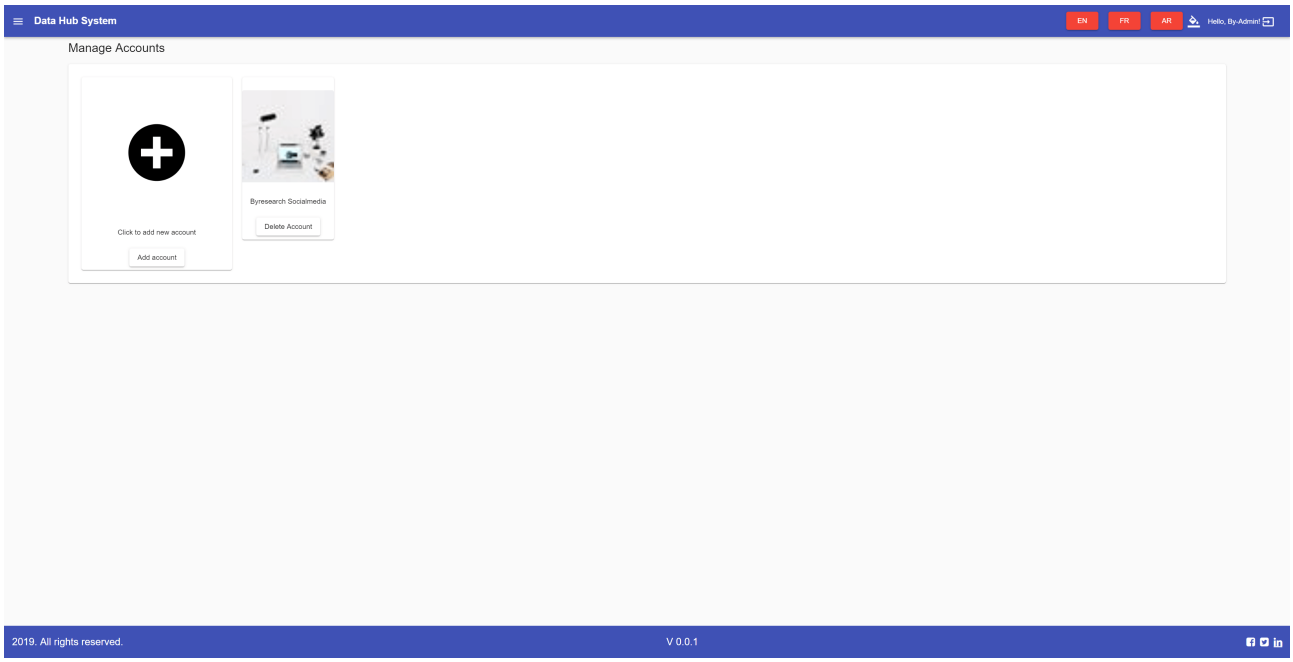


Figure 5.11: Interface gérer les compte social média

Interface de studio de diffusions

Cette interface permet à l'utilisateur du site lancer ou bien planifier des vidéos de diffusion en direct aussi la possibilités de postulé des commentaire sur la vidéo en live.

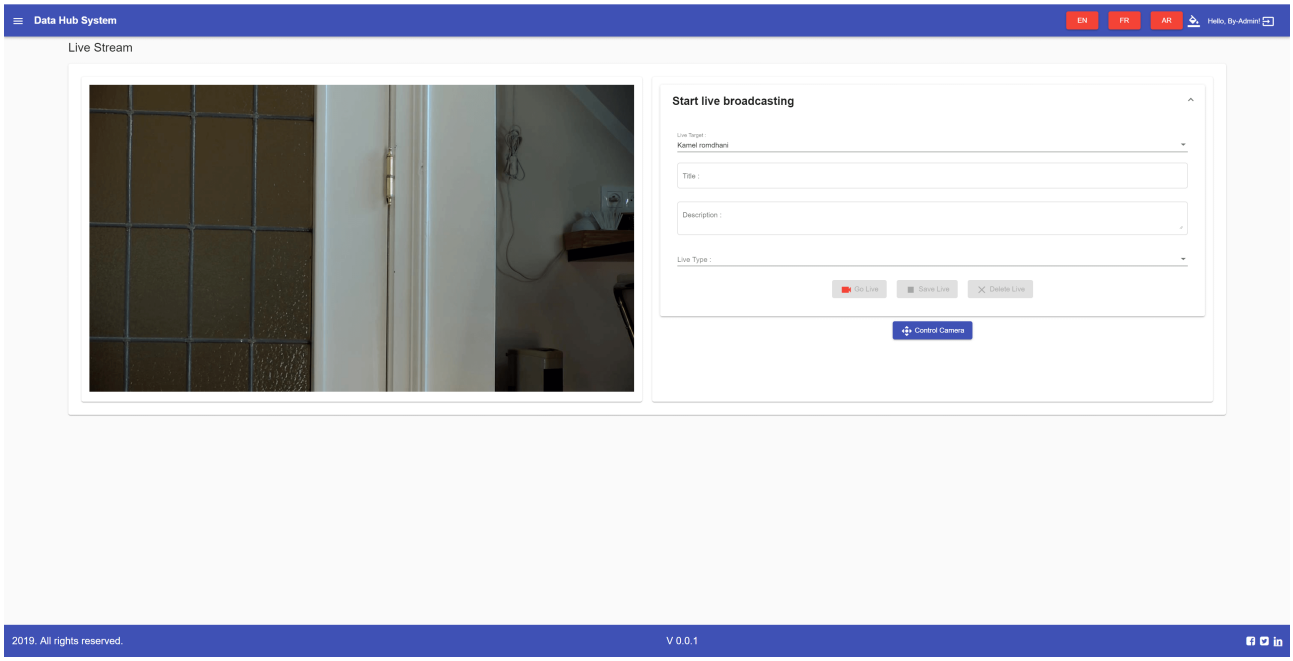


Figure 5.12: Interface diffusion en direct

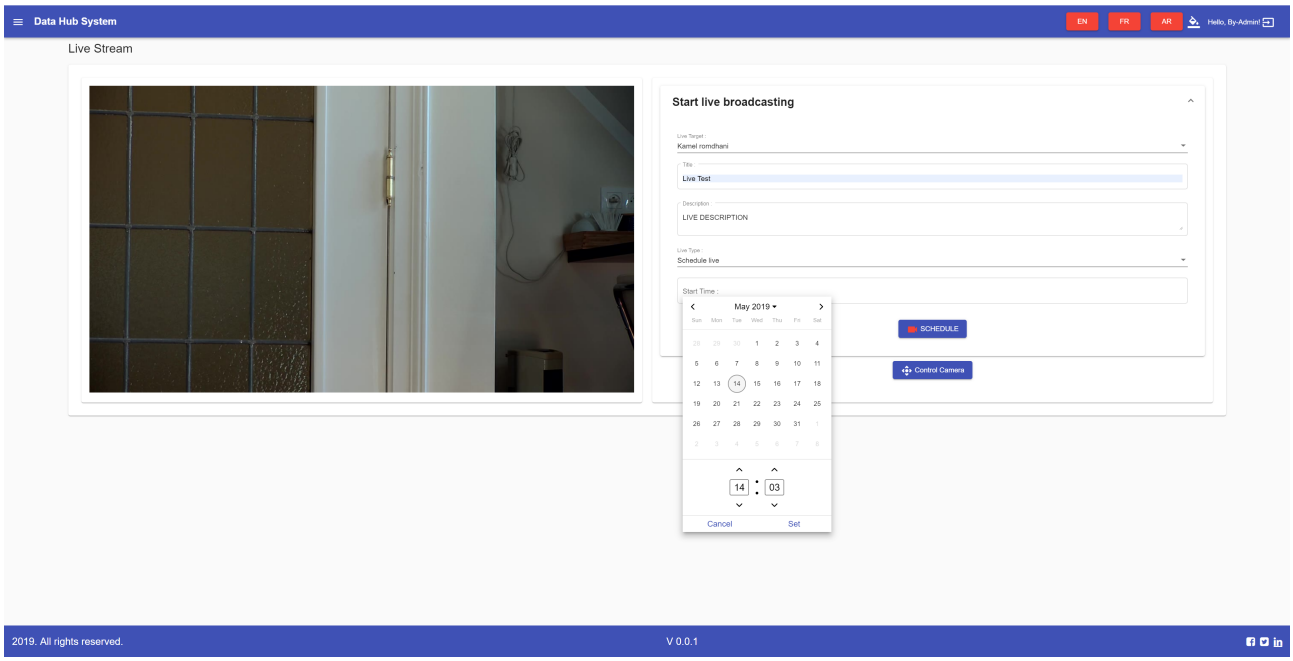


Figure 5.13: Interface planification de diffusion en direct

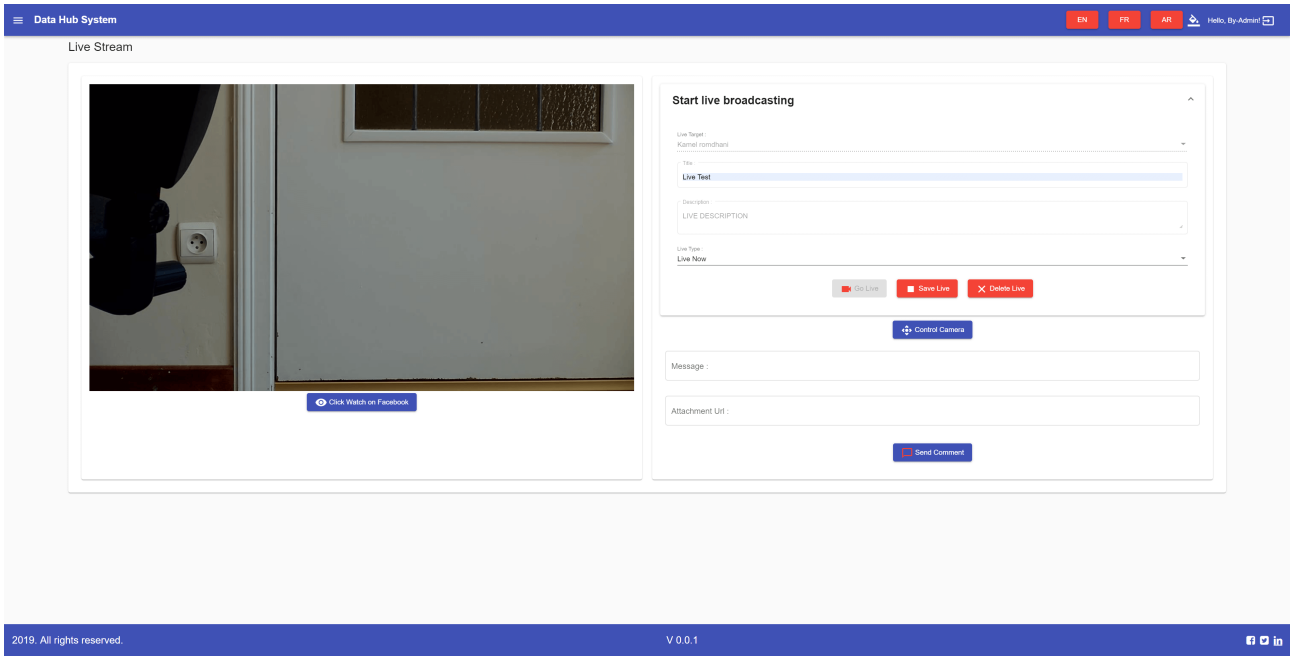


Figure 5.14: Interface d'ajout des commentaire sur la vidéo de diffusion en direct

Interface de gérer les destination de la diffusion en direct

Cette interface permet à l'utilisateur du site d'ajouter des destination de la diffusion en direct.

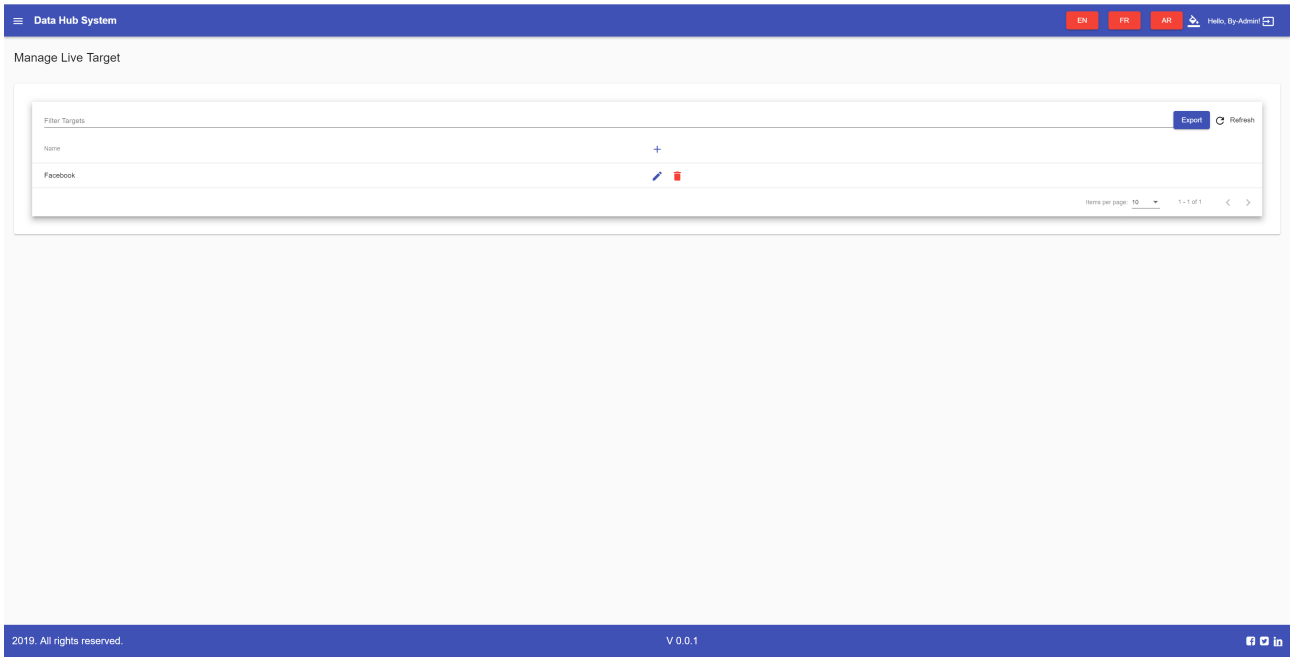


Figure 5.15: Interface gérer les destination de la diffusion en direct

5.5 Conclusion

Dans cette partie nous avons finalisé le troisième sprint de notre projet avec lequel nous avons terminé le module de Facebook Live Streamer. Il reste maintenant la contrôle de caméra et gestion de

modèle des données.

Chapitre 6

Sprint 4 : Contrôle de caméra et gestion de modèle des données

Plan

1 Backlog du sprint 68

2 Analyse 68

3 Conception 69

4 Réalisation 74

5 Conclusion 79

Introduction

Après avoir finir les modules de diffusion en direct on arrive maintenant à présenter le contrôle de la caméra PTZ. On commence tout d'abord à lister les différents contrôles dans le sprint backlog puis la conception et on finit avec la réalisation

6.1 Backlog du sprint

ID	Histoire utilisateur
0.1	Mise à place les composants d'affichage le flux de caméra
0.2	Conversion le flux RTSP en flux HTTP
0.3	Créer les composants de contrôle
0.4	Créer le service de contrôle
0.5	Créer la partie d'enregistrement de presets
0.6	Intégration ce module composant dans la partie de diffusion

6.2 Analyse

Après avoir étudié les différentes tâches du sprint 4, nous avons établi quelques maquettes afin de les valider avec le Product Owner.

La figure 6.15 présente l'interface qui permet de contrôler la caméra PTZ.

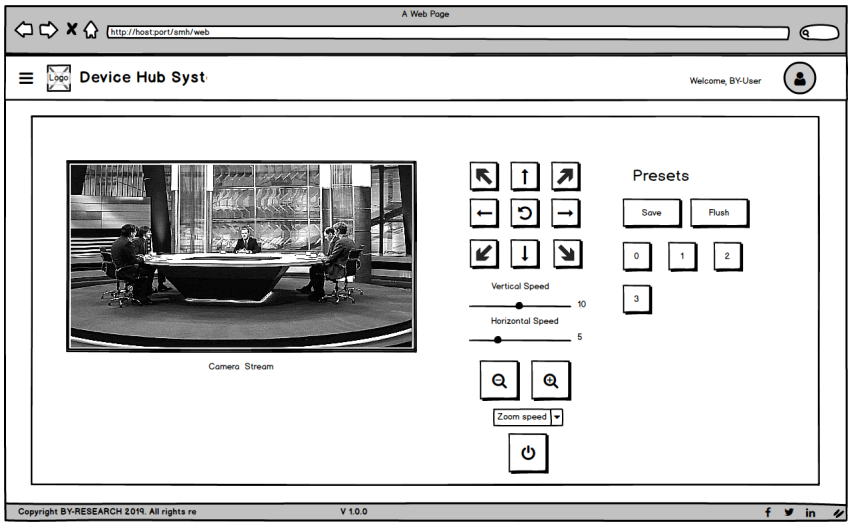


Figure 6.1: Maquette de contrôle de la caméra PTZ.

La figure 6.2 présente l'interface qui permet de contrôler la caméra PTZ a partir de l'interface de diffusion.

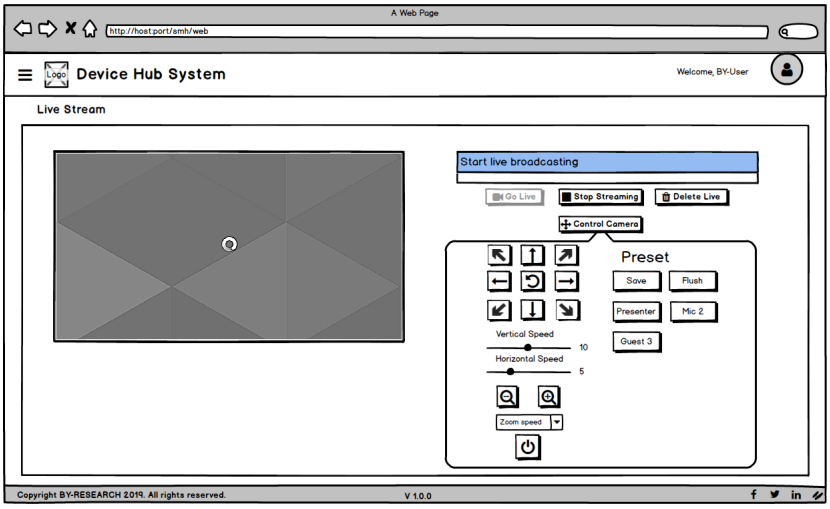


Figure 6.2: Maquette de l'intégration de contrôle dans la diffusions en direct

6.3 Conception

6.3.1 Raffinement des cas d'utilisation

Dans cette section, nous allons présenter un raffinement de quelque cas d'utilisation du diagramme global.

6.3.1.1 Diagramme de cas d'utilisation « Gérer Caméra »

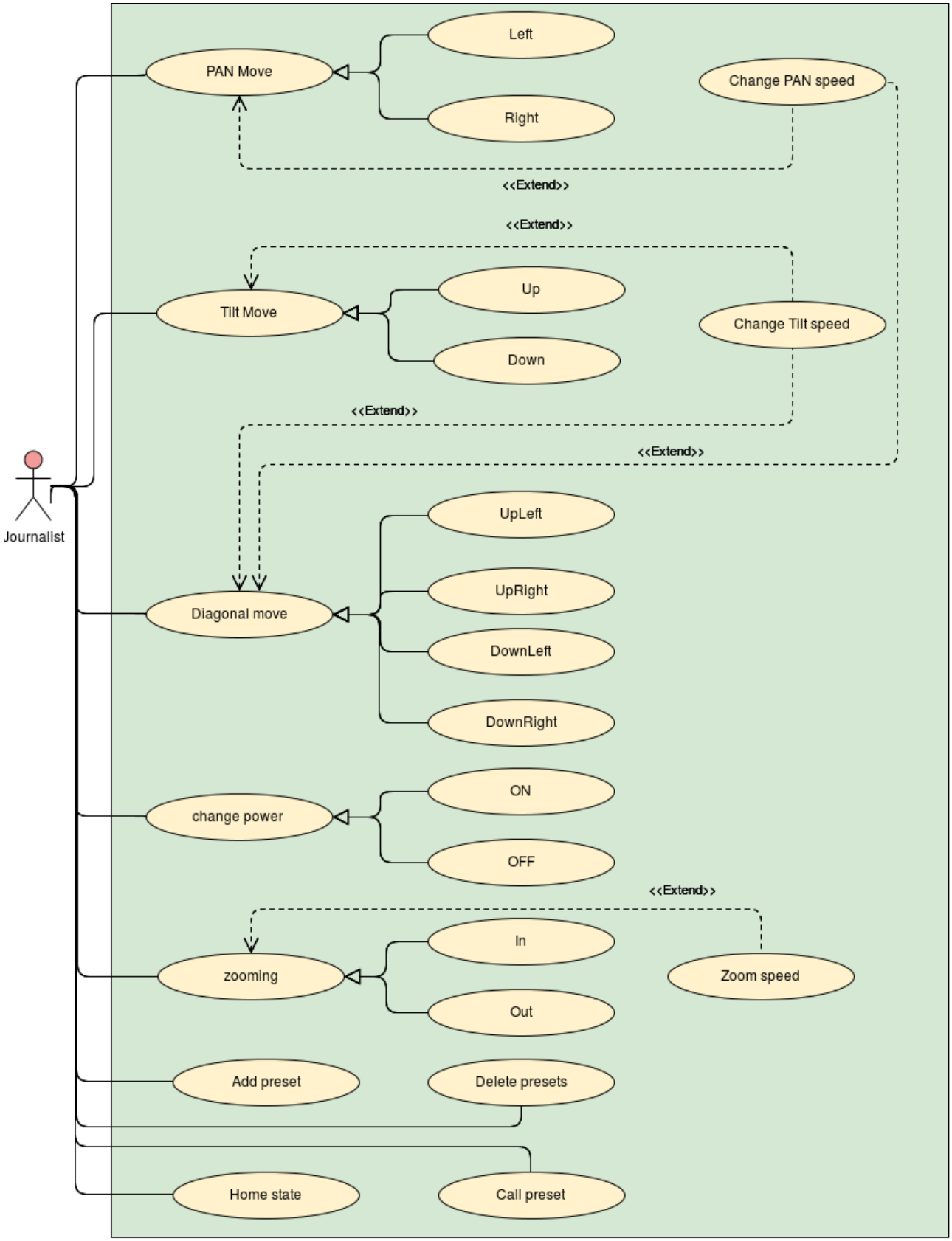


Figure 6.3: Diagramme de cas Gérer Caméra

Le tableau 6.1 présente la description textuelle du cas d'utilisation Ajouter preset.

Tableau 6.1: Tableau d'analyse du cas d'utilisation « Ajouter preset »

Sommaire de Cas d'utilisation « Ajouter preset »	
Titre	Ajouter une preset.
But	L'ajout d'une preset par l'utilisateur.
Acteur	Tous les acteurs
Description des enchaînements	
Pré-condition	**Post-condition**
Authentification	Un nouveau preset est ajouté
Scénario nominal	
1. L'utilisateur demande le formulaire d'ajout d'un preset. 2. Le système affiche le formulaire. 3. L'utilisateur entrée le nom de preset. 4. Le système vérifie les données saisies 5. Le système enregistre une nouvelle preset	
Scénario d'échec	
4. L'utilisateur saisie des données manquantes ou erronés. 4-a-1- Le système affiche une erreur. 4-a-2- Reprise à l'étape 3 de scénario nominal.	

6.3.2 Diagrammes de séquence

6.3.2.1 Déplacer la caméra

La figure suivante présente le diagramme de séquence déplacer la caméra :

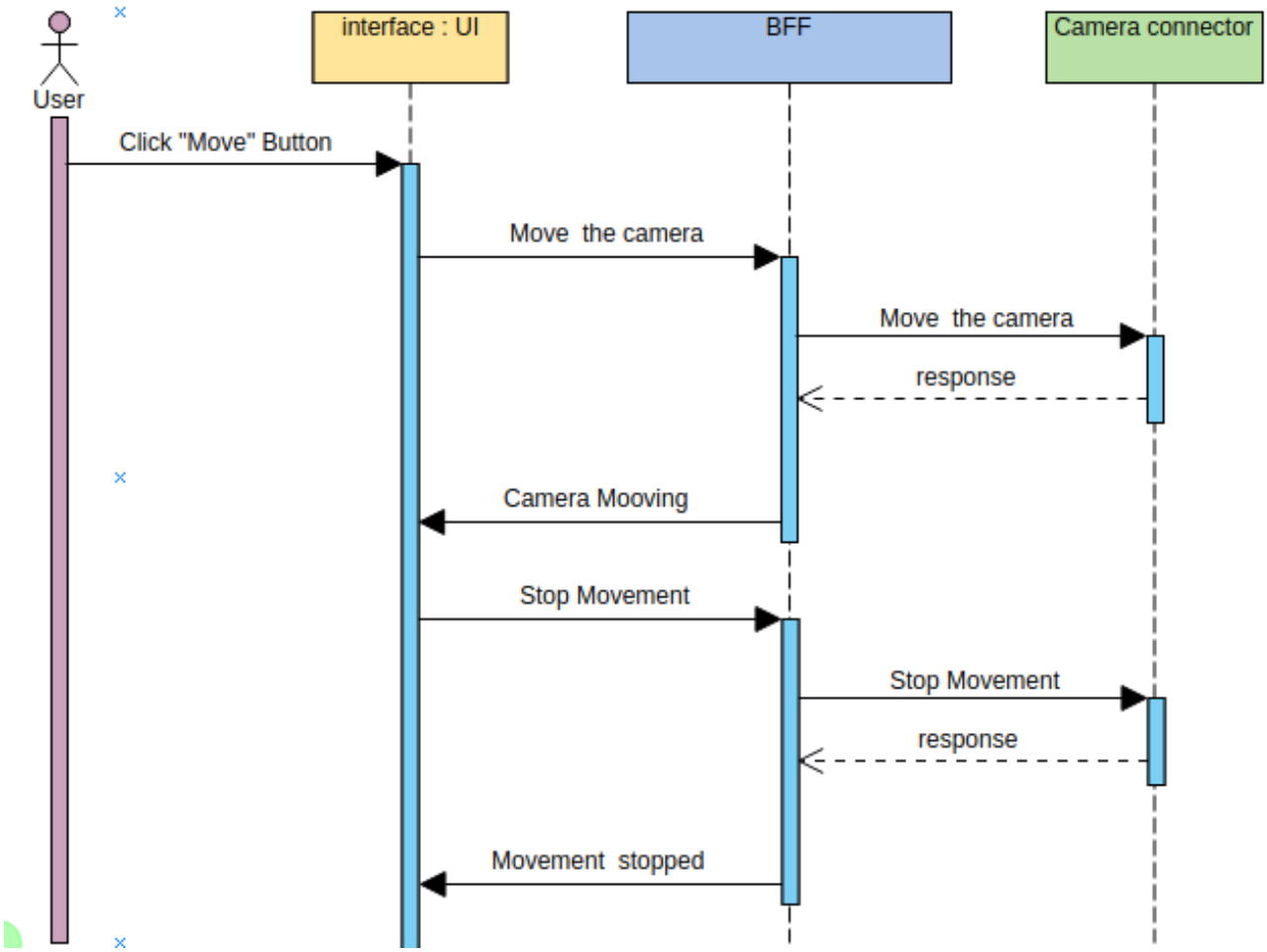


Figure 6.4: Diagramme de séquence Déplacer la caméra

6.3.2.2 Zoomer la caméra

La figure suivante présente le diagramme de séquence zoomer la caméra :

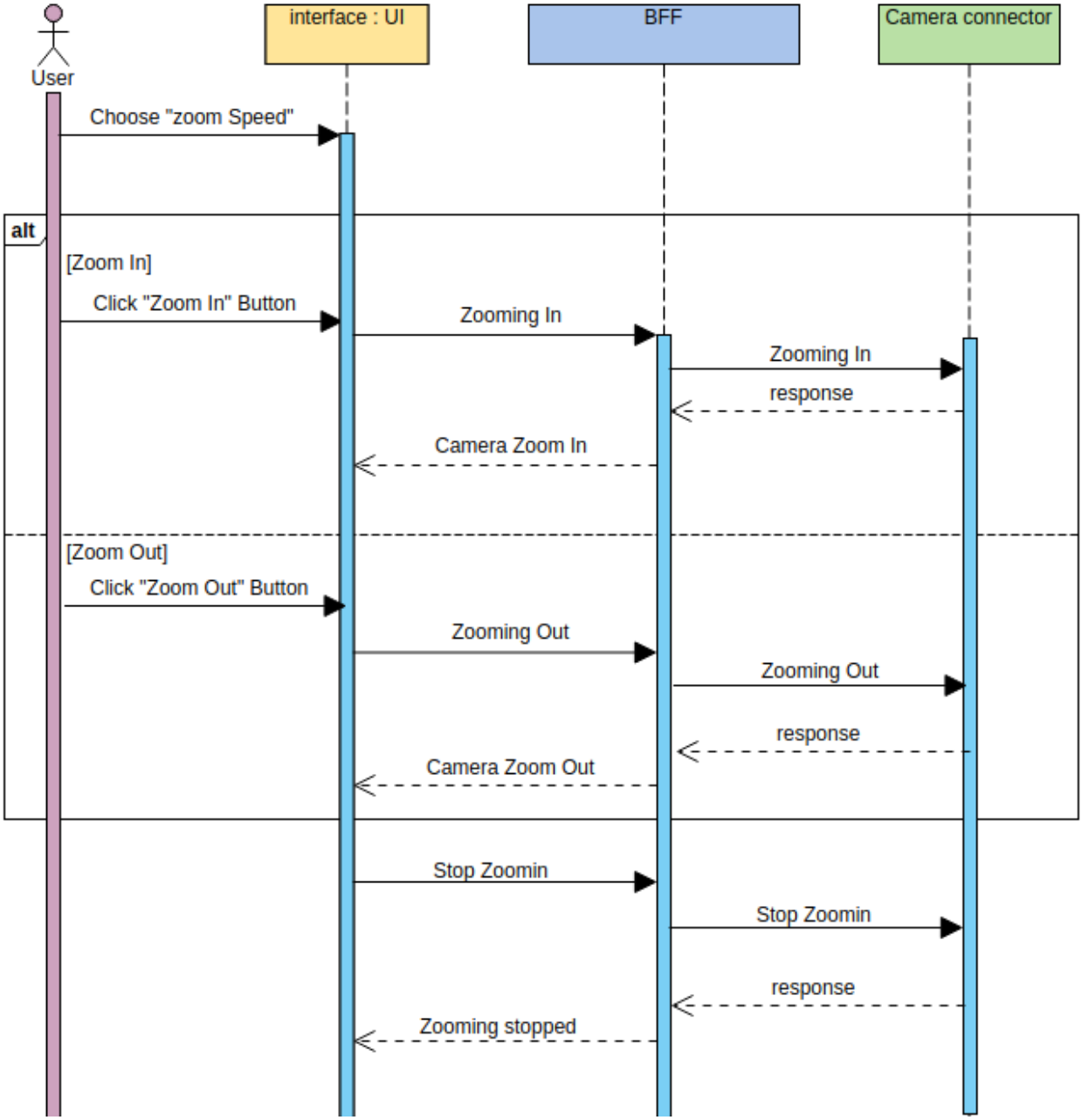


Figure 6.5: Diagramme de séquence Zoomer la caméra

6.3.2.3 Alimenter la caméra

La figure suivante présente le diagramme de séquence alimenter la caméra :

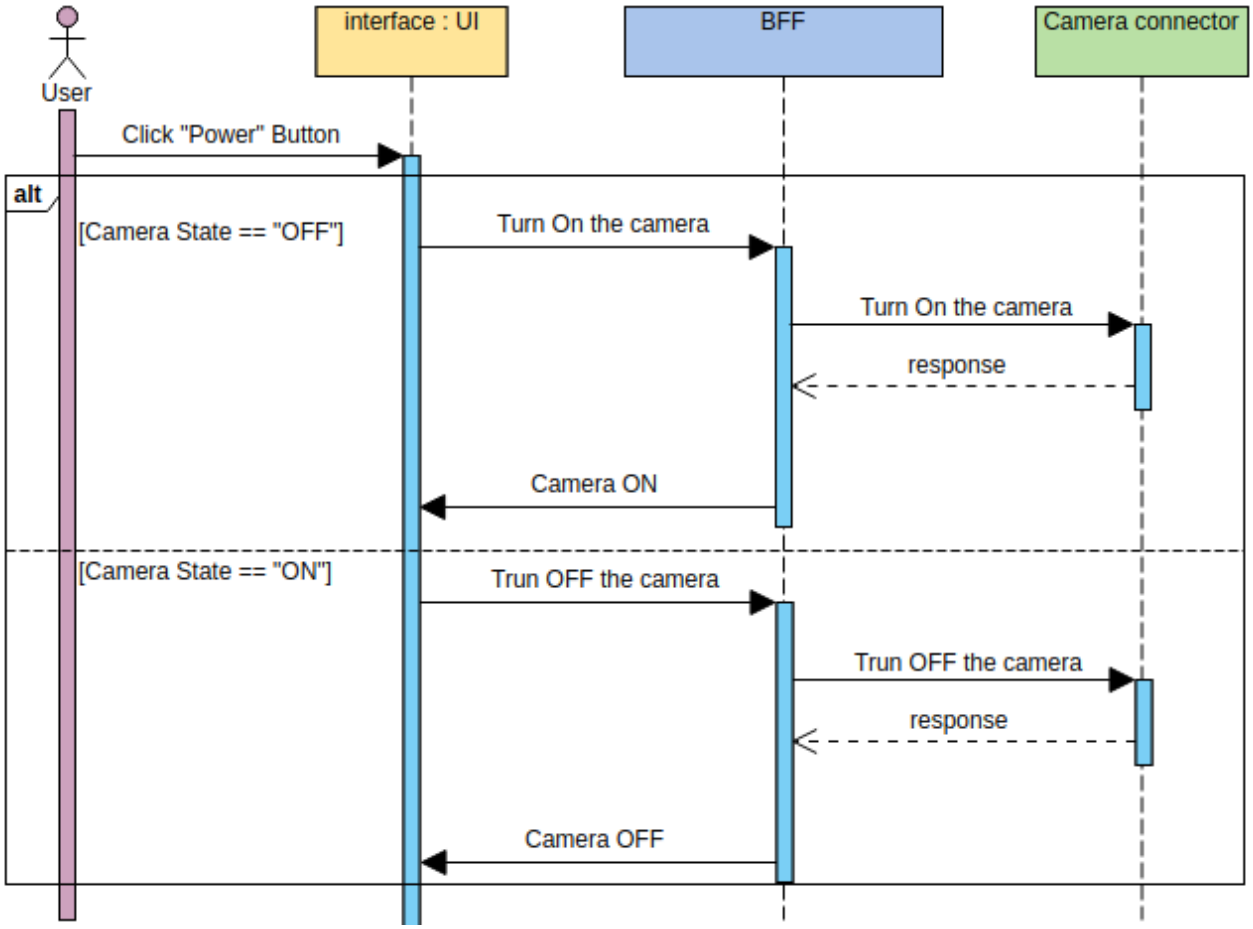


Figure 6.6: Diagramme de séquence Alimenter la caméra

6.3.3 Diagramme de classe

La figure suivante présente le diagramme de classe de sprint 4 :

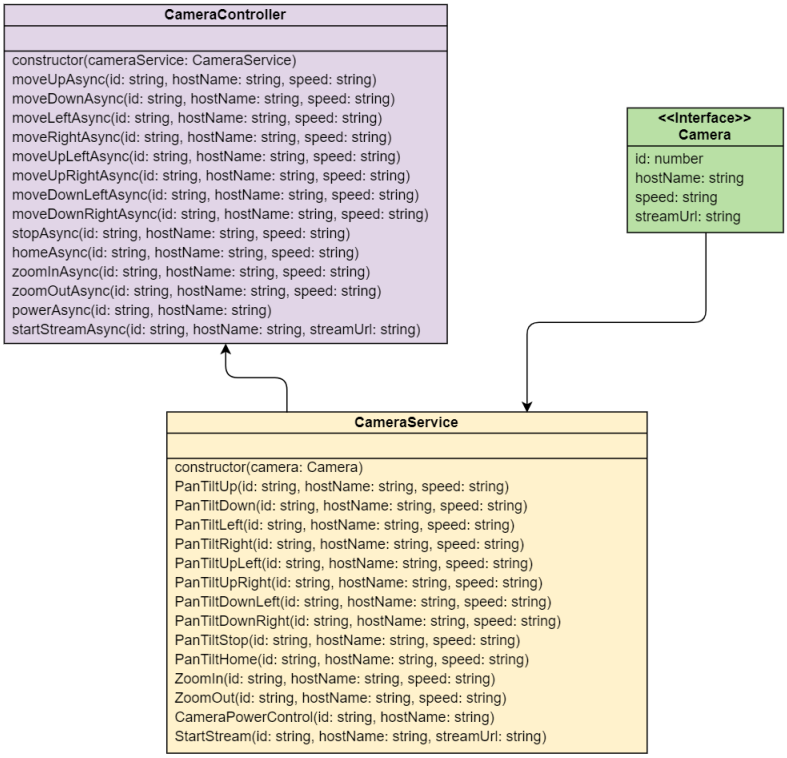


Figure 6.7: diagramme de classe de sprint 4

6.4 Réalisation

Dans cette section, nous allons présenter quelques interfaces qui concerne le Sprint 4.

Interface de la contrôle de la caméra

Cette interface permet à l'utilisateur du site de contrôler la caméra PTZ en mouvement horizontale, verticale ou diagonale ainsi de changer la vitesse de mouvement. Effectuer les zoom in et zoom out aussi avec un vitesse de zoom. Aussi d'enregistrer des presets qui permet d'enregistrer des emplacements et l'appeler après.

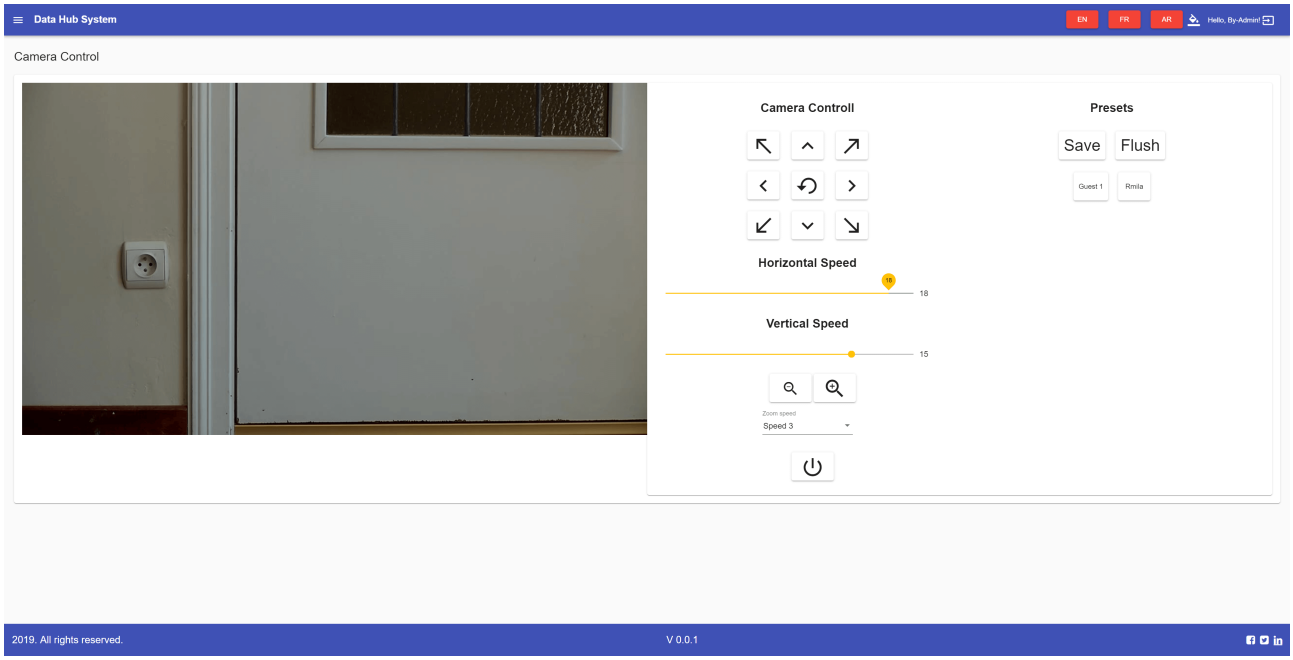


Figure 6.8: Interface la contrôle de la caméra

Interface du contrôle de la caméra intégré dans la diffusion en direct

Cette interface permet à l'utilisateur du site de contrôler la camera PTZ dans le studio de diffusion afin de simplifier la diffusion et le contrôle au même temps.

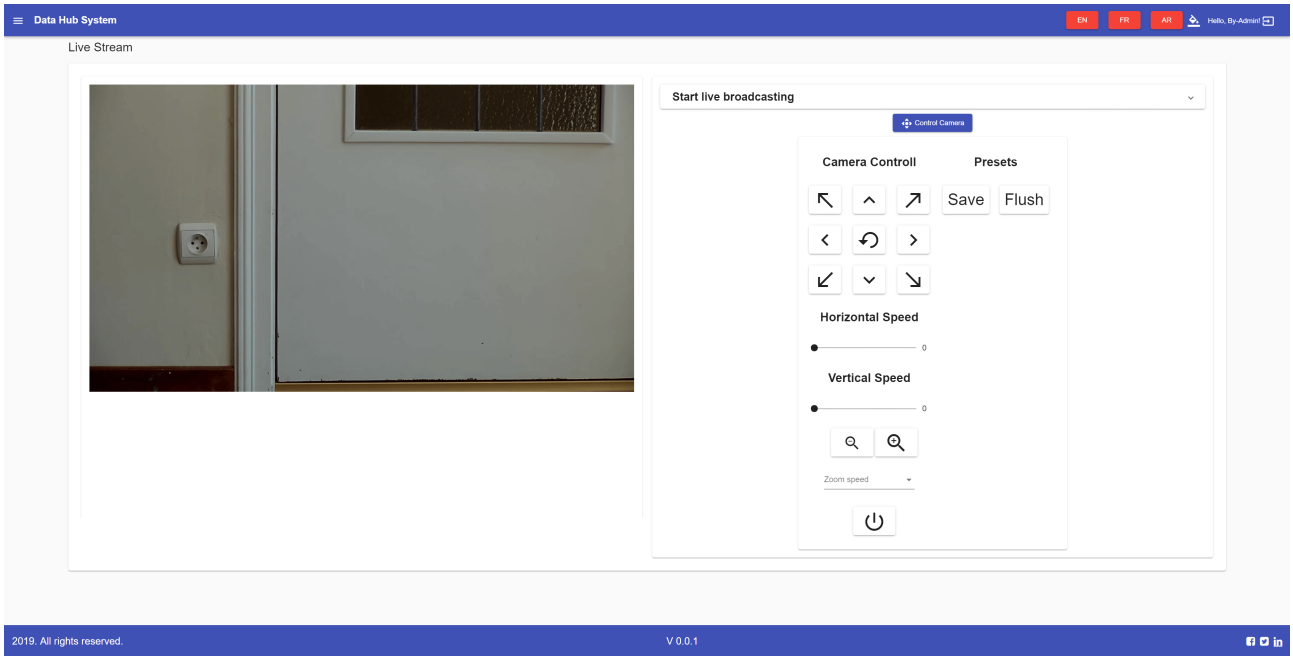


Figure 6.9: Interface le contrôle de la caméra intégré dans la diffusion en direct

Interface de gestion des caméras

Cette interface permet à l'utilisateur du site de gérer les caméras.

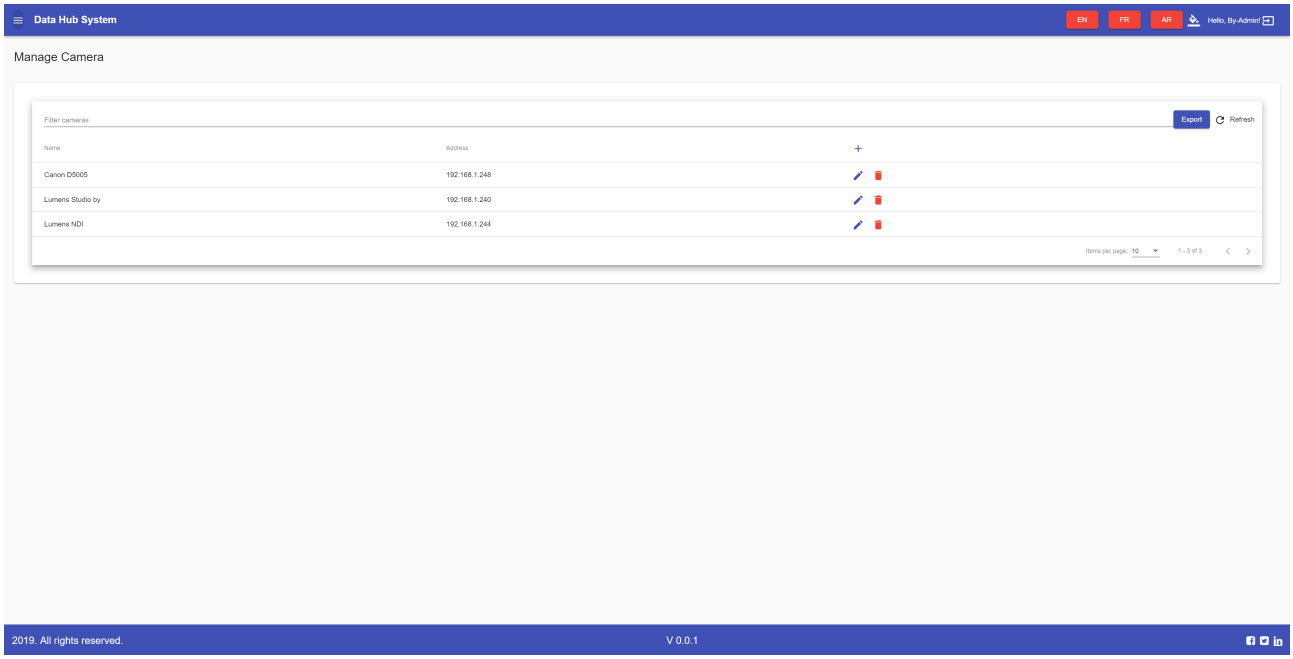


Figure 6.10: Interface gérer les caméras

Interface de gestion de modèle de données

Cette interface permet à l'administrateur le modèle de données de la caméra et voir les différents paramètre de la caméra.

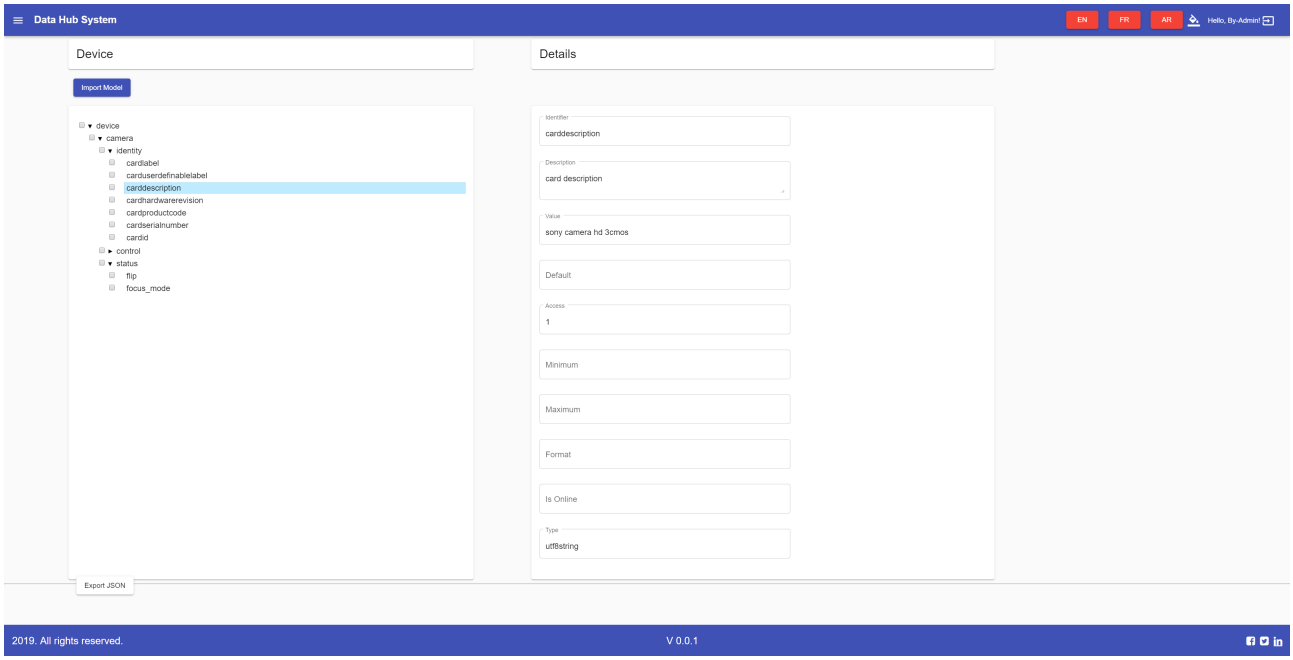


Figure 6.11: Interface gérer le modelé des données

Interface de gestion des vues clients

Cette interface permet à l'administrateur du site de gérer les vues client qui permettent de crée un sous modèle de donnée de la caméra en choisissant quelque paramètre et de donnée la possibilité au

client de le changer.

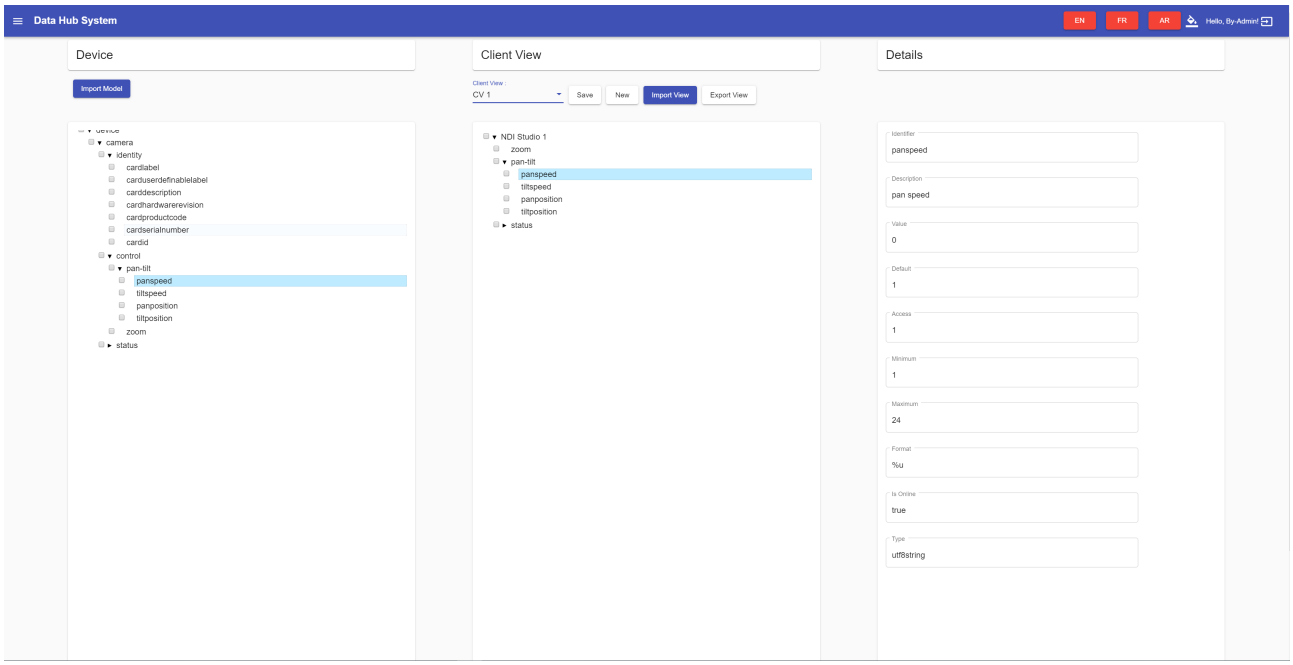


Figure 6.12: Interface gérer les vues clients

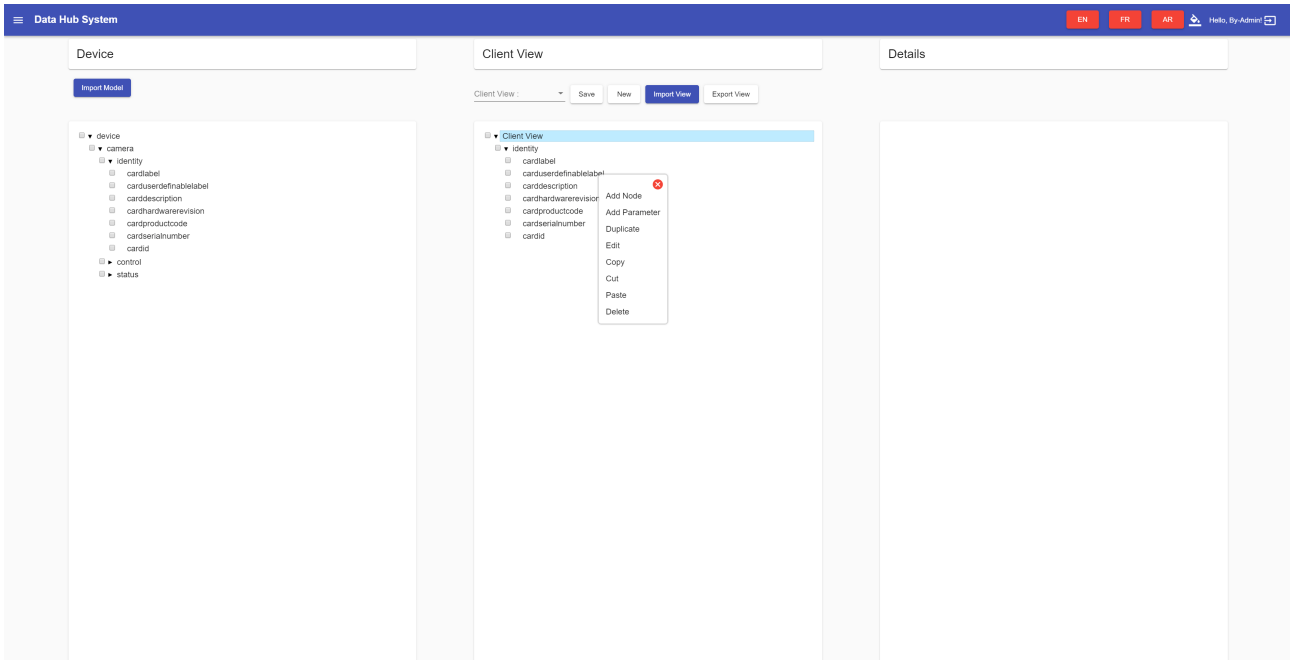


Figure 6.13: Interface gérer les vues clients

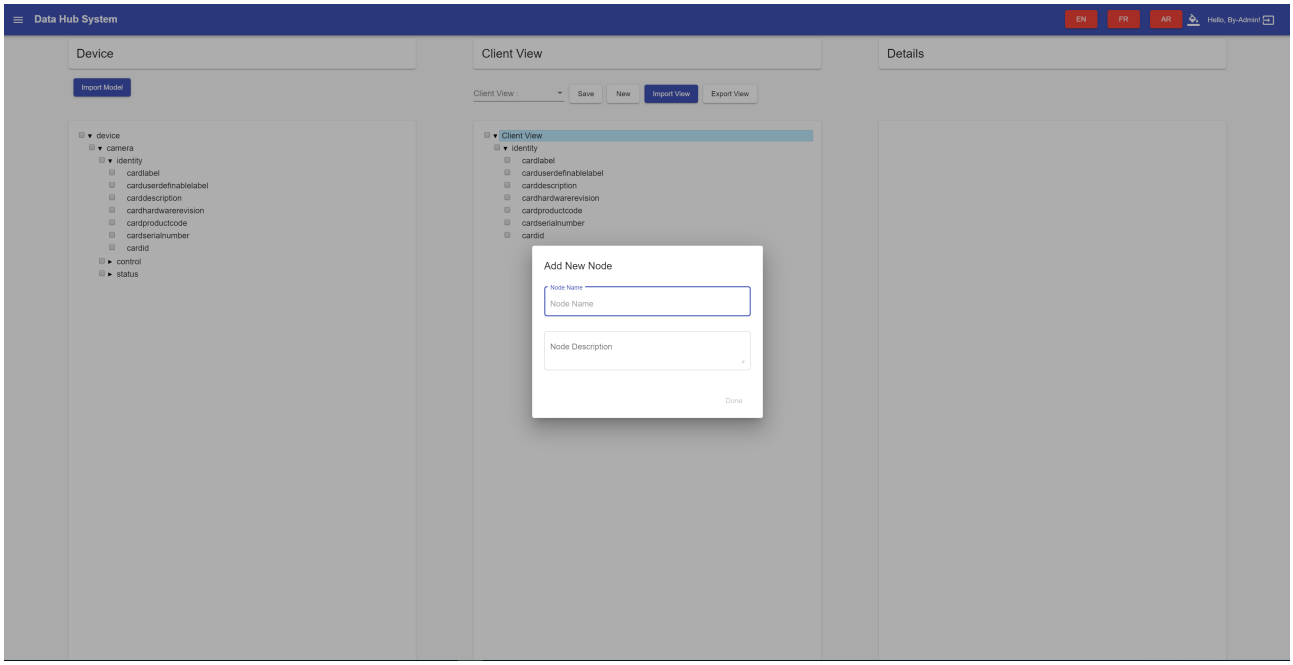


Figure 6.14: Interface ajouter noeud

Interface de gestion les processus sur la diffusion en direct

Cette interface permet à l'administrateur du site de gérer les processus sur la diffusion en direct par exemple (Conversion de stream a partir caméra de RTSP en RTMP puis l'envoyer sur un serveur de diffusion et puis de RTMP vers RTMPS vers Facebook).

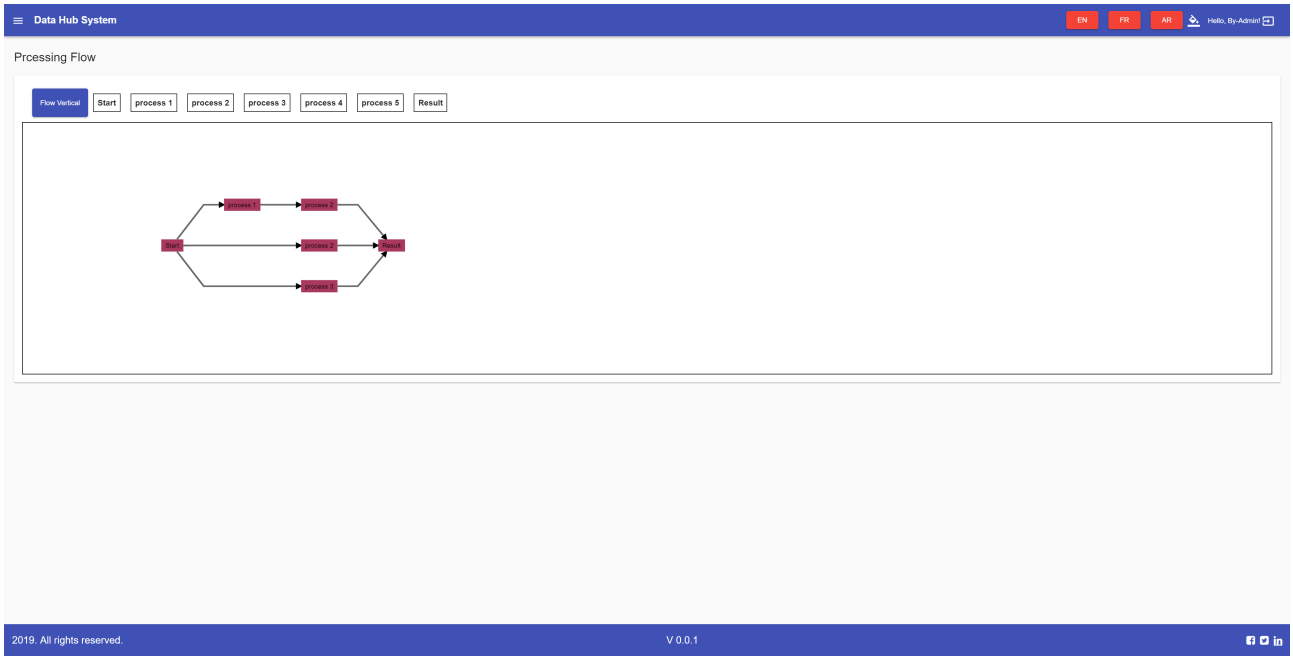


Figure 6.15: Interface gérer les flows

6.5 Conclusion

Dans ce chapitre nous avons fini le dernier sprint de notre projet, il est alors maintenant potentiellement livrable.

Chapitre 7

Déploiement

Plan

1 Virtualisation ou containérisation 81

2 L'outil Docker 83

La phase de déploiement est un étape indispensable dans le processus de développement d'une application. Comme exemple, on peut citer la Virtualisation et la containérisation.

7.1 Virtualisation ou containérisation

7.1.1 Virtualisation

La virtualisation sert à partitionner un seul serveur physique en plusieurs machines virtuelles comme un espace de stockage ou un réseau. Elle permet une consolidation de serveurs avec une grande souplesse d'utilisation. Dans le contexte de l'informatique en nuage, la virtualisation est importante pour la mise en service et le retrait rapide de serveurs. Par ailleurs, la scalabilité est la principale caractéristique du Cloud moderne. Ainsi la virtualisation a permis la création de machines virtuelles, la montée en charge à la demande, la migration à chaud, etc. La virtualisation est une technologie permettant d'arriver à une utilisation rentable des serveurs tout en prenant en charge la séparation entre de multiples locataires d'un matériel physique.

7.1.2 Containérisation

Le noyau Linux permet de lancer plusieurs instances isolées de l'espace utilisateur. Un conteneur est un environnement isolé où un ou plusieurs processus peuvent être exécutés. Les conteneurs se concentrent sur l'isolation des processus au lieu d'émuler une machine physique complète. Dans le noyau Linux a fourni un certain niveau d'isolation en fournissant un environnement pour créer et héberger une copie virtualisée d'un logiciel. Mais le terme «conteneurs» n'est apparu que vers la fin de l'année 2006. «Control Groups» limite et isole l'utilisation des ressources d'un ensemble de processus. Par la suite, l'isolation de l'espace de noms a été ajoutée. Cela a conduit à l'évolution de Linux Containers, un environnement de virtualisation au niveau du système d'exploitation qui est construit sur les fonctionnalités du noyau Linux comme chroot, cgroupes, l'isolation de l'espace de noms, etc.

7.1.3 Etude comparative

Le choix de la solution a été proposé fortement par la société. On a opté pour une solution de containérisation (Docker) dans la mesure où un Benchmark prouve son efficacité et sa légèreté.

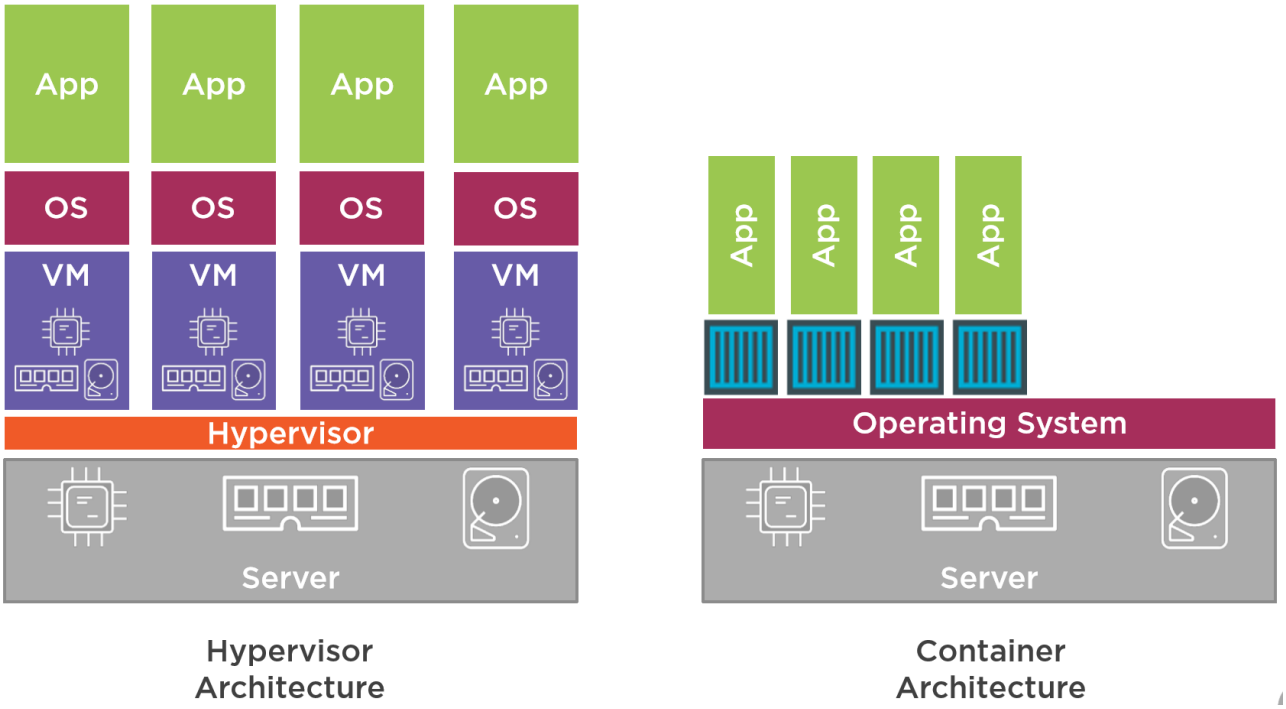


Figure 7.1: Docker VS Machine Virtuelle

Dans la containérisation, ou la virtualisation au niveau du système d'exploitation, l'hôte et les invités partagent le même noyau. Cette approche réduit le gaspillage de ressources puisque chaque conteneur contient uniquement l'application, les bibliothèques et les binaires nécessaires. Le rôle de l'hyperviseur est assuré par un moteur léger de la containérisation comme Docker, qui est installé au-dessus du système d'exploitation hôte. Le principal avantage des conteneurs c'est qu'ils sont libres de l'overhead du système d'exploitation contrairement aux machines virtuelles, ce qui leur rend considérablement plus légers, plus faciles à télécharger ainsi que plus rapides à lancer. Cet avantage permet également au serveur d'héberger potentiellement beaucoup plus de conteneurs que de machines virtuelles. Scalabilité, portabilité et facilité de déploiement sont quelques avantages. Lors de cette étude, nous allons se baser sur un travail réalisé par IBM[16] , où nous allons mettre l'accent sur les performances de Docker et KVM. Le Benchmark concerne 15 machines virtuelles simultanément en marche sous OpenStack.

Tests	Docker	KVM
Temps moyen de démarrage	3.9s	5.88s
Usage CPU démarrage (sys)	0.44%	2.08%
Usage CPU démarrage (usr)	1.14%	12.6%
Usage de mémoire au démarrage	45.8Mb/VM	185Mb/VM
Temps moyen de redémarrage	2.54s	124.45s
Usage CPU redémarrage (sys)	0.11%	0.19%
Usage CPU redémarrage (usr)	0.34%	0.82%
Usage de mémoire au redémarrage	57Mb	467Mb
Temps moyen de suppression	4.09s	4.45s
Temps moyen de Snapshot (VM to Image)	26.39s	42.93s
Usage CPU de Snapshot (sys)	0.11%	1.07%
Usage CPU de Snapshot (usr)	0.4%	1.58%
Usage de mémoire de Snapshot	48Mb	114Mb
Usage de mémoire état stable	3.52s	5.87s
Calcul des nombres premier jusqu'à 20000	15.11s	15.08s
Débit du réseaux (Mbits/s)	940.26	940.56

Figure 7.2: Performances : Docker VS KVM

7.2 L'outil Docker

Docker est une technologie qui simplifie et accélère le processus de déploiement de logiciels en créant un environnement de travail unifié et un emballage dans un conteneur à partir duquel les développeurs peuvent travailler sans perdre de temps. Il permet d'installer les logiciels, les dépendances,

l'environnement de configuration des bibliothèques. Et le conteneur prend aussi en charge le déploiement sur le serveur.

Au cours des tests des technologies, nous avons également passé une semaine et demi à mettre en place le docker et le docker-compose. Actuellement nous avons initialisé un conteneur pour déployer le frontend. Pour créer un environnement docker complet pour le backend, les microservices et la base de données, il faudra plus de temps.

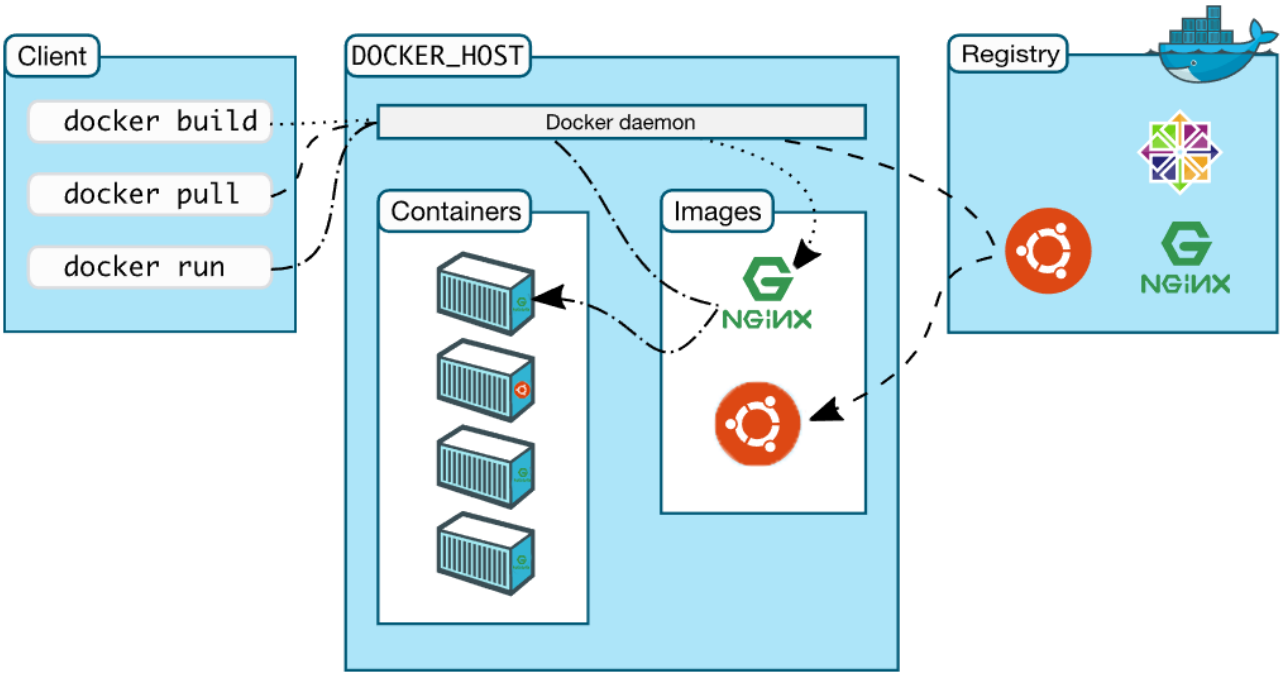


Figure 7.3: Architecture de déploiement Docker

Conclusion générale et perspectives

Le présent document est une présentation du travail réalisé durant notre stage de fin d'études au sein de l'entreprise By-Systems. Le projet a pour objectif du conception et mise en place d'une solution de diffusion pour By-Systems. Cet application Web pour la création et la diffusions de diffusion en direct, la contrôle des caméras et la gestion du modèles des données, c'est un solution multiplateforme et présente un niveau de portabilité, vu la facilité de son utilisation dans différents environnements de travail.

Riche en expériences et en rencontres , ces quatre mois viennent clore notre formation d'ingénieur à l'école. Ce travail a été pour nous, l'occasion de mettre en oeuvre et d'approfondir les connaissances techniques et le savoir académique acquis durant notre cursus à l'école centrale polytechnique privée de Tunis.

Ce stage a été une occasion pour mettre en œuvre nos compétences universitaires, professionnelles et humaines pour un sujet intéressant. C'est une aussi une opportunité pour découvrir le monde professionnel, ses réalités, du point de vue adaptation et difficultés.

Finalement, notre travail ne s'arrête pas à ce niveau, il serait possible d'ajouter certaines d'autres fonctionnalités complémentaires tels que :

- L'analyse d'images dans les flux RSS.
- Traitement des vidéos et les sons.
- Création des nouveau microservices pour des autres médias sociaux.

Webographie

[1] http ://www.aubryconseil.com/post/Le-backlog-de-produit (dernière consultation 20-05-2019)

[2] https ://www.linkedin.com/pulse/méthode-traditionnelle-vs-méthodes-agiles-nicolas-bouliteau-pmp (dernière consultation 20-05-2019)

[3] https ://martinfowler.com/microservices (dernière consultation 20-05-2019)

[4] https ://www.linkedin.com/company/by-systems (dernière consultation 20-05-2019)

[5] https ://www.couchbase.com/about (dernière consultation 20-05-2019)

[6] https ://code.visualstudio.com/docs (dernière consultation 20-05-2019)

[7] https ://material.angular.io/ (dernière consultation 20-05-2019)

[8] http ://www.ceanjs.org/ (dernière consultation 20-05-2019)

[9] https ://alligator.io/angular/angular-7/ (dernière consultation 20-05-2019)

[10] https ://bitbucket.org/product/features (dernière consultation 20-05-2019)

[11] https ://get.slack.help/hc/en-us/articles/115004071768-What-is-Slack- (dernière consultation 20-05-2019)

[12] https ://www.clearvision-cm.com/atlassian/sourcetree/ (dernière consultation 20-05-2019)

[13] https ://www.tutorialsteacher.com/mvc/mvc-architecture (dernière consultation 20-05-2019)

[14] https ://confluence.atlassian.com/confeval/confluence-evaluator-resources/confluence-what-is-confluence (dernière consultation 20-05-2019)

[15] https ://fr.atlassian.com/software/jira/agile (dernière consultation 20-05-2019)

[16] https ://fr.slidesharenet/BodenRussell/kvm-and-docker-lxc-benchmarking-with-openstack(dernière consultation 20-05-2019)

Résumé

Le secteur de la diffusion fait face à un labyrinthe de défis alors que l'ensemble du secteur des médias devient de plus en plus concurrentiel à l'ère numérique. En plus, le paradigme existant de différents produits basés sur des «appareils» provenant de nombreux fabricants crée des couches de complexité qui sont coûteuses à l'achat, difficiles à entretenir, difficiles à intégrer et qui ont besoin de personnel spécialisé. Notre projet entre dans ce cadre pour concevoir et mettre en place 'une solution de diffusion au profit de l'Entreprise By-Systems.

Mots clés : Microservices, Angular 7, Node. JS, Couche Base, Diffusion

Abstract

The broadcasting sector faces many challenges as the entire media are becoming more and more competitive in the digital age. In addition, the existing paradigm of different products based on "devices" from many manufacturers create layers of complexity that are expensive to purchase, difficult to maintain, difficult to integrate, and require specialized personnel.Our project falls within this framework to design and implement a broadcasting solution for the benefit of By-Systems.

Keywords : Microservices, Angular 7, Node.js, CouchBase, Broadcasting